

Projekt USB-Oszilloskop

Samuel Oeser, Nicole Sturm, Daniel Wirth

3. Oktober 2025

Inhaltsverzeichnis

1. Abstract / Zusammenfassung	5
2. Einleitung	6
3. Fachliche Grundlagen	7
3.1. Allgemeiner Aufbau eines DSOs	7
3.1.1. Trigger	9
3.1.2. Frequenzkompensierter Tastkopf / Spannungsteiler	9
3.2. AAF-Entwurf (Nyquisttheorem)	11
3.3. Analog-to-Digital-Converter (ADC)	13
3.3.1. Flash-ADC	14
3.3.2. Pipeline-ADC	14
3.3.3. Wichtige Kenngrößen von AD-Umsetzern	16
3.4. Endliche Zustandsautomaten (Finite State-Machines - FSMs)	18
3.4.1. Definition und formale Darstellung	18
3.4.2. Modellierung und Darstellung	19
3.4.3. Vorteile einer FSM bei der Programmierung	20
3.5. Direct Memory Access - DMA	21
3.5.1. Allgemeines	21
3.5.2. DMA bei STM32-Mikrocontrollern	21
3.6. Digitale Filterung (Preprocessing)	26
3.6.1. Motivation und Grundlagen digitaler Filterung	26
3.6.2. Filterklassifikation	26
3.6.3. Gleitender Mittelwertfilter	28
3.7. Parallele Prozesse und Nebenläufigkeiten in Python	29
3.7.1. Threads in Python	29
3.7.2. Multiprocessing in Python	29
4. Projektkonzeption	31
4.1. Vorgehensweise	31
4.2. Anforderungen	33
4.3. Konzept / Architektur	34

5. Hardware (HW)	36
5.1. Entwurf	36
5.2. Implementierung	37
5.3. HW-Test	37
6. Schnittstelle Hardware - Firmware	38
6.1. Auswahl des Komparators	38
7. Firmware (FW)	39
7.1. Entwurf	39
7.1.1. Auswahl des Mikrocontrollers	39
7.1.2. Konzept des Abtastprozesses	41
7.1.3. Konzept der Firmware als FSM	45
7.1.4. Preprocessing-Konzept	48
7.2. Implementierung	51
7.2.1. Allgemeines zur verwendeten Toolchain	51
7.2.2. Übersicht über die Applikations-Sourcefiles	53
7.2.3. FSM	55
7.2.4. Preprocessing	56
7.3. FW-Test	57
7.3.1. Abtastung	57
7.3.2. FSM	58
7.3.3. Gleitender Mittelwertfilter	60
8. Schnittstelle Firmware - Software	61
8.1. USB Communication Device Class (CDC)	61
8.1.1. Grundlagen zu USB und den verwendeten Schnittstellen	61
8.1.2. Auswahl und Begründung für USB CDC	62
8.1.3. Kommunikationssituation PC ↔ Mikrocontroller	63
8.2. Konzept der Kommunikation	64
9. Software (SW)	65
9.1. Entwurf	65
9.1.1. Auswahl des Frameworks	65

9.1.2. Architektur Gesamtübersicht	65
9.1.3. Konzept Hardwareinterface	66
9.1.4. Konzept Benutzeroberfläche	67
9.1.5. Auswahl und Definition der Messungen	69
9.2. Implementierung	70
9.3. SW-Test	71
10. Zusammenführung	72
10.1. Testaufbau	72
11. Ergebnisse	82
11.1. Vergleich mit ursprünglichen Anforderungen	82
11.2. Benutzung des USB-Oszilloskops	83
11.2.1. Hauptfunktionen	83
11.2.2. Bedienelemente und Einstellmöglichkeiten	84
11.2.3. Besonderheiten und Bedienhinweise	85
12. Fazit und Ausblick	86
13. Literaturverzeichnis	87
14. Abbildungsverzeichnis	90
A. Anhang	93
A.1. Blockschaltbilder	93

1. Abstract / Zusammenfassung

2. Einleitung

Die vorliegende Projektarbeit wurde im Rahmen des Bachelorstudiengangs Elektrotechnik und Informationstechnik an der Fakultät für Elektrotechnik, Feinwerktechnik und Informationstechnik (EFI) der Technischen Hochschule Nürnberg Georg Simon Ohm durchgeführt. Ziel des Projekt-Moduls ist es, den Studierenden die Möglichkeit zu geben, ihr theoretisch erworbenes Wissen in einem praxisnahen, ingenieurwissenschaftlich strukturierten Entwicklungsprojekt anzuwenden. Die Motivation für die Auswahl des Projektthemas lag in der Abbildung des vollständigen Entwicklungsprozesses eines aus Hardware, Firmware und Software bestehenden Gesamtsystems. Auf diese Weise konnten praxisnahe Erfahrungen in allen wesentlichen Entwicklungsdisziplinen gesammelt werden. Darüber hinaus bot das Projekt die Gelegenheit, die grundlegenden Funktionen eines Oszilloskops zu verstehen und im Rahmen eines Prototyps zu realisieren. Ein weiteres wesentliches Auswahlkriterium für das Thema war die klare Unterteilung in abgegrenzte Aufgabenbereiche, sodass die Teammitglieder ihre Aufgaben eigenständig bearbeiten konnten, während gleichzeitig eine Zusammenarbeit im übergeordneten Kontext möglich war.

Das zu Beginn definierte Ziel des Projekts war die Entwicklung eines USB-Oszilloskops. Das Gesamtsystem, bestehend aus selbstentwickelter Hardware und einem über Universal Serial Bus (USB) angeschlossenen Computer, soll die Grundfunktionen eines digitalen Speicheroszilloskops (DSO) abbilden. Die Realisierung sollte durch den Einsatz eines Analog-Digital-Umsetzers (ADC) zur Messwerterfassung sowie eines Mikrocontrollers (μC) als Schnittstelle zwischen der Hardware (ADC-Schaltung) und der Software (Computer) erfolgen.

Das Projektteam setzte sich aus drei Studierenden zusammen: Samuel Oeser war verantwortlich für die Entwicklung der Hardware (HW), Daniel Wirth übernahm die Firmware (FW), während Nicole Sturm die Software (SW) einschließlich der grafischen Benutzeroberfläche (GUI) entwickelte. Die Betreuung des Projekts erfolgte durch Prof. Dr. Sven Loquai, der die Studierenden während des gesamten Entwicklungsprozesses begleitete.

3. Fachliche Grundlagen

3.1. Allgemeiner Aufbau eines DSOs

Das DSO erfasst Eingangssignale, digitalisiert sie und stellt die Messdaten nach Speicherung und Verarbeitung dar. Der grundlegende Aufbau umfasst die Eingangsumschaltung (DC AC GND) mit Vertikalverstärkung/abschwächung, den ADU mit Signalvorverarbeitung, den Datenspeicher, Takt und Steuerung, die Triggereinrichtung sowie die Anzeige. Die digitalisierten Daten werden im Speicher abgelegt und für Darstellung und Auswertung ausgelesen (vgl. [Müh20, S. 216]).

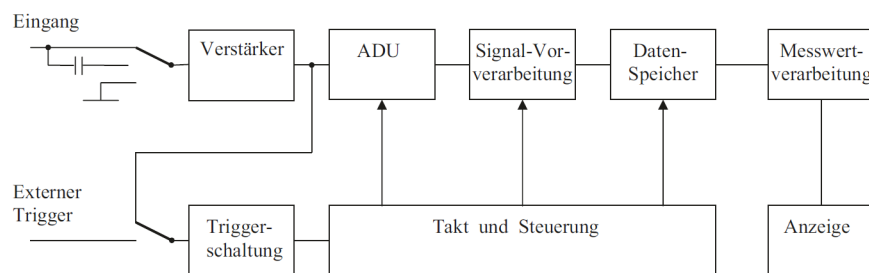


Abbildung 1: Blockschaltbild eines Digitaloszilloskops (Abb. 14.1 aus [Müh20, S. 216])

Für die Abtastung sind zwei Betriebsarten relevant. Die Echtzeitabtastung erfasst den kompletten Verlauf in einem Durchlauf und die zeitliche Auflösung wird durch die Rate des AD-Umsetzers (ADU) begrenzt. Die Äquivalenzzeitabtastung rekonstruiert sehr schnelle periodische Verläufe aus mehreren Durchläufen (vgl. [Müh20, S. 216]).

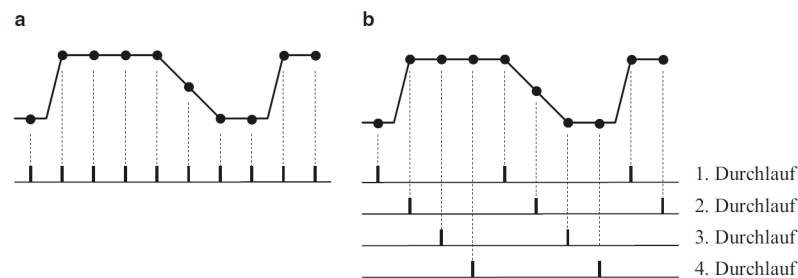


Abbildung 2: Gegenüberstellung Echtzeitabtastung (a) und Äquivalenzzeitabtastung (b) (Abb. 14.2 aus [Müh20, S. 216])

Für eine gut auswertbare Darstellung sind etwa zehn Stützstellen je Periode zweckmäßig und die si-Interpolation (auch sinc-Interpolation genannt) ermöglicht bei erfüllter Nyquist

Bedingung eine nahezu verzerrungsfreie Rekonstruktion. In der Praxis sollte die maximale Signalfrequenz mindestens um den Faktor 2,5 unter der Abtastrate liegen (vgl. [Müh20, S. 218f]).

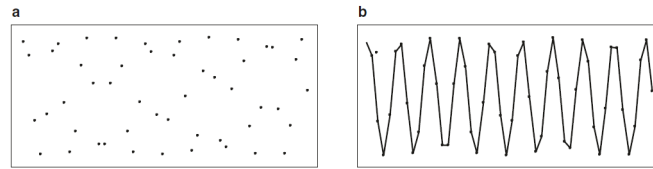


Abbildung 3: Punktedarstellung (a) und lineare Interpolation (b)
(Abb. 14.4 aus [Müh20, S. 218])

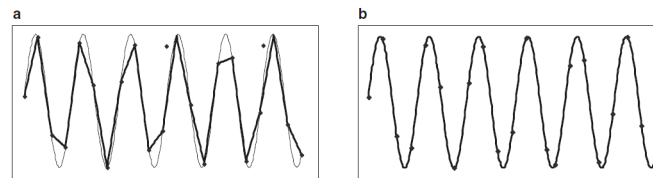


Abbildung 4: Lineare Interpolation (a) und si-Interpolation (b)
(Abb. 14.5 aus [Müh20, S. 219])

In der Praxis werden für die Digitalisierung schnelle Flash-ADUs sowie mehrstufige Sub-ranging- / Pipeline-Varianten eingesetzt. Diese ermöglichen hohe Raten bei moderater vertikaler Auflösung. Übliche Geräte arbeiten mit acht Bit vertikaler Auflösung. Je nach Architektur sind höhere Auflösungen möglich (vgl. [Ber23, S. 114-119]). Für die Datenaufnahme stehen verschiedene Acquisition-Modi zur Verfügung. Der direkte Modus liefert äquidistante Samples. Min-Max oder Peak-Detect erfasst Extremwerte innerhalb eines Abtastintervalls und macht kurze Spikes sichtbar. Average reduziert Rauschen durch wiederholtes Mitteln. Single Shot zeichnet einmalige Ereignisse auf und der Roll-Modus zeigt sehr langsame Verläufe kontinuierlich an (vgl. [Müh20, S. 220f]). Die Genauigkeit wird wesentlich durch die Abtastrate und Abtaststrategie, Interpolation und Quantisierung sowie durch die Eingangsbeschaltung bestimmt. Bei Unterabtastung droht Aliasing und die Rekonstruktion wird unzuverlässig (vgl. [Müh20, S. ??], Grundlagen zur Abtastung und Quantisierung). Wichtige Kenngrößen von DSOs sind analoge Bandbreite, Abtastrate, vertikale Auflösung, Speichertiefe, Triggerfunktionen, Eingangsimpedanz, analoge Eingangsspannung sowie Anzeige und Darstellungsoptionen. Über die Kurvendarstellung hinaus gehören auch automatische Messfunktionen zum Grundumfang.

3.1.1. Trigger

Die Triggerung steuert die Signalspeicherung. Fortlaufend erfasste Abtastwerte laufen in einen Ringspeicher und beim Triggerereignis wird das Überschreiben gestoppt, sodass Pre Trigger und Post Trigger gezielt darstellbar sind (vgl. [Müh20, S. 216f]).

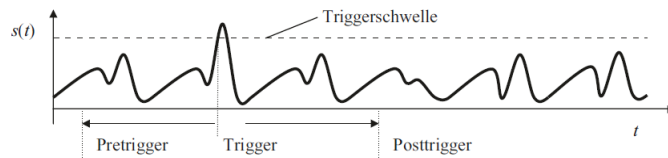


Abbildung 5: Messsignal $s(t)$ mit eingestellter Triggerschwelle und Triggerzeitpunkt (Teilabbildung der Abb. 14.3 aus [Müh20, S. 217])

Die analoge Triggerkette umfasst Quelle, Kopplung DC oder AC, einstellbare Triggerschwelle, Flankenrichtung, Hysterese und Auto Trigger. Sie erzeugt ein robustes Trigger-signal für die Speichersteuerung (vgl. [Müh20, S. 210-212]).

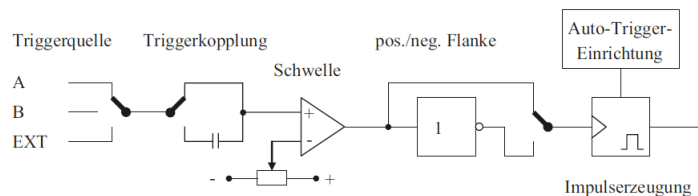


Abbildung 6: Funktionsblöcke einer Triggereinrichtung (Abb. 13.8 aus [Müh20, S. 212])

3.1.2. Frequenzkompensierter Tastkopf / Spannungsteiler

In der Praxis existieren verschiedene Arten von Tastköpfen wie passive, aktive und differenzielle Tastköpfe. Im Folgenden wird ausschließlich der frequenzkompensierte passive Tastkopf betrachtet, da er für das Projekt relevant ist (vgl. [Müh20, Kap. 15]). Dieser Tastkopf basiert auf einem frequenzkompensierten Spannungsteiler (siehe Abbildung 7), realisiert als RC Teiler mit zu den Widerständen parallel geschalteten Kapazitäten und mit einer Anpassung an die Summe aus Bauteil- und Leitungskapazitäten. Die Kompensation mit Trimmkondensator (in Abbildung 7: C_T) wird so abgeglichen, dass das Teilungsverhältnis im Nutz-Frequenzband annähernd konstant bleibt. Der Abgleich erfolgt mit einem Rechtecksignal. Unterkompensation zeigt sich durch eine nach unten geneigte

Oberkante (Abbildung 8a), Überkompensation durch eine nach oben geneigte Oberkante (Abbildung 8c) und bei korrekter Kompensation bleibt die Oberkante über die Pulsbreite hinweg horizontal (Abbildung 8b) (vgl. [SRZ22, S. 104-106]).

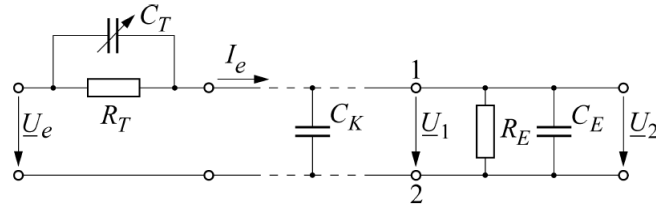


Abbildung 7: Tastteiler am Eingang eines Oszilloskops (Bild 2.42 aus [SRZ22, S. 105])

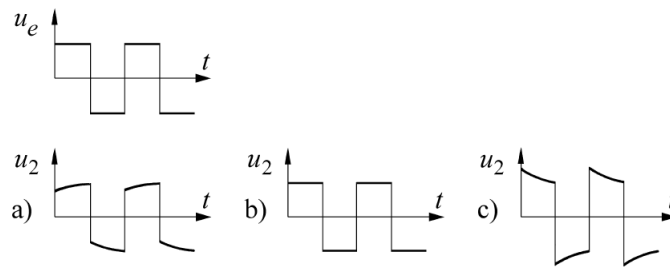


Abbildung 8: Rechteckimpulse an RC-Spannungsteiler (Bild 2.43 aus [SRZ22, S. 106])
(a) unterkompensiert, (b) richtig kompensiert, (c) überkompensiert

Mit $R_1 = R_T$ und $C_1 = C_T$ gilt für die obere Teilimpedanz ($\underline{Z}_1$):

$$\underline{Z}_1 = R_1 \parallel \frac{1}{j\omega C_1} = \frac{R_1}{1 + j\omega R_1 C_1} \quad (1)$$

und mit $R_2 = R_E$ und $C_2 = C_K + C_E$ gilt für die untere Teilimpedanz ($\underline{Z}_2$):

$$\underline{Z}_2 = R_2 \parallel \frac{1}{j\omega C_2} = \frac{R_2}{1 + j\omega R_2 C_2} \quad (2)$$

Über den Spannungsteiler ergibt sich aus (1) und (2) das Verhältnis:

$$\frac{\underline{U}_1}{\underline{U}_2} = \frac{\underline{Z}_1 + \underline{Z}_2}{\underline{Z}_2} = 1 + \frac{\underline{Z}_1}{\underline{Z}_2} = 1 + \frac{R_1}{R_2} \frac{1 + j\omega R_2 C_2}{1 + j\omega R_1 C_1} \quad (3)$$

Dieses Verhältnis nimmt den frequenzunabhängigen Wert $\frac{U_1}{U_2} = 1 + \frac{R_1}{R_2}$ an, wenn gilt:

$$R_1 \cdot C_1 = R_2 \cdot C_2 \quad (4)$$

Durch Einsetzen der Einzelbauelemente für R_1 , R_2 , C_1 und C_2 und Auflösen nach C_T ergibt sich für den Trimmkondensator ein Wert nach Gleichung 5, damit der Spannungsteiler frequenzunabhängig wird (vgl. [SRZ22, S. 104-106]).

$$C_T = \frac{R_E \cdot (C_K + C_E)}{R_T} \quad (5)$$

3.2. AAF-Entwurf (Nyquisttheorem)

Ein Digitaloszilloskop erfasst das analoge Eingangssignal durch Abtastung mit einer bestimmten Frequenz. Diese Abtastung entspricht der Multiplikation des Signals mit einer Reihe von Impulsen, was im Frequenzbereich einer Faltung entspricht. Eine eindeutige Rekonstruktion aus den Abtastwerten setzt ein bandbegrenzte Eingangssignal mit höchster relevanter Frequenz und eine Abtastrate oberhalb des Doppelten dieser Frequenz voraus. Bei Unterschreitung falten sich Spektralkopien in das Basisband zurück und erzeugen Aliasse. Dies wird im Frequenzbild durch die Überlappung periodischer Spektren sichtbar in Abbildung 10 sowie im Zeitbereich durch scheinbar niedrigere Schwingungen in Abbildung 9. Aus dieser Grundlage folgt die Notwendigkeit eines vorgeschalteten Anti-Aliasing-Tiefpasses, der außerbandige Anteile vor der Wandlung ausreichend dämpft (vgl. [Ber23, S. 314-316] und [Ber24, S. 151-156]).

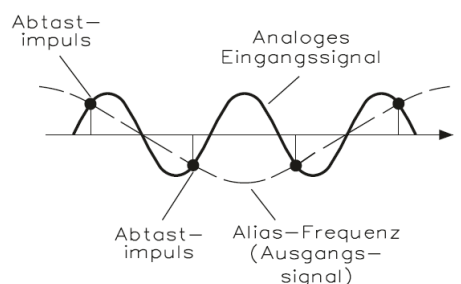


Abbildung 9: Aliasing: Erzeugung einer Scheinfrequenz durch unpassende Abtastrate (Bild 2.71 aus [Ber24, S. 154])

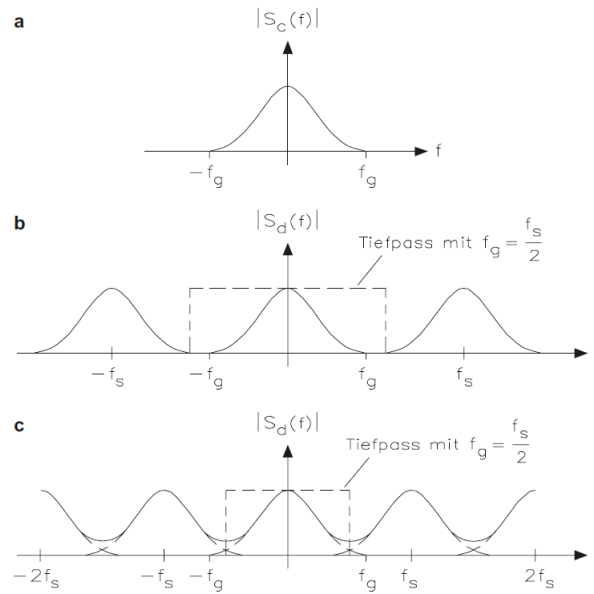


Abbildung 10: Abtasttheorem (Bild 4.30 aus [Ber23, S. 314])

- (a) Betragsspektrum des bandbegrenzten Signals
- (b) Vervielfachung des Basisspektrums durch Abtastung mit $f_s > 2 \cdot f_g$
- (c) Aliasing infolge zu niedriger Abtastrate ($f_s < 2 \cdot f_g$)

Da der Entwurf des Anti-Aliasing-Filters maßgeblich durch den ausgewählten AD-Umsetzer bestimmt wird, sollte dieser zuerst festgelegt werden. Für aktive RC-Filter bewährt sich wegen Bauteiltoleranzen und des nicht idealen Übergangs vom Durchlass- in den Sperrbereich eine Abtastrate oberhalb der Mindestbedingung, typischerweise etwa 2.5 bis 4 mal f_{max} und bei strengen Dämpfungszielen, z.B. wegen höheren Auflösungen, auch 5 bis 10 mal f_{max} . Eine höhere Abtastrate erweitert zudem die Übergangsbreite und senkt damit die notwendige Filterordnung, erleichtert die direkte Darstellung des abgetasteten Signals ohne Rekonstruktionsalgorithmen und erlaubt eine Filterfamilie mit geringerer Gruppenlaufzeitvariation und damit besserem Zeitverhalten. Aus der gewählten Abtastrate folgt dann die Nyquist-Frequenz $f_N = \frac{f_s}{2}$ und die Übergangsbreite $\Delta f = f_N - f_{max}$ [Pin20].

Der theoretische, aus der Auflösung abgeleitete Dynamikbereich setzt eine Obergrenze für die sinnvoll anzustrebende Sperrbanddämpfung. Für einen N -Bit-Umsetzer mit Full-Scale-Range FSR gilt die Codebreite $Q = \frac{FSR}{2^N}$. Ein voll ausgesteuertes Störsignal soll auf einen Pegel unterhalb der Codebreite gedämpft werden, damit keine signifikanten Quantisierungsartefakte entstehen. Die Sperrbanddämpfung, beginnend an der Nyquist-Frequenz,

wird daher so festgelegt, dass aliasingverursachende Anteile unterhalb der Codebreite bleiben:

$$A(f_N)_{max} = 20 \cdot \log_{10} \left(\frac{FSR}{Q} \right) = 20 \cdot \log_{10}(2^N) = 6.02 \cdot N \text{ dB} \quad (6)$$

In der Praxis begrenzt das *SINAD* häufig den nutzbaren Dynamikbereich des gewählten AD-Umsetzers. Gilt $|SINAD| < |A_{max}|$ oder liegt die daraus ermittelte *ENOB* (siehe Unterunterabschnitt 3.3.3 “Wichtige Kenngrößen von AD-Umsetzern“) unter der Auflösung des ADCs, dominieren Rauschen- und Verzerrungen. Bei Einsatz einer Mittelwertbildung kann es dennoch zweckmäßig sein, die Obergrenze A_{max} anzusetzen. Im Regelfall wird die Sperrbanddämpfung an der Nyquistfrequenz so gewählt, dass gilt (vgl. [Pin20]):

$$A(f_N) \approx \min(|A_{max}|, |SINAD_{ADC}|) \quad (7)$$

Die Bewertung des entworfenen Filters erfolgt in einem Tool wie dem Analog Filter Wizard über die Diagramme von Amplitudengang, Phasen- und Gruppenlaufzeit sowie Sprung- und Impulsantwort. Der Frequenzgang belegt Passband-Flachheit, Übergangsbreite und die Dämpfung an f_N . Phase und Gruppenlaufzeit zeigen, ob das Filter im Nutzband annähernd eine konstante Verzögerung liefert und damit Impulse und Flanken formtreu bleiben. Die Sprungantwort offenbart Überschwinger und Einschwingzeit. Eine Toleranzanalyse der zu erwartenden Widerstands- und Kapazitätsstreuungen zeigt, ob die Reserven ausreichend dimensioniert wurden (vgl. [Pin20] und [Pad17]).

3.3. Analog-to-Digital-Converter (ADC)

Ein Digitaloszilloskop benötigt einen AD-Umsetzer als zentrales Hardwareelement, da erst die Wandlung des analogen Eingangssignals in digitale Messwerte die weitere Speicherung, Verarbeitung und Anzeige ermöglicht. Für die Auswahl steht die Topologie am Anfang, weil sie Abtastrate, erreichbare Auflösung, Latenz, Leistungsaufnahme und den Implementierungsaufwand bestimmt.

Für ein USB-Oszilloskop bieten sich Flash- und Pipeline-Topologien an, da beide hohe Abtastraten für breitbandige Zeitbereichsmessungen bereitstellen. Andere Varianten wie SAR oder Delta-Sigma werden in den Quellen überwiegend mittleren Geschwindigkei-

ten oder schmalbandigen Präzisionsaufgaben zugeordnet und sind somit für ein USB-Oszilloskop irrelevant (vgl. [MPS] und [Ana01a]).

3.3.1. Flash-ADC

Abbildung 11 zeigt das Prinzip eines Flash-AD-Umsetzers mit zwei Bit. Eine Widerstandsleiter erzeugt abgestufte Referenzen und eine Komparatorbank vergleicht das Eingangssignal mit diesen Schwellen. Das resultierende Muster wird durch eine Logikschaltung in einen Binärcode überführt. Flash-ADCs erreichen, durch diesen simplen Aufbau, die höchste Geschwindigkeit. Da die Zahl der Komparatoren jedoch $2^N - 1$ beträgt, steigen bei hohen Auflösungen Komplexität, Fläche, Leistungsaufnahme sowie insbesondere die Herstellungskosten annähernd exponentiell an (vgl. [MPS] und [Ana01c]).

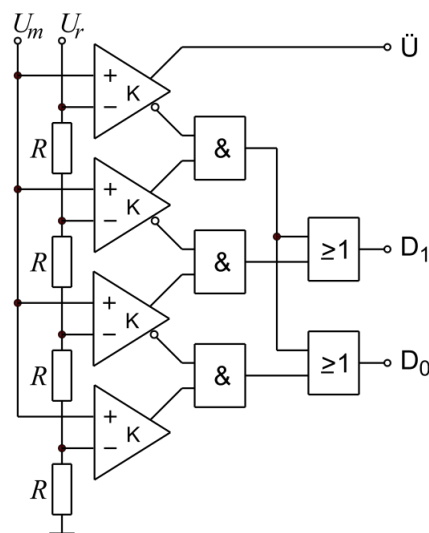


Abbildung 11: Flash-Umsetzer mit 2 Bit, einschl. Kodeumsetzer
Ersteller: Saure, Quelle: [\[Link\]](#) (Lizenz: [CC BY-SA 3.0](#))

3.3.2. Pipeline-ADC

Eine Alternative zur geringeren Skalierbarkeit des Flash-ADCs ist der Pipeline-ADC, der auch im Projekt zur Anwendung kommt. Er baut funktional auf dem Flash-Prinzip auf, indem mehrere niedrigauflösende Flash-ADCs in Stufen hintereinandergeschaltet werden. Am Eingang tastet eine Sample-and-Hold-Stufe das Signal ab. Jede Stufe wandelt in we-

nige Bits, rekonstruiert den quantisierten Anteil über einen DAC, bildet den Restfehler im Subtrahierer und verstärkt diesen für die nächste Stufe. Durch Überlappbits und digitale Fehlerkorrektur können die Genauigkeitsanforderungen an die einzelnen Stufen reduziert werden und die Durchsatzraten erhöht. Durch diesen Aufbau steigt der Aufwand mit zunehmender Auslösung nur linear an. Bauartbedingt entsteht allerdings eine definierte Zykluslatenz, die in ganzen oder halben Taktzyklen angegeben wird. Diese Latenz ist der entscheidende Nachteil eines Pipeline ADCs (vgl. [MPS] und [Ana01b]).

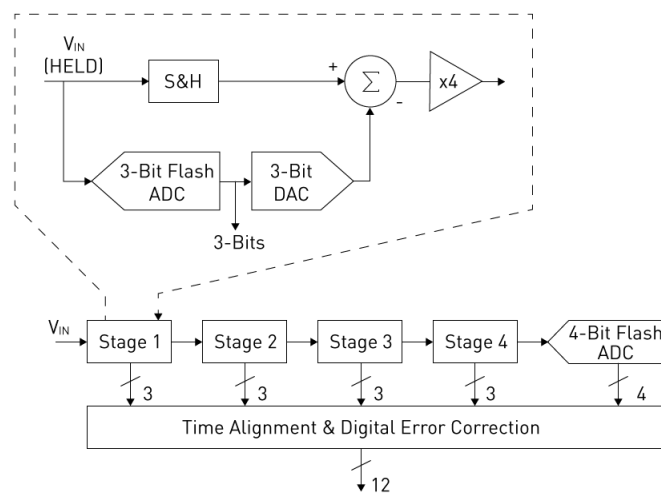


Abbildung 12: 12-bit pipelined ADC
Ersteller: MPS, Quelle: [\[Link\]](#) (Ch. 2, Fig. 4)

3.3.3. Wichtige Kenngrößen von AD-Umsetzern

Die Kenngrößen und deren Definition können dem IEEE Standard 1241-2023 “*Terminology and Test Methods for Analog-to-Digital Converters*“ ([IEE23, S. 22ff]) entnommen werden.

Auflösung und Eingangsbereich

Die Auflösung eines N -Bit-AD-Umsetzers umfasst 2^N binäre Codes. Der differentielle Eingangsspannungsbereich wird als Full-Scale Range (FSR) bezeichnet und in Volt angegeben. Das LSB entspricht $\frac{FSR}{2^N}$.

Maximale Sampling-Rate

Die Abtastrate ist der Kehrwert der Abtastperiode (Abstand zwischen Abtastwerten). Sie bestimmt die zeitliche Auflösung und ist insbesondere in Bezug auf das Nyquisttheorem relevant.

Leistungsaufnahme

Die Leistungsaufnahme ist in erster Linie für die Auslegung der Versorgungsspannung und der zugehörigen Leitungen relevant.

Verstärkungsfehler

Der Verstärkungsfehler beschreibt die Abweichung der Kennliniensteigung von der idealen Steigung nach Offsetkorrektur und wird in LSB oder in Prozent der Full-Scale-Range angegeben.

Offsetfehler

Der Offsetfehler ist die vertikale Verschiebung der Kennlinie relativ zur Referenzgeraden und wird in LSB oder in Prozent der Full-Scale-Range angegeben.

Latenz

Die Latenz wird als Pipeline-Delay in ganzen oder halben Taktzyklen angegeben oder alternativ als Zeitangabe und beschreibt den Abstand zwischen Abtastung und Verfügbarkeit des zugehörigen Digitalworts.

Differentielle Nichtlinearität DNL

Die DNL ist die Abweichung der tatsächlichen Codebreite $W[k]$ von der idealen Codebreite Q , normiert auf Q , und wird in LSB angegeben. Die Codebreite entspricht dem Spannungsbereich, der dem Code zugeordnet ist. Formal gilt $DNL[k] = \frac{W[k]}{Q} - 1$. Die Angabe im Datenblatt entspricht $\max DNL[k]$.

Integrale Nichtlinearität INL

Abweichung des Übergangsspannungsniveaus von einem Code in den nächsten zu dem der idealisierten Geraden, nach Korrektur von Offset und Verstärkung. Damit gilt $INL[k] = \sum_{n=0}^k DNL[n]$. Die Angabe im Datenblatt entspricht $\max INL[k]$.

Rausch- und Verzerrungsmaße

SNR gibt das Verhältnis von Nutzsignal- zu Rauschleistung an:

$$SNR_{dB} = 10 \cdot \log_{10} \left(\frac{P_{Signal}}{P_{Rauschen}} \right) \quad (8)$$

THD ist ein Maß für das Verhältnis von harmonischen Verzerrungen zur Grundschwingung:

$$THD_{dB} = 10 \cdot \log_{10} \left(\frac{P_{Harmonische}}{P_{Signal}} \right) \quad (9)$$

$SINAD$ umfasst harmonische Verzerrungen und Rauschen:

$$SINAD_{dB} = 10 \cdot \log_{10} \left(\frac{P_{Signal}}{P_{Rauschen} + P_{Harmonische}} \right) \quad (10)$$

$ENOB$ ist die effektive Bitzahl und wird aus $SINAD$ abgeleitet:

$$ENOB \approx \frac{SINAD[dB] - 1.76}{6.02} \quad (11)$$

3.4. Endliche Zustandsautomaten (Finite State-Machines - FSMs)

Die Firmware und Software des Projekts sind als synchronisierte Zustandsautomaten implementiert, um einen deterministischen Ablauf der beiden Programme zu gewährleisten.

3.4.1. Definition und formale Darstellung

Ein endlicher Automat (EA - engl. Finite State Machine, FSM) ist ein abstraktes Rechenmodell zur Beschreibung von Systemen. Dieser befindet sich in mindestens einem Zustand von einer Zahl endlicher Zustände. Zustandsübergänge (sog. Transitionen) erfolgen durch Eingaben oder das Auftreten von Ereignissen (spezielle Form der Eingabe).

Zustände modellieren, was das System gerade tut bzw. in welchem internen “Modus“ es sich befindet. Ereignisse sorgen für den Wechsel zwischen Zuständen. Übergänge definieren, wie das System im aktuellen Zustand auf ein Ereignis reagiert.

Ein FSM besteht nach [Bäs19, S. 7ff] typischerweise aus:

- einer Menge von Zuständen S ,
- einem Anfangszustand s_0 ,
- einem Eingabe- oder Ereignisalphabet E (Events),
- einem Ausgabealphabet A ,
- einer Zustandsübergangsfunktion $\delta : S \times E \rightarrow S$,
- einer Ausgabefunktion $\lambda : S \times E \rightarrow A$,
- und einer endlichen (evtl. leeren) Menge der Endzustände F .

Je nach Modellierung kann das Ausgabealphabet und die Ausgabefunktion, sowie die Menge an Endzuständen entfallen.

Unterschieden werden folgende Arten von Endlichen Automaten:

- *Deterministische Endliche Automaten (DEA):*

Ein Automat wird als deterministisch bezeichnet, wenn für jede Kombination aus Eingabedaten und aktuellem Zustand eindeutig festgelegt ist, welcher Folgezustand erreicht wird. Das bedeutet, dass sich das *System zu einem bestimmten Zeitpunkt stets in genau einem definierten Zustand* befindet.

- *Nichtdeterministische Endliche Automaten (NEA)*:

Bei einem NEA sind für einen bestimmten Zustand und eine Eingabe keine, genau ein oder mehrere mögliche Zustandsübergänge definiert.

DEAs sind einfacher als NEAs zu implementieren, wodurch diese in der Praxis häufiger zur Anwendung kommen (vgl. [Bäs19, S. 14]). Im Projekt sind die FSMs als DEAs ausgeführt.

Eine weitere Aufgliederung von DEAs erfolgt nach der Abhängigkeit der Ausgabe, wenn diese vorhanden ist (vgl. [Bäs19, S. 8f]).

- Wenn die Ausgabe nur vom aktuellen Zustand abhängt, handelt es sich um einen *Moore-Automaten*.
- Wenn die Ausgabe vom aktuellen Zustand und der Eingabe abhängt, handelt es sich um einen *Mealy-Automaten*.

Im Projekt kommen Moore-Automaten zur Anwendung.

3.4.2. Modellierung und Darstellung

Die Darstellung eines Zustandsautomaten erfolgt üblicherweise mit einem *Zustandsdiagramm* (auch Zustandsübergangsdiagramm), einem gerichteten Graph¹. Die Knoten stellen die Zustände der Menge S dar und die gerichteten Kanten die Zustandsübergänge. Der Startzustand und die Endzustände werden gesondert gekennzeichnet. Eine übliche Darstellung für einen einfachen Moore-Automaten findet sich in Abbildung 13a (sn sind die Zustände, xn sind Eingaben oder Ereignisse und yn sind Ausgaben; $n = 1, 2, \dots$) (vgl. [Bäs19, S. 9]).

Die Darstellung des Zustandsdiagramms ist auch als Teil der *Unified Modelling Language (UML)*² festgelegt, welche auch durch internationale Normung festgeschrieben ist. In der Praxis (vor allem beim händischen Entwurf von Zustandsdiagrammen) hat sich aber ein Mischform bewährt, welche auf einige Elemente der UML zurückgreift (siehe Abbildung 13b). Es lassen sich beispielsweise auch hierarchische Strukturen integrieren (verschachtelte Zustände).

¹“Graphen sind ein [...] Konzept, um Objekte und die Beziehung zwischen diesen darzustellen.“

Quelle: [\[Link\]](#)

²aktueller UML-Standard (V2.5.1): [\[Link\]](#)

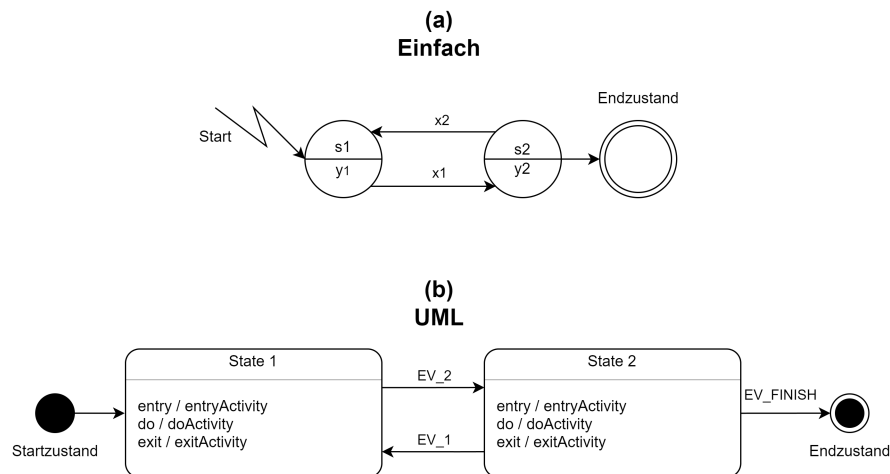


Abbildung 13: (a) Zustandsdiagramm (nach [Bäs19]); (b) UML-Zustandsdiagramm

3.4.3. Vorteile einer FSM bei der Programmierung

- *Klarheit & Übersichtlichkeit*

Der Systemverlauf ist in wohldefinierte Zustände gegliedert. Der Code wird hierdurch verständlicher (kein “Spaghetti-Code”).

- *Deterministisches Verhalten*

Durch die Definitionen, wie auf welches Ereignis reagiert wird, kann die Vorhersagbarkeit und Zuverlässigkeit gewährleistet werden. Außerdem lassen sich undefinierte Zustände durch die explizite Modellierung vermeiden.

- *Modularisierung & Wiederverwendbarkeit*

Einzelne Zustände oder Teile der FSM lassen sich kapseln, segmentieren und wiederverwenden. Teilbereiche können eventuell separat getestet werden.

- *Wartbarkeit & Erweiterbarkeit*

Das System lässt sich durch Hinzufügen neuer Zustände oder neuer Übergänge modular erweitern, ohne die vorhandene Logik stark zu verändern.

- *Kommunikation und Dokumentation*

Zustandsdiagramme können bei der Vermittlung von komplexem Programmverhalten gegenüber Nicht-Programmieren genutzt werden.

- *Verbessertes Debugging*

Da Zustandsübergänge zentralisiert sind, lassen sich Log-Messages und weitere Debugging-Mechanismen leichter verwenden.

3.5. Direct Memory Access - DMA

3.5.1. Allgemeines

DMA bezeichnet den Vorgang, bei dem durch eine dedizierte Einheit ohne Beteiligung einer CPU Daten transferiert werden (also als Hintergrundprozess). Er kommt zum Einsatz, wenn große Datenmengen von der Peripherie oder einem Speicher zu einem anderen transferiert werden müssen. Ein Chip oder eine Teileinheit eines Mikrocontroller (μ C), die solche Transfers durchführt, heißt *Direct Memory Access Controller (DMAC)* (vgl. [Urb20, S. 125]). Im Folgenden soll die Funktionsweise anhand der Realisierung in der im Projekt verwendeten STM32-Mikrocontrollerfamilie erläutert werden.

3.5.2. DMA bei STM32-Mikrocontrollern

Der DMA-Controller ist ein AHB-Modul (AHB - advanced high-performance bus; standardisiertes Bussystem von ARM) und kann wie auch die CPU des μ Cs als Master auf diesen Bus (genauer auf die Bus-Matrix) zugreifen. Durch die Datentransfers, welcher der DMAC nach seiner Konfiguration ohne CPU-Beteiligung ermöglicht, kann die System-Performance deutlich erhöht werden. Die Datentransfers können hierbei per Software oder über angeschlossene Peripherieelemente über sog. *Requests* gestartet werden. Wird als steuerndes Element beispielsweise ein Hardware-Timer genutzt ermöglicht dies die zeitlich exakte Taktung von Datentransfers [STm16, S. 6], was im Projekt für den Abtastvorgang genutzt wird.

Der μ C besitzt zwei unabhängige DMAC (Aufbau siehe Abbildung 14), deren Anbindung an Peripherie und Speicher unterschiedlich ausfällt, um alle Ressourcen des Systems flexibel zu verwenden.

Der DMAC besitzt 3 Schnittstellen:

- *slave port* zur Programmierung des DMAs
- *2 master ports*
 - *peripheral port*: peripherieseitiger Datenanschluss
 - *memory port*: speicherseitiger Datenanschluss

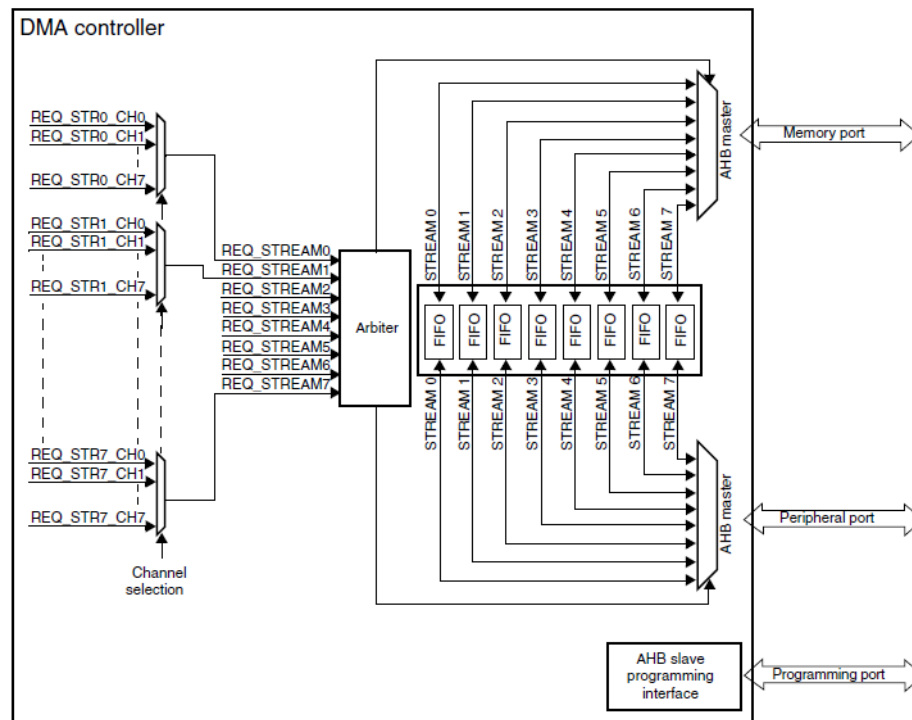


Abbildung 14: Blockdiagramm STM32-DMA-Controller aus [STm16, S. 7]

Jeder Controller besitzt 8 Datenwege (*Streams*), die separat für unterschiedliche Datentransfers konfiguriert werden können (die Transfers können aber nicht gleichzeitig laufen). Die Streams besitzen hierbei eine konfigurierbare Priorität und ein zentrales Modul, der *Arbiter*, regelt den Zugriff der Streams auf die Ports in Abhängigkeit der Priorität und sorgt für einen deterministischen Ablauf der Transfers. Die einzelnen Streams besitzen noch eine Anzahl an *Channels*, über die der entsprechende Peripherie-Request für einen Stream ausgewählt werden kann (vgl. [STm16, S. 7ff]). Die Zuordnung eines Peripherie-Requests zu einer Channel-Stream-Kombination kann dem Reference-Manual des Controllers entnommen werden (für den STM32F767: [STm24, S. 252]).

Jeder Stream besitzt außerdem einen 4-stufige *FIFO*-Pufferspeicher (First-In-First-Out), welcher Latenzen beim Zugriff auf das Übertragungsmedium (Bus-Switch-Matrix) überbrücken kann und ein *Verpacken/Entpacken der Daten* erlaubt (z.B.: input: 8bit-Pakete, output: 32bit-Pakete; siehe Abbildung 15).

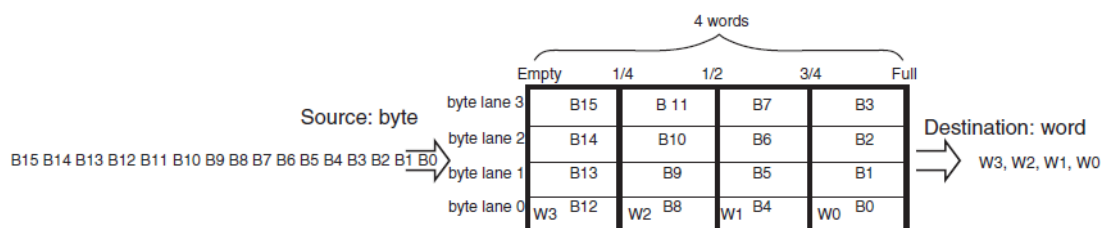


Abbildung 15: FIFO-Struktur aus [STm16, S. 11] (Teilabbildung)

DMA-Transfers

Ein Transfer wird zunächst über die *Quelladresse (source address)* und *Zieladresse (destination address)* charakterisiert, hier kann der DMA so konfiguriert werden, dass die Adressen nach einem Transfer automatisch inkrementiert werden können. Somit lassen sich, im Speicher hintereinanderliegende, Daten einfach übertragen. Quell- oder Zieladresse können jeweils aber auch konstant gehalten werden (siehe Abbildung 17).

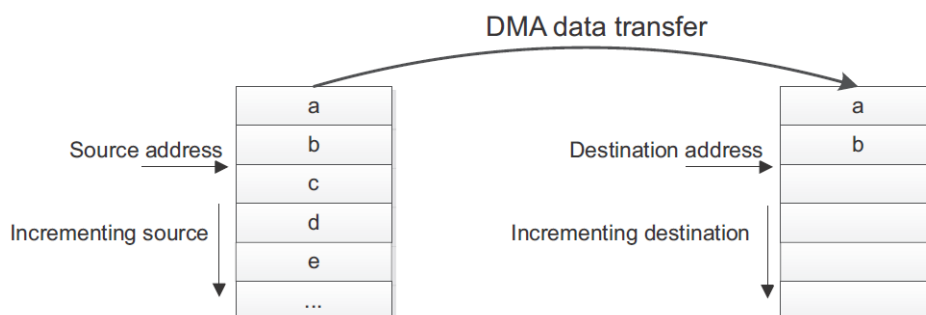


Abbildung 16: DMA Quell- und Zieladressinkrementierung aus [STm16, S. 10]

Weitere wichtige Parameter sind die Übertragungsgröße (transfer size), die in einem dedizierten Register (*NDTR* - Number of Data Transfers Register) abgelegt ist, und die Datenbreite (Byte, Half-word, Word). Der Inhalt des NDTR wird nach jedem Transfer entsprechend der Größe des Transfers dekrementiert. Hier wird noch zwischen Circular mode und Normal mode unterschieden. Beim Normal mode ist eine Transaktion (bestehend aus NDTR Transfers) bei NDTR=0 beendet. Der Stream wird deaktiviert und es

finden bis zum nächsten Aktivieren des Streams keine Transfers mehr statt. Beim Circular mode wird bei NDTR=0 das Register NDTR mit dem Initialwert geladen und die Transfers beginnen erneut (auch die Adressregister werden mit den Initialwerten geladen) → Kreislauf (circular).

Die drei möglichen Transfer-Modi sind:

- Peripheral-to-Memory (siehe Beispiel in Abbildung 17))
- Memory-to-Peripheral
- Memory-to-Memory

Bei einem Timer-Überlauf des Hardware-Timers TIM1 findet ein DMA-Request statt (ein Stream wurde hierbei entsprechend konfiguriert). Dieser Request löst einen Daten-Transfer zwischen Ziel und Quelle aus (die Datenbreite wurde im Beispiel auf 1 Byte festgelegt). Anschließend werden die Adressen entsprechend der Datenbreite inkrementiert, um die Ziel- und Quelladresse für den nächsten Transfer vorzubereiten. Dieser Prozess geschieht so lange, bis “Number of Transfers“ durchgeführt worden sind. Der Wert von NDTR wird nach dem Transfer dekrementiert.

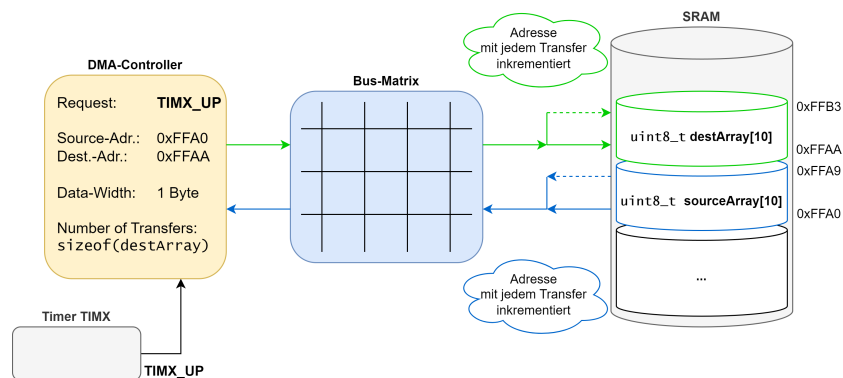


Abbildung 17: peripheral-to-memory-Transfer

Im Abtastsystem ist keine Inkrementierung der Quelladresse notwendig, da nur das Eingangsdatenregister ausgelesen werden muss, welches die Schnittstelle zum parallelen ADC-Interface darstellt (siehe Abbildung 27 im Entwurfsteil der FW).

DMA-Transferwege

Gemeinsames Übertragungsmedium ist die Bus-Switch-Matrix, die auch in den vorherigen Abbildungen schon dargestellt wurde. Der Zugriff auf diese wird mit Hilfe einer Arbitrierung nach einem round-robin-Algorithmus geregelt. Durch den Arbitrierungsvorgang oder die Blockierung des Übertragungsmediums durch einen anderen Bus-Master (z.B. die CPU) kann eine *Latenz beim DMA-Transfer* entstehen.

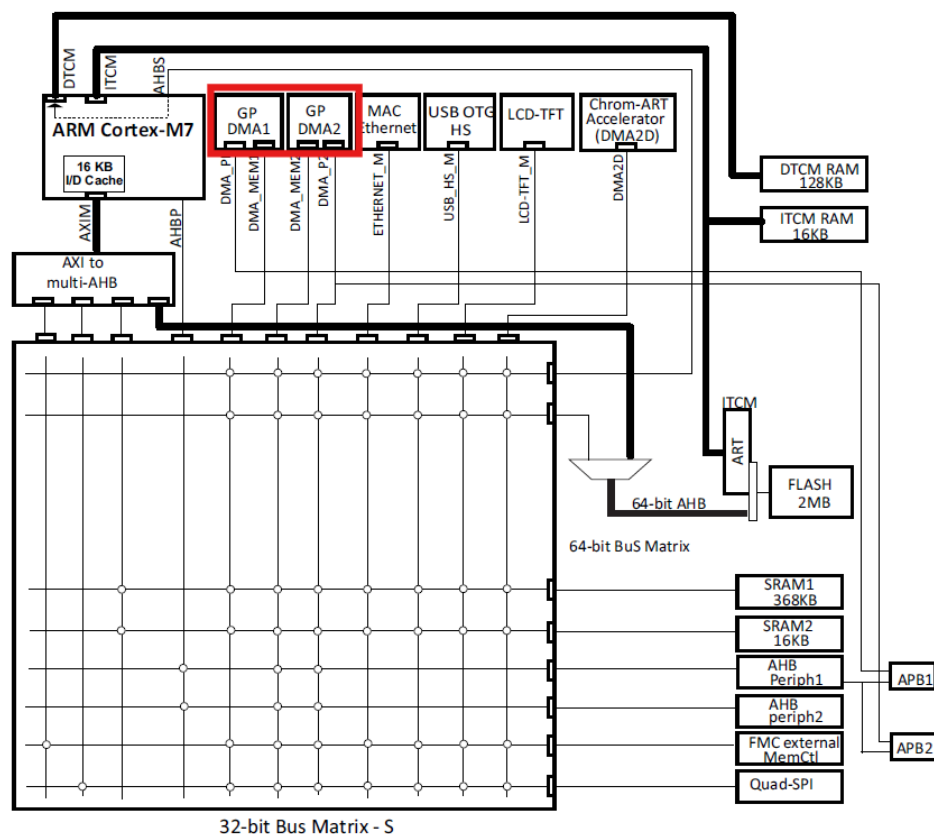


Abbildung 18: Systemarchitektur für STM32F76xxx μ C aus [STm24, S. 72]

Nur Bus-Master (z.B.: Prozessor, DMA-Controller (in Abbildung 18 rot hervorgehoben)) können Lese- und Schreiboperationen initiieren.

3.6. Digitale Filterung (Preprocessing)

3.6.1. Motivation und Grundlagen digitaler Filterung

Digitale Filterung ist ein zentrales Element der digitalen Signalverarbeitung (DSP). Filter werden im Allgemeinen für zwei Hauptzwecke eingesetzt:

- Trennung von Signalen, die miteinander überlagert wurden,
- Wiederherstellung von Signalen, die auf ihrem Übertragungsweg oder bei der Messung verzerrt wurden [Smi99, S. 261].

Während analoge Filter durch elektronische Schaltungen realisiert werden, bieten digitale Filter eine wesentlich höhere Präzision und Flexibilität. Sie können sehr steile Übergänge zwischen Durchlass- und Sperrbereich (passband und stopband) erzeugen und zeigen kaum Toleranzprobleme wie analoge Filter. Beim analogen Filterdesign ist die Beherrschung von Bauteiltoleranzen von wesentlicher Bedeutung [Smi99, S. 262]. Digitale Filterung bietet sich bei Anwendungen mit Mikrocontrollern an, da die Filter-Algorithmen direkt einprogrammiert werden können und eine analoge Filterschaltung ersetzen oder reduzieren können.

3.6.2. Filterklassifikation

Einteilung nach Signalcharakteristik

Die Einteilung der Filter richtet sich einerseits danach, in welcher Form die Information im Signal enthalten ist [Smi99, S. 265].

- *Zeitbereich (time-domain):*

Die Information ist direkt in der Form des Signals enthalten.

Bsp.: Glätten von Messdaten, DC-Offset-Entfernung, Signalerkennung.

→ Die Sprungantwort (step-response) ist hier entscheidend.

- *Frequenzbereich (frequency-domain):*

Die Information steckt in den Frequenzkomponenten des Signals.

Bsp.: Trennung von Audiosignalen oder Herausfiltern bestimmter Störfrequenzen.

→ Der Frequenzgang (frequency-response) ist hier entscheidend.

Einteilung nach Implementierung

Die Einteilung der Filter richtet sich einerseits danach, in welcher Form die Information im Signal enthalten ist [Smi99, S. 265].

- *FIR-Filter (Finite Impulse Response):*

Umsetzung erfolgt durch **Faltung (Convolution)** des Eingangssignals mit einem endlichen Filterkern (*engl. filter kernel*). Ein Ausgangswert wird berechnet, indem Eingangswerte mit zuvor definierten Gewichtungsfaktoren multipliziert und anschließend aufaddiert werden.

Eigenschaften:

- Stabil und vorhersagbar,
- Endliche Impulsantwort,
- Einfach zu implementieren, aber rechenintensiv.
- Beispiel: Moving Average Filter.

- *IIR-Filter (Infinite Impulse Response):*

Umsetzung durch **Rekursion**, bei der sowohl aktuelle Eingangswerte als auch frühere Ausgangswerte zur Berechnung herangezogen werden.

Eigenschaften:

- Sehr effizient (weniger Rechenaufwand),
- Theoretisch unendliche Impulsantwort,
- Gefahr der Instabilität bei fehlerhafter Parametrierung.
- Beispiel: digitale Tiefpassfilter auf Basis analoger Schaltungen.

3.6.3. Gleitender Mittelwertfilter

Der *gleitende Mittelwertfilter* (engl. *Moving Average Filter - MAF*) ist einer der einfachsten und am häufigsten eingesetzten digitalen Filter. Er arbeitet, indem er für jedes Ausgangssample den Mittelwert aus einer festen Anzahl aufeinanderfolgender Eingangssamples berechnet [Smi99, S. 277]. Seine Hauptaufgabe besteht darin, zufälliges Rauschen zu reduzieren (siehe Abbildung 19), während plötzliche Signaländerungen (Steps) möglichst gut erhalten bleiben.

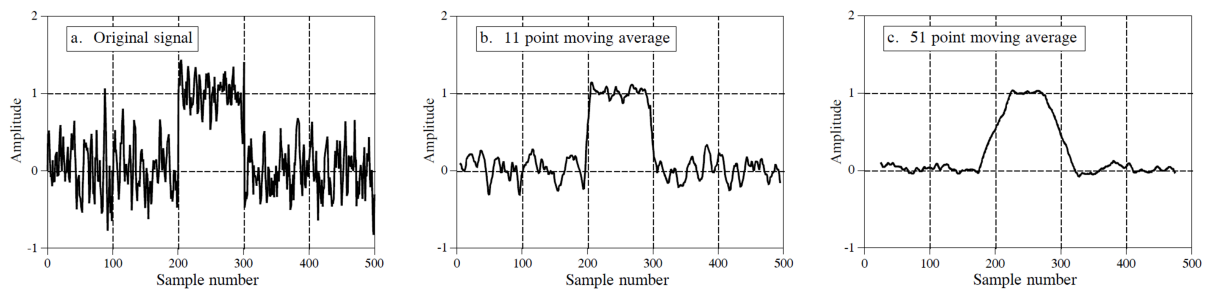


Abbildung 19: Anwendung eines MAF unterschiedlicher Fensterbreite

(a) Eingangssignal, (b) Ausgangssignal $M = 11$, (c) Ausgangssignal $M = 51$
(Figure 15-1 aus [Smi99, S. 279])

Grundidee Für ein Fenster der Breite M ergibt sich das Ausgangssignal $y[i]$ aus dem Eingangssignal $x[i]$ durch:

$$y[i] = \frac{1}{M} \sum_{j=0}^{M-1} x[i+j] \quad (12)$$

Es ist zu erkennen, dass die rechtsseitigen Eingangswerte zum Ausgangswert aufsummiert werden. Eine alternative Umsetzung wäre es die aufzusummierenden Eingangswerte symmetrisch um den Ausgangswert zu wählen (Beispiel für einen Ausgangswert):

$$y[80] = \frac{x[78] + x[79] + x[80] + x[81] + x[82]}{5} \quad (13)$$

Damit entspricht der Moving Average Filter einer Faltung des Eingangssignals mit einem rechteckigen Filterkern [Smi99, S. 277f].

Für das Projekt wurde der Filter kumulativ und rekursiv implementiert und nach dem Algorithmus in ?? im Unterunterabschnitt 7.1.4 umgesetzt.

3.7. Parallele Prozesse und Nebenläufigkeiten in Python

Die folgenden Ausführungen orientieren sich maßgeblich auf den Erkenntnissen von Niklas Lang ...

3.7.1. Threads in Python

Ein Thread ist ein paralleler Ausführungsstrang innerhalb eines Programms. Alle Threads teilen sich den selben Speicherbereich. Dadurch kann es zu Wettlaufbedingungen kommen oder zu unvorhersehbaren Verhalten. Der Zugriff auf gemeinsame Daten muss daher synchronisiert werden, z.B. mit Locks, Semaphoren oder Warteschlangen. Durch den Global Interpreter Lock (GIL), ein zentrales Konzept im Standard-Python-Interpreter CPython, kann zu einem Zeitpunkt immer nur ein Thread, Python-Bytecode ausführen. Das heißt, auch wenn für einen Job mehrere Threads gestartet werden, arbeitet immer nur ein Thread gleichzeitig. Dieses Prinzip widerspricht zwar der Idee von paralleler Ausführung, jedoch darf ein anderer Thread Programmcode ausführen, wenn der Thread gerade auf eine Eingabe wartet. Das GIL wird in Python genutzt, da es die Verwaltung von Python-Objekten im Speicher vereinfacht. Dies ist vor allem wichtig, da der Speicher in Python typischerweise automatisch verwaltet wird (vgl. [Lan24] Abschnitt “Was sind Threads in Python”).

Threading eignet sich vor allem für E/A-lastige Aufgaben, die mit Datenaustausch verbunden sind.

Synchronisation

- Locks: Sperren, die verhindern, dass mehrere Threads gleichzeitig auf dieselben Daten zugreifen.
- Semaphoren: Begrenzen die Anzahl gleichzeitiger Zugriffe auf eine Ressource.
- Warteschlange (Queues): Erlauben thread-sicheren Austausch von Datenobjekten, nach dem FIFO-Prinzip.

3.7.2. Multiprocessing in Python

Beim multiprocessing werden mehrere, vollständig unabhängige Prozesse gestartet. Jeder Prozess ist eine eigene Instanz des Programms mit eigenem Speicherbereich. Dies führt

dazu, dass die Kommunikation zwischen den jeweiligen Prozessen komplexer ist. Hierfür stellt das Python-Modul multiprocessing Funktionen und Klassen bereit.

Multiprocessing kann die Leistungsfähigkeit bei rechenintensiven Aufgaben deutlich steigern, bringt aber auch Overhead durch Speicherbedarf und zusätzliche Komplexität in der Verwaltung mit.

Synchronisation

- Pipes: Unidirektionale Kanäle für Datenübertragung zwischen Prozessen.
- Shared Memory: Gemeinsamer Speicherbereich für Datenaustausch.

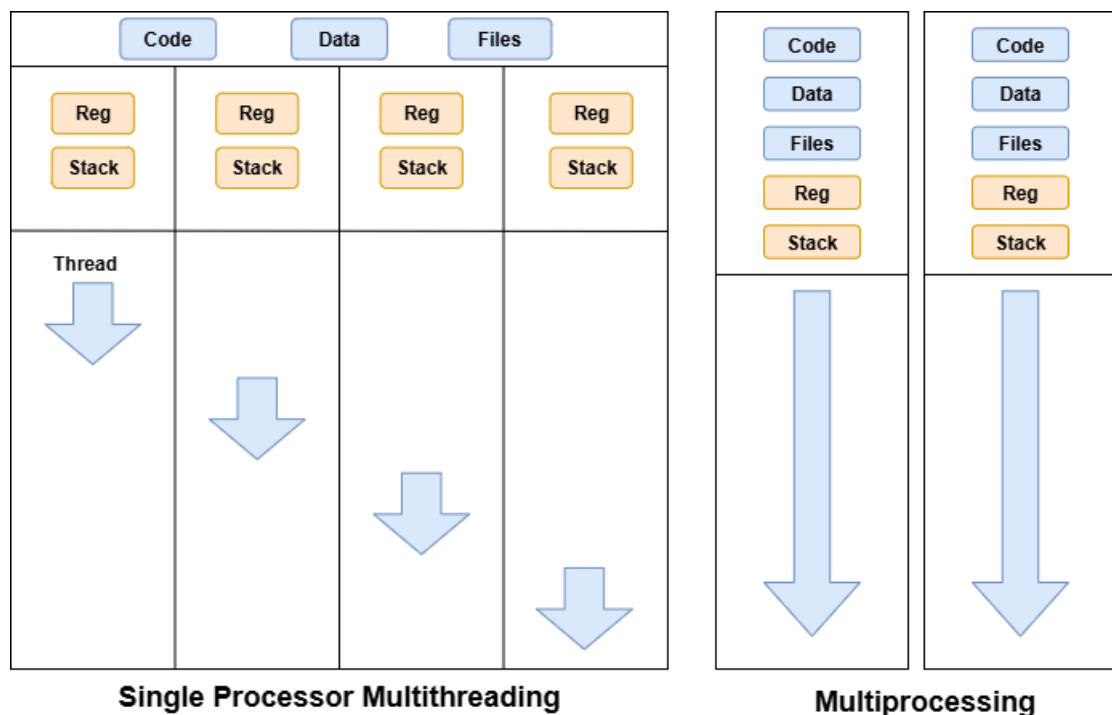


Abbildung 20: Vergleich Multi-Threading und Multi-Processing

In unserem Projekt wurden Threads verwendet, da die Anwendung hauptsächlich E/A-gebunden ist. Zusätzlich wird ein gemeinsamer Speicher benötigt, damit die Messdaten an die Benutzeroberfläche übergeben werden können. Die Kommunikation wurde mit zwei Warteschlangen realisiert, eine dient zur Übergabe von Statusmeldungen von der FSM zur Benutzeroberfläche (GUI) und die andere zur Übergabe von Befehlen von der GUI an die FSM (vgl. [Lan24] Abschnitt “Was ist Multiprocessing in Python?”).

4. Projektkonzeption

4.1. Vorgehensweise

Das übergeordnete Vorgehen folgte einem strukturierten Prozess, der sich gut durch das **V-Modell** visualisieren lässt. Diese Vorgehensweise eignete sich insbesondere aus drei Gründen: *Erstens*, Änderungen an Hardware sind nach deren Umsetzung nur schwer möglich, was iterative Prozesse sehr aufwendig macht. *Zweitens* standen die Projektziele von Beginn an eindeutig fest. *Drittens* waren die Aufgabenbereiche klar voneinander abgegrenzt. Zudem handelte es sich um ein überschaubares Projekt.

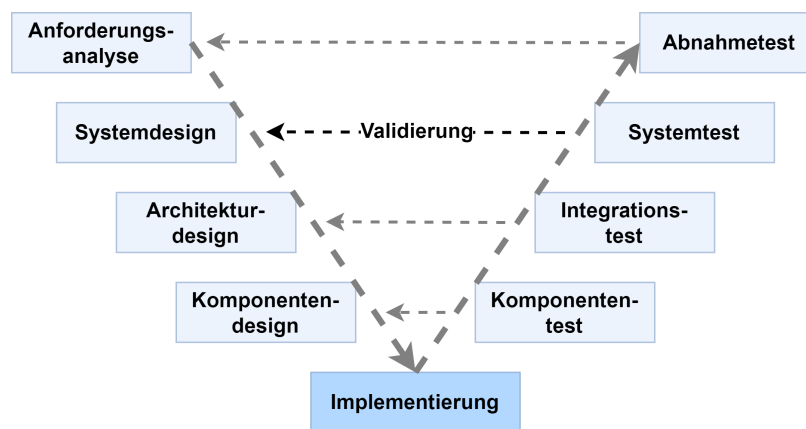


Abbildung 21: V-Modell

Im Folgenden werden die einzelnen Phasen und zugehörigen Validierungsschritte kurz erläutert.

Anforderungsanalyse und Abnahmetest

In enger Abstimmung mit dem betreuenden Professor wurden die grundlegenden Anforderungen definiert. Dazu gehörten unter anderem Bandbreite, Abtastrate, Auflösung und Eingangsspannungsbereich. Der spätere Abnahmetest bestand darin zu überprüfen, ob das entwickelte System diese Vorgaben in einer realistischen Versuchsumgebung erfüllte.

Systemdesign und Systemtest

Anschließend wurden die grundlegenden Systemfunktionen festgelegt, die zur Erfüllung der Anforderungen notwendig waren. Dazu zählten Signalaufnahme, Digitalisierung, Weiterleitung über USB und Visualisierung am PC. Im Systemtest wurde das fertige Gerät als Gesamtsystem geprüft.

Architekturdesign und Integrationstest

In dieser Phase erfolgte die Aufteilung des Systems in die Bereiche Hardware, Firmware und Software. Dabei wurden die Schnittstellen definiert, die ermöglichen sollten, die Module weitgehend unabhängig voneinander zu entwickeln. Diese wurden während des Komponentendesigns bei Bedarf konkretisiert. Im Integrationstest wurde die Zusammenarbeit dieser Module überprüft, insbesondere die zuverlässige Kommunikation zwischen den Subsystemen.

Komponentendesign und Komponententest

In dieser Phase wurden die Teilsysteme detailliert ausgearbeitet. Es wurden Konzepte erstellt, Bauteile gewählt, Implementierungsmöglichkeiten abgewogen etc. Jedes Subsystem wurde einzeln getestet, z. B. Spannungsversorgung und ADC in der Hardware, Speicherzugriffe in der Firmware oder Datenanzeige in der Software.

Implementierung

In der zentralen Phase wurden die geplanten Komponenten umgesetzt. Dies umfasste das Layout der Schaltungen, die Programmierung der Firmware sowie die Entwicklung der PC-Software. Ziel war es, die entworfenen Module funktionsfähig zu realisieren und für die Integration bereitzustellen.

Organisation und Zusammenarbeit

Die Zusammenarbeit erfolgte hauptsächlich über ein gemeinsames Projektteam in Microsoft Teams. Dieses war in mehrere Kanäle gegliedert: einen allgemeinen Kanal für projektübergreifende Informationen, jeweils eigene Kanäle für Hardware, Firmware und Software sowie zusätzliche Kanäle für die Schnittstellen. Diese Struktur erleichterte die Ablage von Quellen, Materialien und Informationen und stellte die Übersicht über das gesamte Projekt sicher.

Die Abstimmung innerhalb des Teams erfolgte in regelmäßigen Treffen, in denen abgeschlossene Arbeitspakete dokumentiert und neue Zielvorgaben bis zum nächsten Treffen festgelegt wurden. Die Ergebnisse wurden jeweils in Protokollen festgehalten. Eine detaillierte Auflistung der Tätigkeiten sowie deren Zuordnung zu den einzelnen Teammitgliedern findet sich im Anhang A.

4.2. Anforderungen

Die Anforderungen an das Projekt wurden in Zusammenarbeit mit dem betreuenden Professor sowie dem Laboringenieur des MCT-Labors Hr. Lenkowski nach einer ersten Machbarkeitsanalyse im Hinblick auf die definierten Projektziele festgelegt. Neben den obligatorischen Grundfunktionen eines digitalen Speicheroszilloskops (siehe Unterabschnitt 3.1 “Allgemeiner Aufbau eines DSOs“) wurden folgende zentrale technische Parameter bestimmt:

- Bandbreite: 1 MHz (Grenzfrequenz des Anti-Aliasing-Filters)
- Abtastrate: mindestens 10 MS/s (siehe Unterabschnitt 3.2)
- Auflösung: mindestens 10 Bit
- Analoge Eingangsspannung: ± 5 V
- Offset: ± 5 V (bezogen auf die Eingangsspannung)

Ein weiterer wesentlicher Aspekt war die *Spannungsversorgung* des Systems, für die bewusst keine strikten Anforderungen formuliert wurden. Auf diese Weise sollte während des Entwicklungsprozesses ein höheres Maß an Flexibilität gewährleistet werden, um den Fokus auf die grundlegende Funktionalität des Gesamtsystems legen zu können. Die Signalaufnahme sollte auf *einen Kanal* beschränkt bleiben, um Schwierigkeiten bei der Synchronisation mehrerer ADCs sowie den erhöhten Verwaltungsaufwand für Mehrkanalsysteme zu vermeiden. Außerdem sollte die Möglichkeit bestehen, dass Oszilloskop mit einem Standardtastkopf zu betreiben. Zusätzlich war vorgesehen, die Baugruppe als *erweiterbares Steckboard für ein Mikrocontroller-Entwicklungsboard* umzusetzen, wodurch die Komplexität reduziert und die Funktionalität der Baugruppe in den Vordergrund gestellt werden konnte.

Für die Schnittstelle zwischen ADC und Mikrocontroller wurde eine *parallele Anbindung* vorgesehen, da eine serielle Übertragung über SPI bei den geforderten Abtastraten eine zu hohe Taktfrequenz erfordert hätte. Dies hätte einen direkten Datentransfer in den Speicher des Mikrocontrollers nicht mehr zuverlässig ermöglicht. Für die zentrale Recheneinheit wurde sich auf die *Mikrocontroller der STM32-Familie* beschränkt, um vorhandene Kenntnisse aus den Modulen Mikrocomputertechnik und Embedded Systems gezielt einsetzen zu können. Die Kommunikation mit dem PC sollte über eine *USB-Schnittstelle* erfolgen, um eine möglichst uneingeschränkte Kompatibilität zu gewährleisten. Auf dem

PC war die Entwicklung einer *eigenen Anwendung zur Visualisierung und Verarbeitung der Messdaten* vorgesehen. Diese sollte über *genormte Kommandos* kommunizieren, sodass auch eine Integration in gängige Entwicklungs- und Analyseumgebungen wie *LabVIEW* oder *MATLAB* möglich wäre.

Neben diesen zentralen Vorgaben wurden auch *optionale Erweiterungen* definiert, die nicht als kritisch für den Projekterfolg eingestuft wurden oder den vorgesehenen Arbeitsumfang überschritten. Dazu zählten insbesondere die Implementierung eines *Logikanalysators* mit Protokolldekodierung gängiger Schnittstellen (z. B. SPI, I²C, UART), die Realisierung einer *Fast-Fourier-Transformation (FFT)* zur Spektrumanalyse, die Erweiterung um einen *zweiten analogen Kanal*, die Integration von *Mathematik- und Messfunktionen* sowie die Nutzung unterschiedlicher *Triggerbedingungen*. Weitere Optionen im Hardwarebereich betrafen eine *Optimierung hinsichtlich elektromagnetischer Verträglichkeit (EMV)*, eine *Minimierung des Rauschverhaltens*, die *Stromversorgung über USB* sowie die *Integration des Mikrocontrollers direkt auf der Baugruppe*.

4.3. Konzept / Architektur

Parallel zu den Anforderungen wurde ein Konzept entwickelt, um die grundlegende Architektur und Aufgabenverteilung des Projekts festzulegen. Zu Beginn entstand ein grundlegendes, rein funktionales Übersichtsbild (siehe Abbildung 58 in Anhang A). Dieses stellt die wesentlichen Signalflüsse und Verarbeitungsschritte dar, die für die Realisierung des USB-Oszilloskops erforderlich sind. Der Signalpfad führt dabei vom Device under Test (DUT) über die Signalerfassung zur Speicherung der Rohdaten und weiter zur Verarbeitung auf PC-Ebene. Dieses Schema diente als Orientierungshilfe für die weiteren konzeptionellen Schritte, ohne jedoch bereits konkrete Funktionseinheiten Bauteile oder Schnittstellen festzulegen.

Auf Basis dieser funktionalen Skizze sowie des in der Fachliteratur beschriebenen Grundaufbaus eines digitalen Speicheroszilloskops (siehe Unterabschnitt 3.1) wurde schließlich ein Gesamtkonzept entwickelt, das Funktionseinheiten definiert und den jeweiligen Bereichen Hardware, Firmware und Software zuordnet (siehe Abbildung 22).

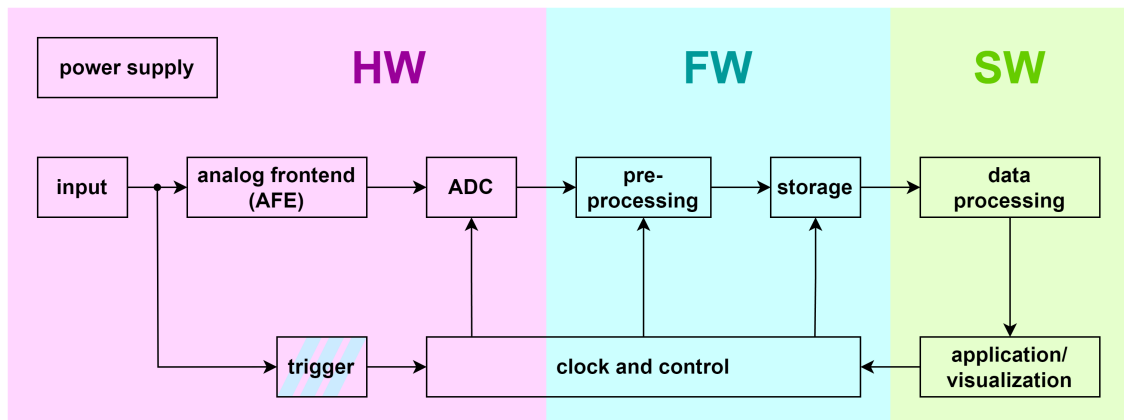


Abbildung 22: Gesamtkonzept des USB-Oszilloskops (nach [Müh20, S. 216])

Ausgehend von diesem Konzept und der klaren Aufteilung in die Bereiche Hardware, Firmware und Software, wurden die entsprechenden Schnittstellen festgelegt und die Aufgabenbereiche verteilt.

5. Hardware (HW)

5.1. Entwurf

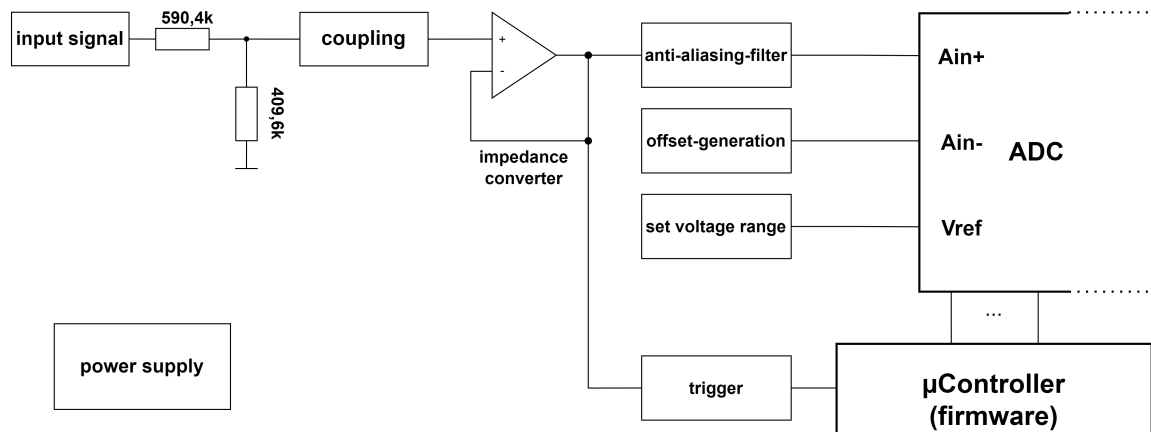


Abbildung 23: Prinzipschaltbild des Analog-Frontends (AFE)

5.2. Implementierung

5.3. HW-Test

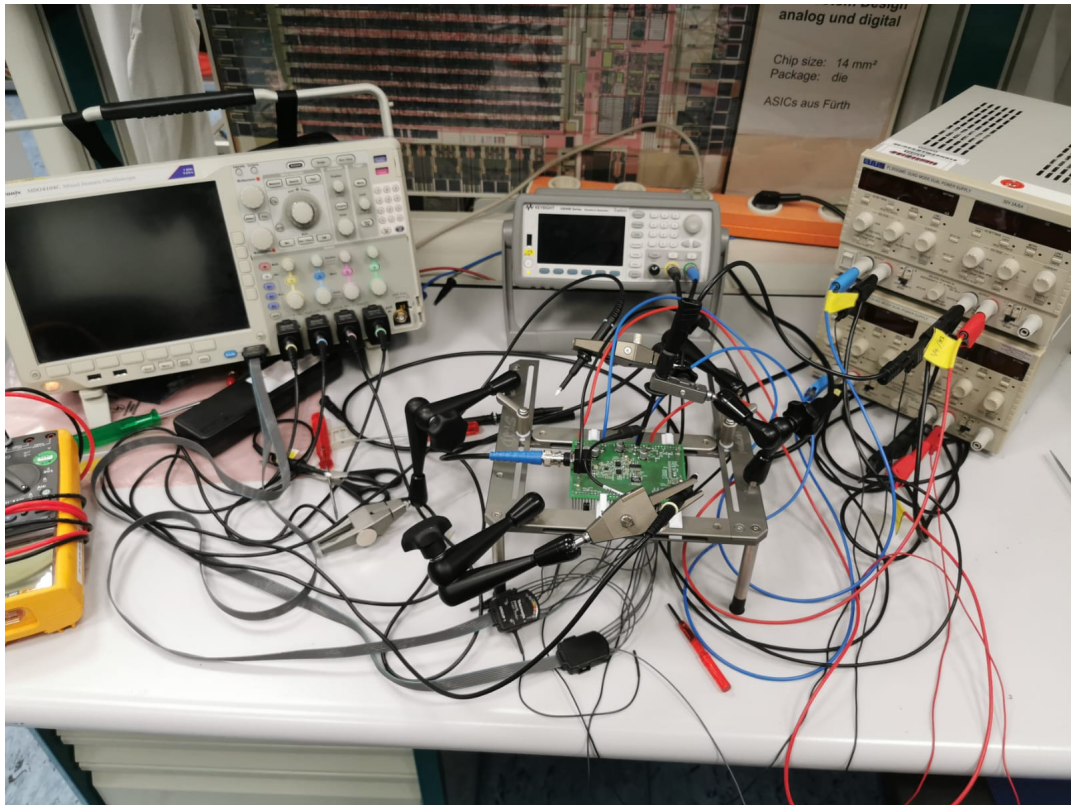


Abbildung 24: Aufbau zum Test der Hardware

6. Schnittstelle Hardware - Firmware

- Anforderungen HW-seitig und FW-seitig (wie gelöst?)
- Explizite Erwähnung, dass das Routing der ADC-Datenpins zu einem Port des uC geschehen musste! Aufwand im Layout)
- Pinbelegung
- Komparator-Auswahl (LT1714)
- Pegelwandlung: Trigger und Clock
- Pegelwandlung: Pull-Ups+Open-Drain; Spannungsteiler
- DAC-Ansteuerung

6.1. Auswahl des Komparators

7. Firmware (FW)

7.1. Entwurf

7.1.1. Auswahl des Mikrocontrollers

Eine zentrale Entscheidung im Entwurf der Firmware war die Auswahl eines geeigneten Mikrocontrollers (μC). Dieser stellt den zentralen Knotenpunkt der Gesamtanwendung dar. Hierzu wurden nach der Erstellung des Gesamtkonzepts zunächst die Anforderungen definiert, die an den Mikrocontroller gestellt werden. Bei der Auswahl musste berücksichtigt werden, dass ein *Direct Memory Access (DMA)* vorhanden ist. Dieser ermöglicht die Speicherung der Daten nach Eintreten eines Trigger-Ereignisses ohne Umweg über die CPU. Außerdem wurde gemäß der Anforderungen im Gesamtkonzept eine *USB-Schnittstelle* benötigt, um die Kommunikation des Mikrocontrollers (USB-Device) mit dem PC (USB-Host) sicherzustellen. Dabei war es von Vorteil, wenn der Controller mindestens Full-Speed-Übertragungen mit 12 Mbit/s, idealerweise High-Speed-Übertragungen mit 480 Mbit/s unterstützt. Weiterhin war es erforderlich, dass ein *erschwingliches Entwicklungsboard* verfügbar ist, welches für den ersten Prototypen genutzt werden kann. Dieses sollte zunächst zum Einsatz kommen, bevor der Mikrocontroller zusammen mit den übrigen Schaltungsteilen auf eine gemeinsame Leiterplatte integriert wird. Darüber hinaus sollten *standardisierte Schnittstellen* wie I²C und SPI vorhanden sein, um die Anbindung eines externen ADCs auch auf andere Weise zu ermöglichen, falls die parallele Schnittstelle nicht umsetzbar wäre. Ein *hoher GPIO Input Speed* war notwendig, um die geplante parallele Anbindung eines externen ADCs mit 12 Bit Auflösung über zwölf parallele GPIO-Leitungen bei der geplanten Abtastrate zu gewährleisten. Außerdem musste der Mikrocontroller über *ausreichend großen SRAM-Speicher* verfügen, um die erfassten Abtastwerte ablegen zu können.

$$[\text{Benötigter Speicher}] = [\text{Anzahl Abtastwerte}] \cdot \left\lceil \frac{2\text{Byte}}{\text{Abtastwert}} \right\rceil \quad (14)$$

$$[\text{Anzahl Abtastwerte}] = \frac{[\text{gewünschte Abtastzeit}]}{[\text{Abtastperiode}]} \quad (15)$$

Die Anforderungen wurde unter der Annahme formuliert, dass ein externer ADC eingesetzt werden sollte. Die angestrebte Abtastrate von etwa 10 MHz musste auch mikrocontrollerseitig erreichbar sein.

Auf Grundlage der positiven Erfahrungen mit STM32-Mikrocontrollern der Firma STMicroelectronics sowie der integrierten Entwicklungsumgebung STM32CubeIDE im MCT-Praktikum, wurde entschieden, einen Mikrocontroller aus dieser Serie auszuwählen. Ein zusätzlicher Vorteil dieser Produktfamilie ist die Verfügbarkeit kostengünstiger Entwicklungsboards (NUCLEO-Boards), die sich hervorragend für die ersten Entwicklungsschritte eignen und die Materialkosten des Projekts reduzieren.

Nach einem Vergleich verschiedener STM32-Produktfamilien in Bezug auf die definierten Anforderungen fiel die Wahl auf die [STM32F7-Serie](#). Diese Mikrocontroller basieren auf einem ARM Cortex-M7 und gehören zum High-Performance-Bereich der STM32-Serie. Lediglich die STM32H7-Serie bietet noch höhere Leistung, da sie über einen weiteren Prozessorkern (Cortex-M4-Core) verfügen. Schlussendlich wurde der [STM32F767ZI](#) als Mikrocontroller ausgewählt. Dieser bietet im Vergleich zu anderen Controllern derselben Familie einen größeren Flash- und SRAM-Speicher. Zudem ist ein passendes Entwicklungsboard, das [NUCLEO-F767ZI](#), verfügbar. Ein weiterer Vorteil dieses Boards ist das integrierte Programmiergerät, das durch einen separaten Mikrocontroller mit entsprechender Firmware realisiert wird.

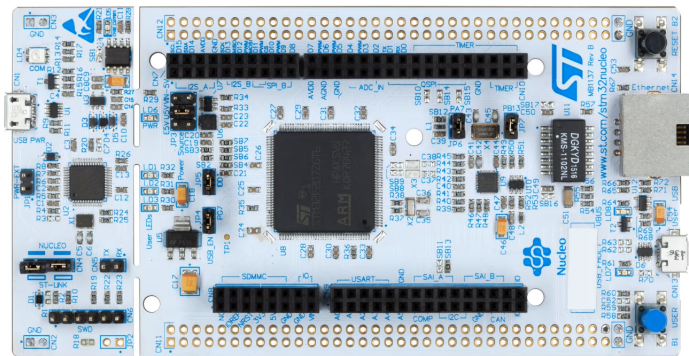


Abbildung 25: Beispielabbildung für ein NUCLEO-144-Board, wie das NUCLEO-F767ZI (Quelle: [\[Link\]](#))

Nachdem die Entwicklungsboards in Absprache mit dem MCT-Labor beschafft worden waren, erfolgte eine erneute Einarbeitung in die Arbeit mit ARM Cortex-M Mikrocontrollern und den spezifischen Funktionen der STM32-Hardware.

7.1.2. Konzept des Abtastprozesses

Der mikrocontrollerseitige Abtastprozess gliedert sich in zwei Hauptaufgaben:

- Das *Feststellen des Aufzeichnungsbeginns* durch ein definiertes Triggerereignis und
- die *Aufnahme und Speicherung* der digitalisierten Abtastwerte.

Das Konzept der Abtastung ist als Funktionsschema in Abbildung 26 dargestellt.

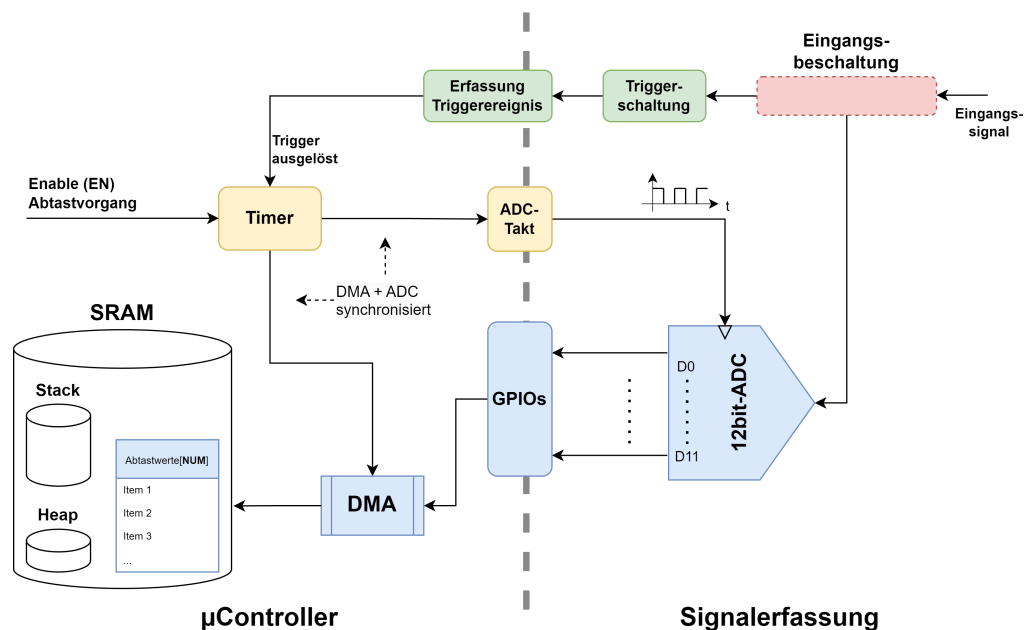


Abbildung 26: Funktionsschema der Abtastung

Parallele Datenanbindung

Für die Übertragung der Messdaten vom externen ADC zum Mikrocontroller wurde eine *parallele Schnittstelle* umgesetzt. Jedes der zwölf Bits des 12-Bit-ADCs wird dabei über eine eigene Datenleitung an den Mikrocontroller herangeführt. Der wesentliche Vorteil dieser Anbindung liegt in der vergleichsweise einfachen Implementierung, da kein komplexes serielles Übertragungsprotokoll notwendig ist. Ein Nachteil besteht jedoch darin, dass bei hohen Abtastraten Synchronisationsprobleme auftreten können.

Die GPIO-Pins des Mikrocontrollers sind in Ports mit jeweils 16 Pins organisiert, die sich ein gemeinsames *Input Data Register (IDR)* teilen. Da für die Parallelschnittstelle

zwölf Leitungen benötigt werden, können alle Bits der ADC-Ausgabe in einem einzigen Lesezugriff erfasst werden. Die Aktualisierung des IDR erfolgt mit jedem Zyklus des AHB-Busses [STm24, S. 224], der mit bis zu 216 MHz betrieben werden kann (siehe Clock-Tree in [STm24, S. 154]). Dieser Wert wird aber durch eine vorhandene Eingangskapazität der GPIOs reduziert, welche eventuell auf einen neuen Wert (Spannungspegel) umgeladen werden müssen. Die tatsächliche, maximal mögliche Abtastfrequenz hängt also von dieser Eingangskapazität und der treibenden Schaltung ab. Im Datenblatt des μC ist für die Eingangskapazität ein Wert von $C_{IO} = 5pF$ angegeben ([STm25a, S. 157]). Maximale Änderungsfrequenzen sind aber nur für die Nutzung der GPIOs als Output angegeben (die treibende Stufe ist dann die Ausgangsstufe des GPIO). Hier werden, je nach Einstellung des GPIO-Speeds (konfigurierbar), selbst im langsamsten Modus noch Frequenzen im einstelligen MHz-Bereich erreicht ([STm25a, S. 160f]). Unter der Annahme, dass die Treiberfähigkeit der ADC-Datenausgänge höher ist, kann die Abtastrate von 10 MHz realisiert werden.

Speicherorganisation

Die erfassten Abtastwerte werden zunächst im internen SRAM des Mikrocontrollers zwischengespeichert. Der STM32F767 verfügt über insgesamt 512 KB SRAM, aufgeteilt in mehrere Speicherblöcke [STm24, S. 77ff] (Memory Map). Für das Projekt wurde zunächst der *SRAM1-Bereich* (368 KB) für die Speicherung der Messdaten reserviert, während der *DTCM-Bereich* (128 KB) für Stack und Heap vorgesehen wurde. Diese Aufteilung gewährleistet eine klare Trennung zwischen Programmdaten und Messwertspeicher und kann bei der Implementierung im Linker-Script festgelegt werden.

DMA-basierter Datentransfer

Ein direkter Zugriff der CPU auf die GPIO-Datenregister und das anschließende Abspeichern der Daten in den SRAM würde den Prozessor vollständig auslasten und keine parallele Ausführung anderer Aufgaben ermöglichen. Aus diesem Grund wurde der Datentransfer mithilfe des Direct Memory Access (DMA) realisiert (siehe Unterabschnitt 3.5).

Beim gewählten Ansatz wurde der DMA-Controller so konfiguriert, dass er den Wert des Eingangsdatenregisters der GPIOs automatisch in den SRAM schreibt, ohne dass die CPU beteiligt ist (siehe Schema in Abbildung 27). Jeder DMA-Transfer wird durch

einen Hardware-Timer (TIM1) angestoßen. Dadurch kann eine exakte Synchronisation zwischen dem Clock-Signal des ADC und der Datenübertragung erreicht werden. Die Quelladresse bleibt während der gesamten Aufzeichnung konstant (Adresse des IDR), während die Zieladresse nach jedem Transfer um zwei Byte inkrementiert wird, um den nächsten Speicherplatz im SRAM anzusprechen.

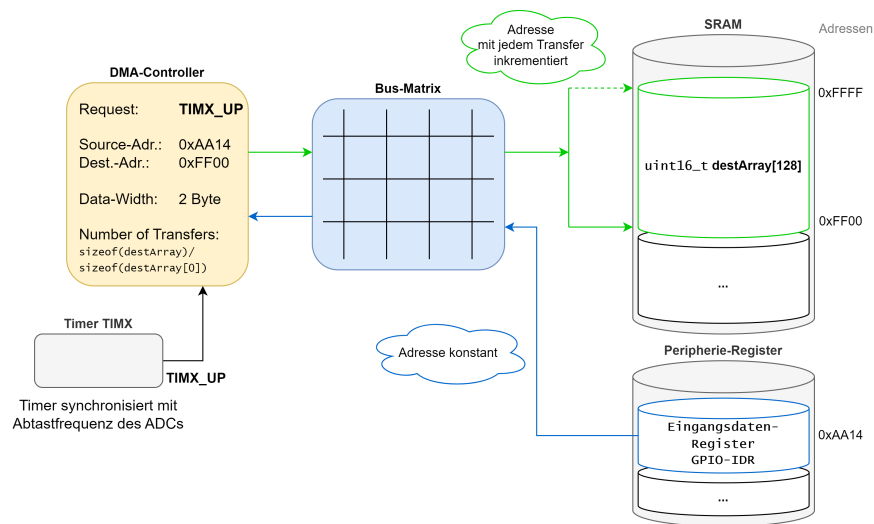


Abbildung 27: Funktionsprinzip DMA Abtastung - Adresswerte dienen nur der Übersicht

Da das NDTR-Register (Number of Data Transfers Register) maximal 65.535 Transfers speichern kann [STm24, S. 276], wird der Double-Buffer Mode (DBM) des DMA-Controllers eingesetzt (benötigt werden z.B. 100.000 Transfers für 100.000 Abtastwerte). In diesem Modus werden abwechselnd zwei getrennte Zieladress-Zeiger genutzt, um kontinuierliche Transfers ohne Datenverlust zu ermöglichen. Sobald ein Speicherbereich befüllt ist, wird automatisch in den anderen Bereich weitergeschrieben. Gleichzeitig kann die Software den gefüllten Speicherbereich auswerten oder für den USB-Transfer vorbereiten, ohne den laufenden Datentransfer zu unterbrechen.

Trigger-Mechanismus

Der Beginn der Aufzeichnung wird durch ein definiertes Triggerereignis gesteuert. Nach dem Auslösen dieses Ereignisses startet der DMA automatisch mit dem Füllen des vorgesehenen Speicherbereichs. Die Größe dieses Bereichs sowie die Anzahl der Transfers wurde zuvor in der Firmware festgelegt. Sobald der Speicher vollständig gefüllt ist, wird der Ab-

tastprozess automatisch beendet, und die Daten stehen für die anschließende Übertragung an den PC bereit. Der Trigger-Mechanismus wurde zunächst mit Hardware-Interrupts geplant (bei STM32: Extended Interrupts - EXTI). Mit dem EXTI-Modul kann ein Aufruf eines Exception-Handlers³ bei einer steigenden oder fallenden Flanke an einem GPIO erfolgen, in welchem der DMA für die Datentransfers aktiviert wird⁴.

Synchronisation mit ADC-Takt

Für eine störungsfreie Datenaufnahme muss der Datentransfer exakt auf den ADC-Takt abgestimmt werden. Hierfür wurde das Taktsignal für den ADC durch denselben Timer erzeugt, der auch die DMA-Requests auslöste. Die Hardware-Timer der STM32-Controller besitzen sogenannte Capture-Compare-Einheiten mit sogenannten Output-Compare-Anschlüssen (OC), welche mit bestimmten GPIOs des μC verbunden werden können (siehe [STm24, S. 877]). Der Hardware-Timer kann hierdurch einen GPIO ohne CPU-Beteiligung ansteuern. Um ein Setup-Violation-Problem zu vermeiden, wurde der Takt invertiert: Die positive Flanke des Signals erzeugt am ADC-Ausgang einen neuen Wert, während synchron zur negativen Flanke der DMA-Transfer ausgelöst wird. Dadurch können sich die ADC-Daten für eine halbe Abtastperiode $\frac{T_s}{2}$ stabilisieren, bevor sie eingelesen werden. Dies stellt eine zuverlässige Digitalisierung sicher (siehe Abbildung 28).

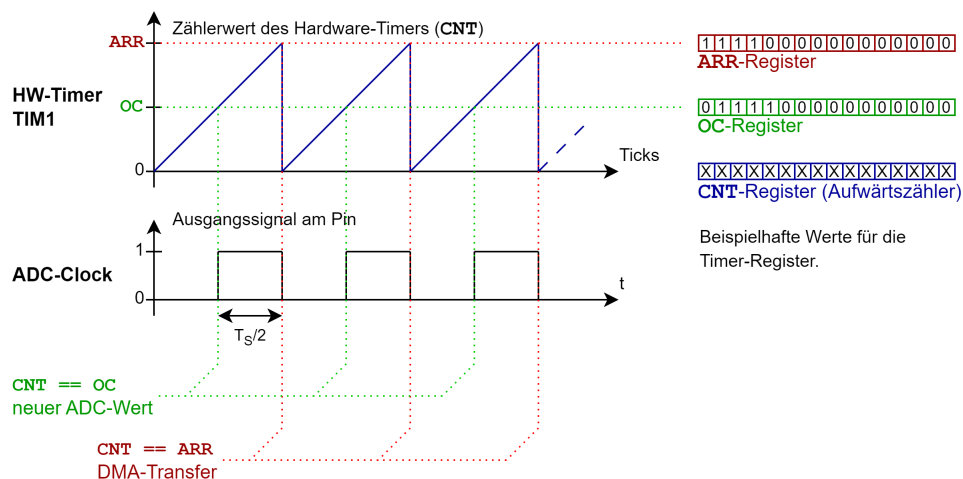


Abbildung 28: Zeitdiagramm und Registerübersicht des Hardware-Timers für die Synchronisation von ADC und DMA (Sampling am μC)

³Bezeichnung für eine Interrupt-Service-Routine (ISR) bei ARM-Cortex- μC .

⁴Die Zeit zwischen der Triggerflanke und DMA-Aktivierung im Exception-Handler ist die Triggerlatenz.

7.1.3. Konzept der Firmware als FSM

Überblick

Die Firmware des USB-Oszilloskops wurde auf der Basis einer Finite State Machine (FSM) zur Steuerung der Betriebszustände erstellt. Dieser Ansatz ermöglicht eine klare Trennung der verschiedenen Funktionsphasen sowie eine strukturierte und erweiterbare Implementierung. Die FSM verarbeitet asynchrone Ereignisse (Events; Präfix EV_), die aus unterschiedlichen Quellen stammen können, wie z.B.:

- Benutzereingaben (z. B. Button auf dem Board),
- Befehle der PC-Anwendung über USB,
- interne Signale der Hardware, wie z. B. Abschluss eines DMA-Transfers.

Die fachlichen Grundlagen zu FSMs wurden bereits in Unterabschnitt 3.4 erläutert.

Um diese Ereignisse effizient zu verarbeiten, wird eine Event-Queue verwendet. Jedes Event wird in diese Warteschlange (Queue) eingereiht („enqueued“). Die FSM verarbeitet die Ereignisse dann sequenziell, wodurch sichergestellt wird, dass auf asynchrone Signale deterministisch reagiert wird.

Architektur der FSM

Die FSM besteht aus mehreren definierten Zuständen, die jeweils einer bestimmten Betriebsphase des Oszilloskops entsprechen. Jeder Zustand wird durch eine eigene Funktion realisiert. Diese Funktionen beinhalten drei Hauptbereiche:

1. *Entry-Block:*

Wird beim Eintritt in einen Zustand ausgeführt (z. B. Initialisierungen, Statusmeldungen).

2. *Event-Handling:*

Reaktion auf ankommende Events innerhalb des aktuellen Zustands.

3. *Exit-Block:*

Wird beim Verlassen des Zustands ausgeführt (z. B. Aufräumarbeiten).

Das State Diagram (siehe Abbildung 29) visualisiert die Abfolge der Firmwarezustände sowie die möglichen Übergänge basierend auf bestimmten Events. Die Hauptzustände sind Tabelle 1 zu entnehmen.

Zustand	Beschreibung
Init	Initialisierung der Hardware-Peripherie. Wird nur einmal beim Start ausgeführt.
Idle	Warten auf Konfigurationsbefehle oder Startsignal der PC-Anwendung.
Acquisition Start	Aktivieren des Triggers und Starten der Datenerfassung.
Acquisition Done	Abschluss der Datenerfassung, Vorbereitung der Daten für die Übertragung.
Data Transmission	Übertragung der erfassten Daten an den PC.
Wait for ACK	Warten auf Bestätigung vom PC, ob ein weiterer Erfassungsvorgang gestartet werden soll.

Tabelle 1: Übersicht über die Hauptzustände der Firmware-FSM

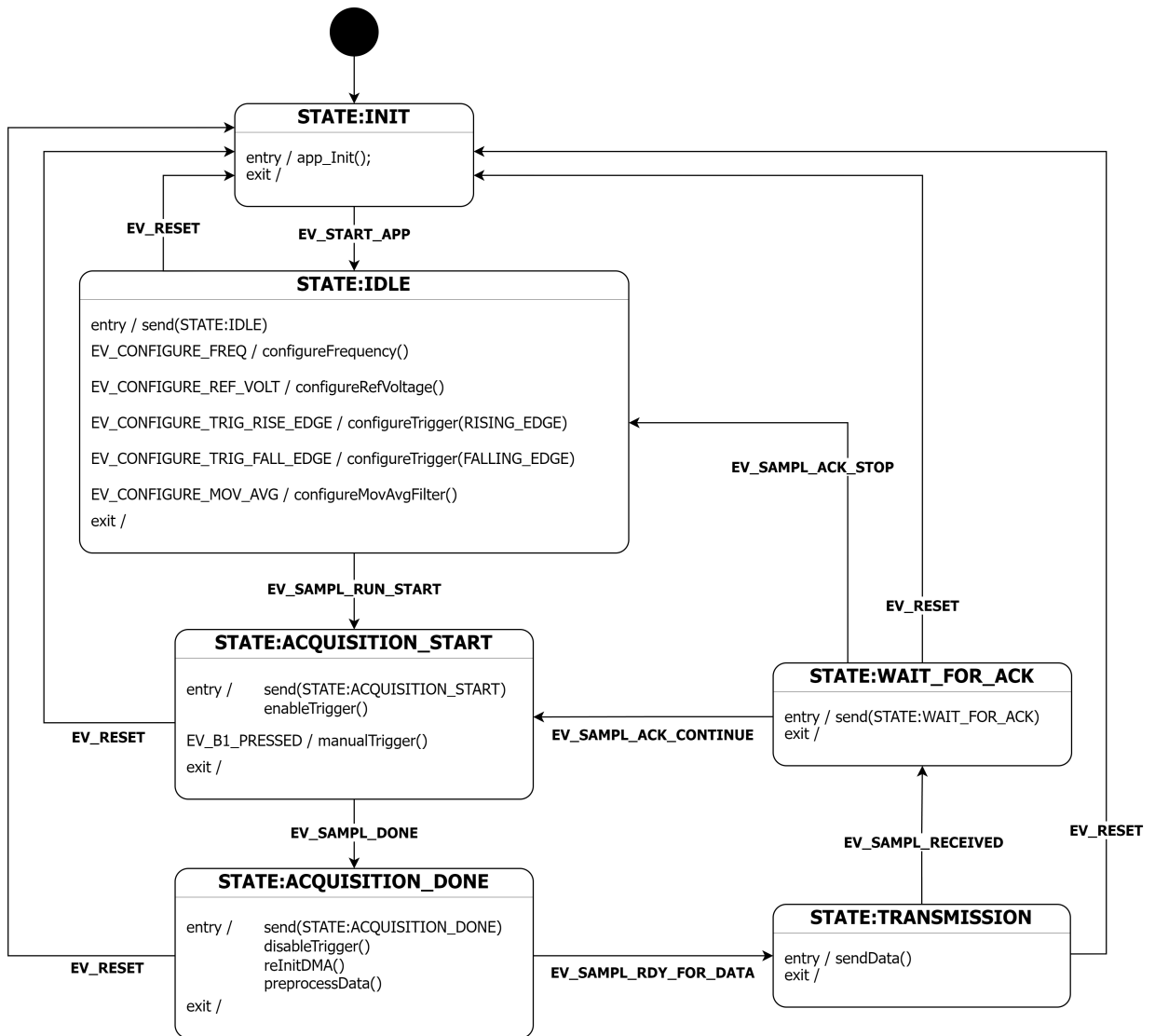


Abbildung 29: Zustandsdiagramm der Firmware

Event-Queue

Die FSM verarbeitet Ereignisse über eine zentrale Event-Queue. Dies erlaubt eine entkoppelte Kommunikation zwischen verschiedenen Modulen der Firmware und stellt sicher, dass Ereignisse nicht verloren gehen, auch wenn mehrere nahezu gleichzeitig auftreten.

Prinzipieller Ablauf:

1. Ein Modul erkennt ein Ereignis (z. B. Button-Druck, DMA-Abschluss, USB-Befehl).
2. Das Ereignis wird als Event-Code in die Queue geschrieben („enqueued“).
3. Die Hauptschleife der Firmware liest fortlaufend Events aus der Queue („dequeued“) und leitet sie an die FSM weiter.
4. Die FSM entscheidet anhand des aktuellen Zustands und des Event-Codes, wie reagiert wird und ob ein Zustandswechsel nötig ist.

Vorteile dieses Ansatzes:

- Asynchrones Verhalten wird klar handhabbar.
- Saubere Trennung zwischen Ereigniserzeugung und -verarbeitung.
- Erweiterbar für zukünftige Ereignisse, ohne die Hauptarchitektur zu ändern.

Zustandsübergänge

Die wichtigsten Zustandsübergänge lassen sich wie folgt zusammenfassen:

- *Systemstart:*
Init → Idle nach erfolgreicher Initialisierung.
- *Start der Datenerfassung:*
Idle → Acquisition Start nach Empfang des Startbefehls von der PC-Anwendung.
- *Abschluss der Datenerfassung:*
Acquisition Start → Acquisition Done nach Abschluss der DMA-Operation.
- *Übertragung der Daten:*
Acquisition Done → Data Transmission sobald PC signalisiert, dass er bereit ist.
- *ACK-Verarbeitung:*
 - Wait for ACK → Acquisition Start für weitere Messung.
 - Wait for ACK → Idle für Beendigung der Erfassung.

7.1.4. Preprocessing-Konzept

Umrechnung 12bit signed (ADC) in 16bit signed

Die Übertragung der Abtastwerte erfolgt in Einheiten von Halfwords (2 Byte). Da die Rohdaten des ADCs als 12-Bit signed Integer vorliegen, werden sie vor der Übertragung in einen gängigen Datentyp umgerechnet. Verwendet wird dabei ein 16-Bit signed Integer (`int16_t` in C), da ein nativer 12-Bit Integer-Datentyp nicht existiert.

Durch diese Umwandlung können die Daten auf der PC-Seite direkt weiterverarbeitet werden, ohne dass zusätzliche komplexe Konvertierungsschritte erforderlich sind. Insbesondere kann die Empfangssoftware (Python-Anwendung) die empfangenen Werte unmittelbar als 16-Bit signed Integer interpretieren und anschließend in den gewünschten Zieldatentyp umwandeln. Ein weiterer Vorteil der Umrechnung liegt in der verbesserten internen Weiterverarbeitung auf Mikrocontroller-Seite. So konnten die Daten vor dem Versand beispielsweise effizient für die Anwendung eines gleitenden Mittelwertfilters aufbereitet werden (vgl. nächster Abschnitt).

Nachfolgend sind mögliche Algorithmen als Python-Code beschrieben:

1. Sign-Extend (Shift-Methode)

```
1 def sign_extend_12_to_16(value12: int) -> int:
2     # 12-Bit Zahl um 4 nach links, dann arithmetisch nach rechts
3     return (value12 << 4) >> 4
```

Listing 1: Sign-extend (Arithmetisches Shift schiebt das MSB nach)

2. Zweierkomplement rückwärts: invertieren $\rightarrow +1$

```
1 def twos_complement_backward_invert_plus1(value12: int) -> int:
2     if value12 & 0x800: # Vorzeichenbit pruefen
3         inverted = (~value12) & 0xFFF # Nur 12 Bits
4         magnitude = inverted + 1
5         return -magnitude
6     else:
7         return value12
```

Listing 2: Zweierkomplement rückwärts aufgelöst.

3. Zweierkomplement rückwärts: -1 → invertieren

```
1 def twos_complement_backward_minus1_invert(value12: int) -> int:
2     if value12 & 0x800: # Vorzeichenbit pruefen
3         decreased = (value12 - 1) & 0xFFF
4         magnitude = (~decreased) & 0xFFF
5         return -magnitude
6     else:
7         return value12
```

Listing 3: Alternative zu 2.: Erst -1 → dann invertieren.

Gleitender Mittelwertfilter

In Unterabschnitt 3.6 wurden bereits die Grundlagen zur digitalen Filterung von Signalen geliefert. Im Projekt wurde die DSP mittels *gleitendem Mittelwertfilter* umgesetzt, welcher zufälliges Rauschen aus dem Signal herausfiltern kann. Dies ist insbesondere bei verrauschten Signalquellen mit kleiner Ausgangsamplitude relevant. In Listing 4 ist ein Filteralgorithmus beschrieben, der einen kumulativen gleitenden Mittelwertfilter umsetzt.

```
100 'MOVING AVERAGE FILTER
110 'This program filters 5000 samples with a 101 point moving
120 'average filter, resulting in 4900 samples of filtered data.
130 '
140 DIM X[4999] 'X[ ] holds the input signal
150 DIM Y[4999] 'Y[ ] holds the output signal
160 '
170 GOSUB XXXX 'Mythical subroutine to load X[ ]
180 '
190 FOR I% = 50 TO 4949 'Loop for each point in the output signal
200 Y[I%] = 0 'Zero, so it can be used as an accumulator
210 FOR J% = -50 TO 50 'Calculate the summation
220 Y[I%] = Y[I%] + X(I%+J%)
230 NEXT J%
240 Y[I%] = Y[I%]/101 'Complete the average by dividing
250 NEXT I%
260 '
270 END
```

Listing 4: Pseudocode: Kumulativer Moving-Average-Filter aus [Smi99, S. 278]

Ein Problem der kumulativen Variante ist die lange Ausführungsdauer, da für jeden gefilterten Abtastwert eine große Summe (abhängig von der Fenstergröße) gebildet werden muss. Dies kann mitunter eine große Verzögerungszeit vor der Übertragung zur Folge haben und damit einer geringen Refreshrate des dargestellten Signals. Eine rekursive Variante des Filters muss nur einmal eine große Summe über das erste Fenster bilden. Bei den

nachfolgenden Werten wird der aus dem Fensterbereich entfallene Wert von der Summe subtrahiert und der neue Wert im Fensterbereich addiert. Die Berechnung für jeden Abtastwert (abgesehen vom ersten) ist selbst für große Fenstergrößen gleich schnell. Der Algorithmus ist Listing 5 zu entnehmen (vgl. [Smi99, S. 277-284]).

```

100 'MOVING AVERAGE FILTER IMPLEMENTED BY RECURSION
110 'This program filters 5000 samples with a 101 point moving
120 'average filter, resulting in 4900 samples of filtered data.
130 'A double precision accumulator is used to prevent round-off drift.
140 '
150 DIM X[4999] 'X[ ] holds the input signal
160 DIM Y[4999] 'Y[ ] holds the output signal
170 DEFDBL ACC 'Define the variable ACC to be double precision
180 '
190 GOSUB XXXX 'Mythical subroutine to load X[ ]
200 '
210 ACC = 0 'Find Y[50] by averaging points X[0] to X[100]
220 FOR I% = 0 TO 100
230 ACC = ACC + X[I%]
240 NEXT I%
250 Y[50] = ACC/101
260 ' 'Recursive moving average filter (Eq. 15-3)
270 FOR I% = 51 TO 4949
280 ACC = ACC + X[I%+50] - X[I%-51]
290 Y[I%] = ACC/101
300 NEXT I%
310 '
320 END

```

Listing 5: Pseudocode: Rekursiver Moving-Average-Filter aus [Smi99, S. 284]

7.2. Implementierung

Die nachfolgende Beschreibung gibt einen Überblick über die Umsetzung der Firmware. Hierbei handelt es sich um das Programm des verwendeten Mikrocontrollers STM32F767ZI. Für genaue Informationen zur Implementierung sei auf die Quellcode-Dateien verwiesen.

7.2.1. Allgemeines zur verwendeten Toolchain

Überblick über die Entwicklungsumgebung

Für die Entwicklung der Firmware wurde die STM32CubeIDE verwendet. Diese ist ein von STMicroelectronics bereitgestelltes integriertes Entwicklungswerkzeug, das IDE-Funktionalität, Code-Generierung, Debugging und Konfigurationswerkzeuge vereint. Sie basiert auf dem Eclipse/CDT-Framework und nutzt den GCC-Compiler sowie GDB für Debugging. Die IDE integriert die Funktionalitäten von STM32CubeMX, so dass die *Hardwarekonfiguration* (Pins, Peripherie, Clock-Tree etc.) *direkt im Projekt erstellt* und entsprechende *Initialisierungscode*s *automatisch generiert* werden können. Über STM32CubeMX werden u.a. der Pinout & Configuration, der Clock-Tree (Takteinstellungen) und Middleware (z. B. USB-Device) konfiguriert [STm25b]. In unserem Projekt diente das Tool besonders zur Konfiguration des Clock-Tree und der USB-Device-Treiber – d.h. die Taktquellen und -verzweigungen sowie die USB-Funktionalität wurden über CubeMX festgelegt.

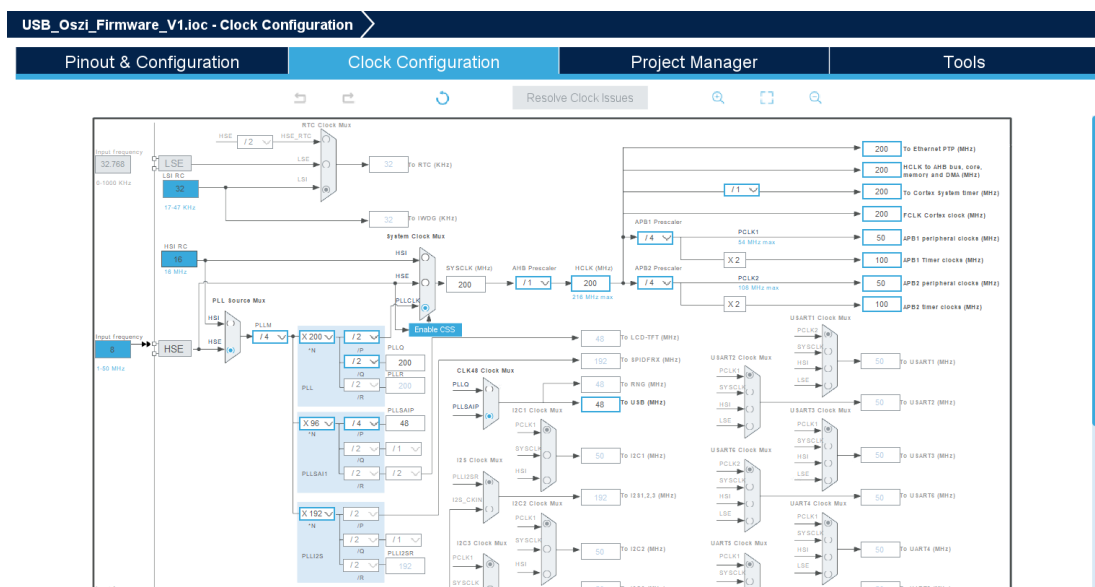


Abbildung 30: Clock-Tree-Ansicht in STM32CubeMX

Build-Prozess

Der Build-Prozess in STM32CubeIDE folgt dem klassischen Ablauf von C-basierten Embedded-Projekten. Laut “STM32CubeIDE user guide“ ([STm25b, S. 41ff]) umfasst dieser:

1. Preprocessing – Auflösen von Makros und Includes
2. Kompilierung – Übersetzen in Objektcode
3. Assemblierung – Erzeugen der Objektdateien
4. Linking – Verknüpfen mit Hilfe eines Linkerskripts
5. Erzeugung des finalen Images / Executable

Der Linker nutzt ein Linker-Skript (.ld), das die Speicherbereiche (z. B. Flash, RAM) festlegt – dieses Skript wird ebenfalls durch das Projekt generiert bzw. angepasst. Das Endprodukt (z. B. im ELF-Format) liegt in den Binaries des Projekts vor.

Debugging

Die STM32CubeIDE bietet einige Debug-Funktionen unter Verwendung von GDB und dem ST-Link-Probe (auf dem Entwicklungsboard integriert). Zu den typischen Debug-Features gehören:

- Setzen von Breakpoints
- Schrittweise Ausführung (Step-In, Step-Over, Step-Out)
- Überwachung von Variablen und Ausdrücken (Watch Window)
- Einsicht in Register, Peripherieregister und Speicher

Projektstruktur

Die standardmäßig von STM32CubeIDE & CubeMX generierte Projektstruktur folgt einem festen Schema. Wichtige Komponenten sind:

- `Projektname.ioc` – die CubeMX-Konfigurationsdatei
- `Core/` – enthält `main.c`, Systeminitialisierung, Applikationslogik
- `Drivers/` – enthält HAL-Treiber, ggf. LL (Low-Layer) Treiber und CMSIS
- `Includes/` – Pfaddefinitionen für Headerdateien
- `Binaries/` (oder Build-Ausgabeordner) – kompiliertes Ergebnis (z. B. ELF)
- `Debug/` – temporäre Build-Artefakte (Objektdateien, LST, Map-Dateien)
- Linker-Skripte (z.B. `.ld`) – Speicherlayout und Segmentdefinitionen

Rolle der HAL und Low-Layer-Treiber (LL)

Für die STM32F7-Controllerfamilie stellt ST eine HAL (Hardware Abstraction Layer) bereit, die als Zwischenschicht dient, um Hardwarezugriffe abstrahiert zu kapseln. Die HAL-APIs prüfen zusätzlich Laufzeitbedingungen und Input-Parameter, um Robustheit zu erhöhen. Die LL-Treiber (Low-Layer) bieten einen direkteren Zugriff, nahe an den Hardware-Registeroperationen, mit geringerer Abstraktion und somit höherer Effizienz, aber weniger Komfort. In Kombination ermöglichen HAL und LL ein gutes Gleichgewicht zwischen Portierbarkeit, Lesbarkeit und Performance [STm23].

Die HAL-Funktionen wurden im Projekt in Bereichen verwendet, in denen STM32CubeMX automatisch Initialisierungscode erzeugt hat (z.B. zur Konfiguration des Taktes in der Funktion `SystemClock_Config()` im Sourcefile `main.c`).

7.2.2. Übersicht über die Applikations-Sourcefiles

Im Folgenden wird die Funktion und der Zweck der Einzel-Sourcefiles, in welchen der Hauptteil der Applikation implementiert ist, erläutert.

app.c / app.h

Das Modul `app` beinhaltet die Applikationslogik und stellt Funktionen zur Initialisierung der Firmware bereit. Es ist für die Steuerung der Status-LEDs sowie die Hardwarevorbereitung für den Messbetrieb verantwortlich. Außerdem sind hier die Benutzerfunktionen zur Kommunikation über USB definiert..

Wichtige Funktionalitäten:

- `app_Init()` – Initialisiert die Anwendung und richtet benötigte Hardwarekomponenten ein.
- USB-Benutzerfunktionen (z.B. `sendCommand()`, `sendData()`, ...)

Abhängigkeiten: Dieses Modul verwendet die Event-Makros von `fsm.c/fsm.h`.

queue.c / queue.h

Dieses Modul stellt eine *FIFO-Queue* für Events bereit. Es dient als Schnittstelle zwischen asynchronen Ereignissen (z. B. Interrupts, USB-Kommandos) und der FSM.

Wichtige Funktionen:

- `enqueueEvent(event)` – Fügt ein neues Event am Ende der Queue ein.
- `dequeueEvent()` – Gibt das älteste Event zurück und entfernt es aus der Queue.

Abhängigkeiten: Dieses Modul arbeitet unabhängig von der Hardware und hat keine direkten HAL-Bezüge.

fsm.c / fsm.h

Dieses Modul implementiert die *Finite State Machine (FSM)* zur Ablaufsteuerung der Firmware. Jeder Zustand der FSM wird durch eine eigene Funktion abgebildet. Zustandswechsel erfolgen durch die Funktion `fsm_transition()`.

Kernkonzepte:

- Ereignisgesteuerte Zustandsmaschine mit definierten `EV_ENTRY`-, `EV_EXIT`- und regulären Events.
- Verarbeitung externer Befehle und interner Hardware-Events über die Event-Queue.
- Klare Trennung zwischen Zustandslogik und Event-Handling.

Abhängigkeiten: Dieses Modul verwendet Funktionen aus `queue.h` zur Event-Verarbeitung sowie Funktionen aus `app.h` zur Hardwaresteuerung.

Eine nähere Erläuterung zum Modul findet sich in Unterunterabschnitt 7.2.3.

main.c / main.h

Die Datei `main.c` bildet den Einstiegspunkt der Firmware. Hier werden Systeminitialisierung, HAL-Setup und der Start der FSM durchgeführt.

Zentrale Aufgaben:

- Aufruf von `HAL_Init()` und der SystemClock-Konfiguration.
- Initialisierung der FSM mit dem Startzustand.

Abhängigkeiten: Starke Kopplung an `fsm.c` sowie HAL-Treiber.

stm32f7xx_it.c / stm32f7xx_it.h

Dieses Modul enthält alle Interrupt-Handler des Systems. Bei eintreffenden Interrupts werden entsprechende Events erzeugt und an die Queue übergeben.

Beispiele:

- `DMAx_IRQHandler()` – Fügt ein Event `EV_SAMPL_DONE` in die Queue ein.
- `EXTIx_IRQHandler()` – Reagiert auf externe Trigger-Signale oder Button-Eingaben.

startup_stm32f767zitx.s

Diese Assembly-Datei enthält den Start-Up-Code. Hier werden der Stackpointer initialisiert, die Vektortabelle definiert und die `Reset_Handler`-Funktion aufgerufen.

Zentrale Aufgaben:

- Definition des Exception-Vector-Table.
- Aufruf von `SystemInit()` und anschließend `main()`.

7.2.3. FSM

Die Implementierung der Firmware basiert auf einer ereignisgesteuerten FSM-Architektur, die sich an einem Open-Source-Beispielprojekt orientiert, welches unter https://github.com/NinjaNymo/C_StateMachines/ verfügbar ist.

FSM-Modul (`fsm.c` / `fsm.h`)

Das Modul `fsm` implementiert die in Unterabschnitt 3.4 beschriebene Finite State Machine (FSM). Die Firmware ist als deterministischer endlicher Automat (DEA) ausgeführt, konkret als Moore-Automat, bei dem die Ausgaben nur vom aktuellen Zustand abhängen. Dies ermöglicht ein vorhersagbares, deterministisches Verhalten.

Aufbau

Jeder Zustand wird durch eine eigene C-Funktion repräsentiert, die drei Event-Typen verarbeitet: `EV_ENTRY` (Aktionen beim Eintritt), `EV_EXIT` (Aktionen beim Verlassen) sowie zustandsspezifische Events. Übergänge zwischen Zuständen erfolgen über die Funktion `fsm_transition()`, die automatisch `EV_EXIT` und `EV_ENTRY` auslöst. Die Funktion `fsm_dispatch()` leitet eingehende Events an die aktuelle Zustandsfunktion weiter.

Zentrale Zustände

- `fsm_state_init()`: Einmalige Initialisierung der Hardware (`app_Init()`), Übergang zu `IDLE`.
- `fsm_state_idle()`: Konfiguration, Start der Messung bei Event `EV_SAMPL_RUN_START`.
- `fsm_state_acquisition_start()`: Aktivieren des Triggers und Start der Datenerfassung.
- `fsm_state_acquisition_done()`: Aufräumen, Datenvorbereitung für Übertragung.
- `fsm_state_data_transmission()`: Versand der Messdaten an den PC.
- `fsm_state_wait_for_ACK()`: Warten auf PC-Bestätigung für Neustart oder Rückkehr in `IDLE`.

Bezug zur Theorie

Die Implementierung bildet die Theorie direkt ab:

- Zustandsmenge S : Zustandsfunktionen `fsm_state_*`
- Ereignismenge E : definierte Events (z.B. `EV_SAMPL_DONE`)
- Übergangsfunktion δ : `fsm_transition()`
- Ausgabefunktion λ : Aktionen in `EV_ENTRY`-Blöcken

Die FSM ist damit klar strukturiert, erweiterbar und unterstützt sauberes Debugging.

Queue-Modul (`queue.c` / `queue.h`)

Das Queue-Modul verwaltet alle asynchron auftretenden Ereignisse. Es stellt eine FIFO-Queue bereit, die als Bindeglied zwischen Interrupts, PC-Befehlen und der FSM dient. Ereignisse werden beim Auftreten erst einmal *eingereicht* (`enqueueEvent()`) und später von der FSM sequenziell *abgearbeitet* (`dequeueEvent()`).

Wichtige Funktionen

- `enqueueEvent(event)`: Fügt ein neues Event am Ende der Queue hinzu (z.B. aus Interrupt).
- `dequeueEvent()`: Gibt das älteste Event zurück und entfernt es.

Vorteile

- Ereignisse gehen nicht verloren, auch wenn mehrere gleichzeitig auftreten.
- Entkopplung der Event-Erzeugung von deren Verarbeitung.
- Deterministische, nachvollziehbare Abarbeitung in der FSM.

7.2.4. Preprocessing

Der Algorithmus zur Umrechnung 12bit signed (ADC) in 16bit signed aus Listing 3 wurde in der Funktion `preprocessData()` in `app.c` umgesetzt. Hier erfolgt auch die optionale Anwendung des gleitenden Mittelwertfilter durch Aufruf der Funktion `applyMovingAverageFilter()` in `app.c`. Die Berechnung erfolgt in der aktuellen Implementierung rekursiv nach dem Algorithmus in Listing 5.

Hinweis:

Aufgrund der späten Fertigstellung im Projektverlauf kann der Filter in der Benutzeroberfläche aktuell nicht aktiviert werden, da keine entsprechenden Interaktionselemente für die Konfiguration verfügbar sind. Ein Test des Filters konnte aber durchgeführt werden (siehe Unterunterabschnitt 7.3.3).

7.3. FW-Test

7.3.1. Abtastung

Die Funktionsweise der Abtastung (gemäß Unterunterabschnitt 7.1.2) wurde zunächst mit dem integrierten Debugging-Tool der IDE überprüft. Ein wichtiger Schritt war die Überprüfung des konfigurierten DMA-Streams mit definiertem Start und Ende.

Hierzu wurden die DMA-Transfers manuell gestartet und das Ergebnis in der “Expressions“-Debug-Ansicht der IDE verifiziert. Mit dieser können die tatsächlich im SRAM abgelegten Werte betrachtet werden. Zunächst wurden durch Pull-Up- und Pull-Down-Widerstände definierte Pegel angelegt, um Testmuster am Eingangsdatenregister der GPIOs der parallelen Schnittstelle zwischen ADC und μ C zu erzeugen.

Nachdem die FSM implementiert wurde, konnte auch explizit im Zustand nach Beendigung der Abtastung (STATE:ACQUISITION_DONE) ein Breakpoint gesetzt und die Abtastwerte auf Plausibilität überprüft werden (siehe Abbildung 31).

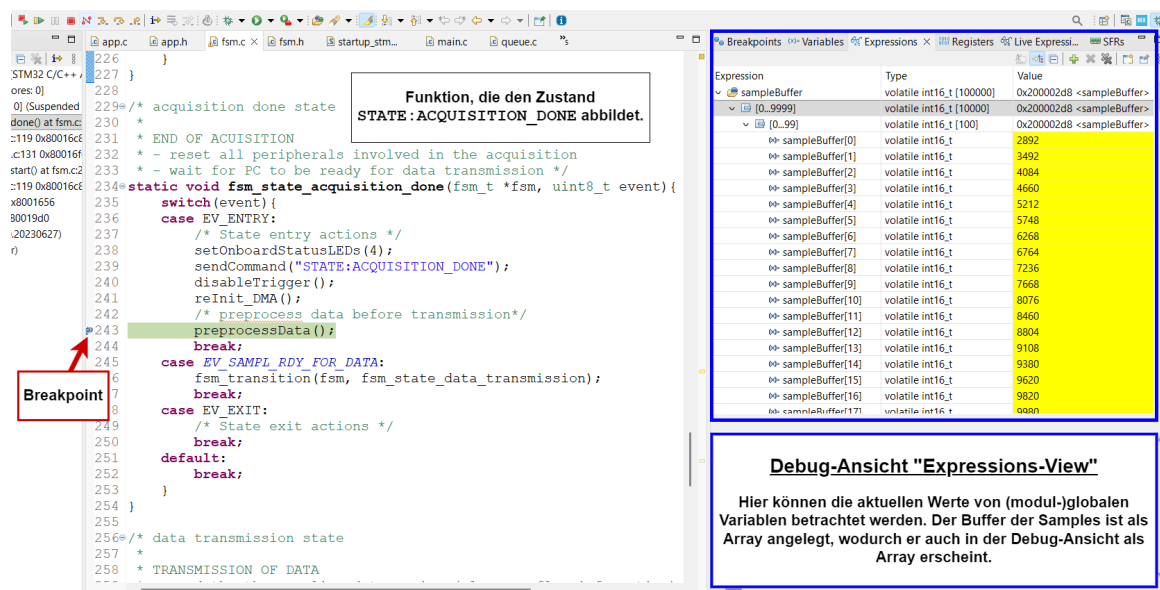


Abbildung 31: Expression-Debug-Ansicht zur Überprüfung der Abtastwerte

7.3.2. FSM

Bevor die Funktionsweise der State-Machine überprüft werden konnte, wurde das Kommunikationskonzept (siehe Unterabschnitt 8.2) umgesetzt. Hierdurch war eine externe Steuerung der Firmware-FSM möglich, wie es auch in der Endanwendung vorgesehen war. Die Steuerung erfolgte mittels String-Befehlen, ein genauer Ablauf kann Abbildung 37 entnommen werden. Bei der USB-Spezifikation wurde sich für die USB Communication Device Class (CDC) entschieden (siehe Unterabschnitt 8.1), wodurch der μ C einen virtuellen COM-Port (VCP) für die Kommunikation zur Verfügung stellt (als COM-Port auf dem Rechner sichtbar).

Ausgehend von dieser Grundlage wurde der Ablauf zunächst durch manuelles Senden der einzelnen Befehle und das Empfangen der entsprechenden Antworten (Acknowledgement, Daten) überprüft. Hierzu wurde das Terminal-Programm *hterm*⁵ genutzt, welches die Verbindung mit einem COM-Port ermöglicht und einen zweckmäßigen Funktionsumfang bietet (siehe Abbildung 32).

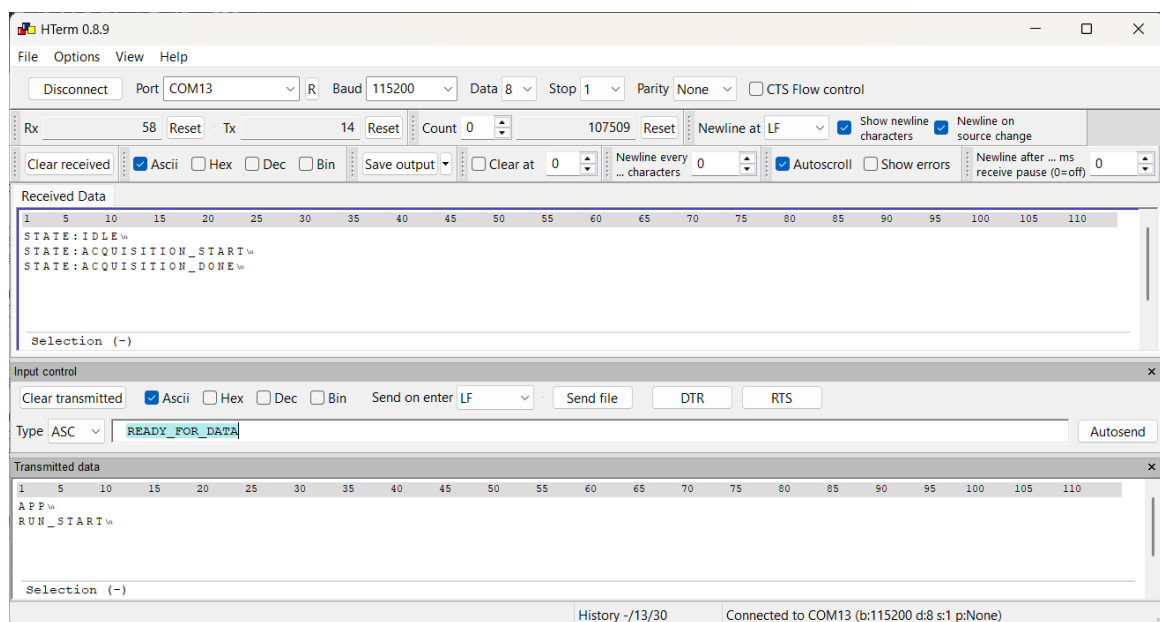
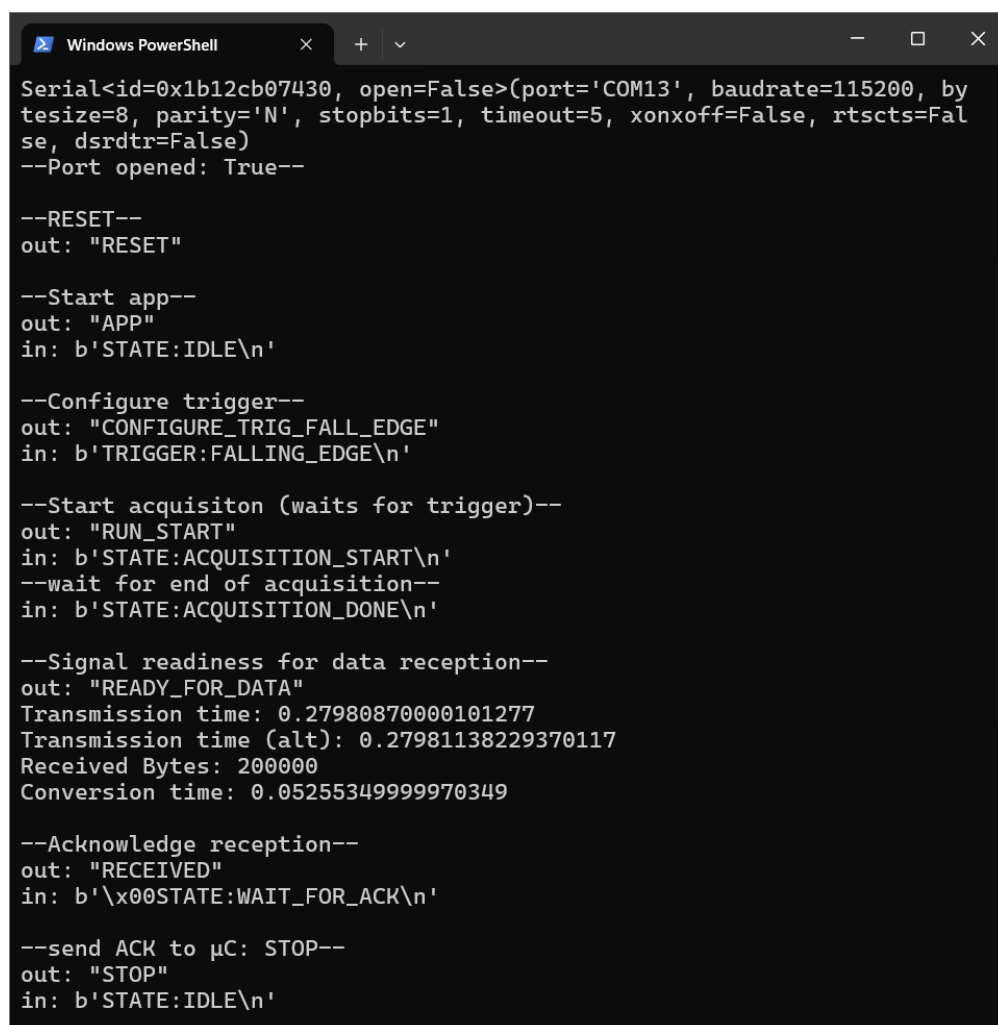


Abbildung 32: Kommunikationsablauf im Terminal-Programm *hterm*

⁵www.der-hammer.info/pages/terminal.html (benutzte Version: 0.8.9)

Nachdem die Auswahl des SW-Frameworks feststand (Python-Framework siehe Unterunterabschnitt 9.1.1) und sich auf die Nutzung von USB CDC geeinigt wurde, wurde als rechnerseitige Schnittstelle, das `pyserial`-Paket ausgewählt, welches eine große Bandbreite an Funktionen für die serielle Kommunikation bereitstellt. Da durch das manuelle Senden von Einzelbefehlen mit *hterm* keine realistischen Bedingungen bezüglich Timing gegeben waren, wurden mehrere Testskripts in Python angefertigt, mit denen die Funktion der FSM tiefergehend überprüft wurde. Eine beispielhafte Konsolenausgabe eines Testskripts kann Abbildung 33 entnommen werden. Anhand dieser Tests konnten bereits vor der Zusammenführung von Software und Firmware einzelne Fehler ausgebessert werden.



```
Windows PowerShell
Serial<id=0x1b12cb07430, open=False>(port='COM13', baudrate=115200, bytesize=8, parity='N', stopbits=1, timeout=5, xonxoff=False, rtscts=False, dsrdtr=False)
--Port opened: True--

--RESET--
out: "RESET"

--Start app--
out: "APP"
in: b'STATE:IDLE\n'

--Configure trigger--
out: "CONFIGURE_TRIG_FALL_EDGE"
in: b'TRIGGER:FALLING_EDGE\n'

--Start acquisition (waits for trigger)--
out: "RUN_START"
in: b'STATE:ACQUISITION_START\n'
--wait for end of acquisition--
in: b'STATE:ACQUISITION_DONE\n'

--Signal readiness for data reception--
out: "READY_FOR_DATA"
Transmission time: 0.27980870000101277
Transmission time (alt): 0.27981138229370117
Received Bytes: 200000
Conversion time: 0.05255349999970349

--Acknowledge reception--
out: "RECEIVED"
in: b'\x00STATE:WAIT_FOR_ACK\n'

--send ACK to µC: STOP--
out: "STOP"
in: b'STATE:IDLE\n'
```

Abbildung 33: Beispielhafte Konsolenausgabe bei Ablauf eines Python-Testskripts

7.3.3. Gleitender Mittelwertfilter

Zum Test des gleitenden Mittelwertfilters wurden in einer vorläufigen Version der Benutzeroberfläche die vorverarbeiteten übertragenen Daten geplottet. Der Filter wurde manuell durch Setzen einer Zustandsvariable in der Firmware aktiviert. Die Abbildung 34 zeigt das empfangene Eingangssignal ohne Filter (Rechtecksignal mit überlagertem Rauschen). Abbildung 35 zeigt das Signal nach Aktivierung des Filters.

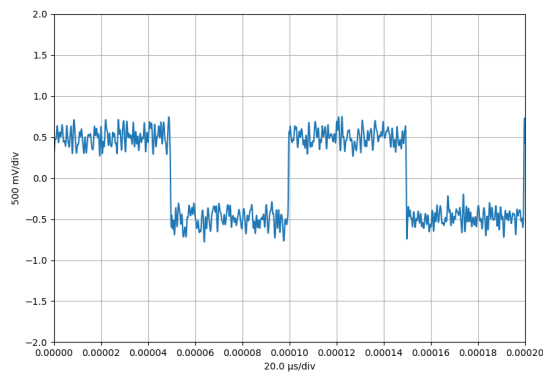


Abbildung 34: Signalverlauf ohne Filter
10kHz Rechtecksignal mit überlagertem Rauschen (1Vpp; Bandbreite: 1MHz)

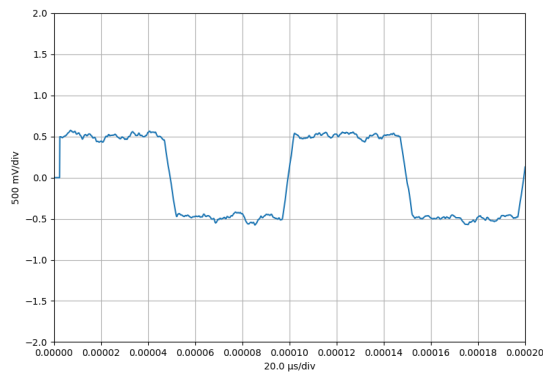


Abbildung 35: Signalverlauf rekursiver gleitender Mittelwertfilter (M=51)
10kHz Rechtecksignal mit überlagertem Rauschen (1Vpp; Bandbreite: 1MHz)

Es ist zu erkennen, dass die Anwendung des Filters, wie erwartet, eine Rauschunterdrückung bewirkt. Weiterhin wird aber auch die Flankensteilheit des Signals verringert.

8. Schnittstelle Firmware - Software

8.1. USB Communication Device Class (CDC)

8.1.1. Grundlagen zu USB und den verwendeten Schnittstellen

Universal Serial Bus (USB) ist ein Host-gesteuertes Kommunikationsprotokoll, bei dem der Host (typischerweise ein PC) die Kommunikation initiiert und die Endgeräte („Devices“) auf Anfragen reagieren. Jedes USB-Gerät wird durch einen Device Descriptor beschrieben, der unter anderem Informationen wie die unterstützte USB-Version, die Hersteller- und Produkt-ID (VID und PID) sowie die verwendeten Klassen und Konfigurationen enthält [USB25b].

Kommunikationskanäle werden in USB durch sogenannte Endpoints realisiert. Zwischen diesen Endpoints werden sogenannte virtuelle Datenkanäle aufgebaut, über die der Host mit dem Device Daten austauscht. Jeder Endpoint besitzt dabei eine Richtung (IN für Device → Host, OUT für Host → Device) und ist in einem Interface organisiert. Der Zugriff erfolgt nicht direkt, sondern über sogenannte Pipes, die auf Host-Seite eingerichtet werden [USB25b, Kap. 5]. Die Kommunikation ist beidseitig (Host und Device) in Schichten eingeteilt, die bestimmte Aufgaben haben (vgl. mit mit ISO-OSI-Modell⁶). In Abbildung 36 ist die grobe Schichteneinteilung dargestellt.

Beim verwendeten Mikrocontroller *STM32F767ZI* wurden die von STMicroelectronics bereitgestellten *USB-CDC-Treiber* verwendet, welche sich über das Konfigurationstool *STM32CubeMX* automatisch generieren lassen. CubeMX erstellt hierbei die notwendigen Deskriptoren (z. B. Device Descriptor, Configuration Descriptor, Interface Descriptor), konfiguriert die Endpoints und bindet die Middleware der USB-Device-Library von ST ein [STm19].

⁶ISO/IEC 7498-1:1994

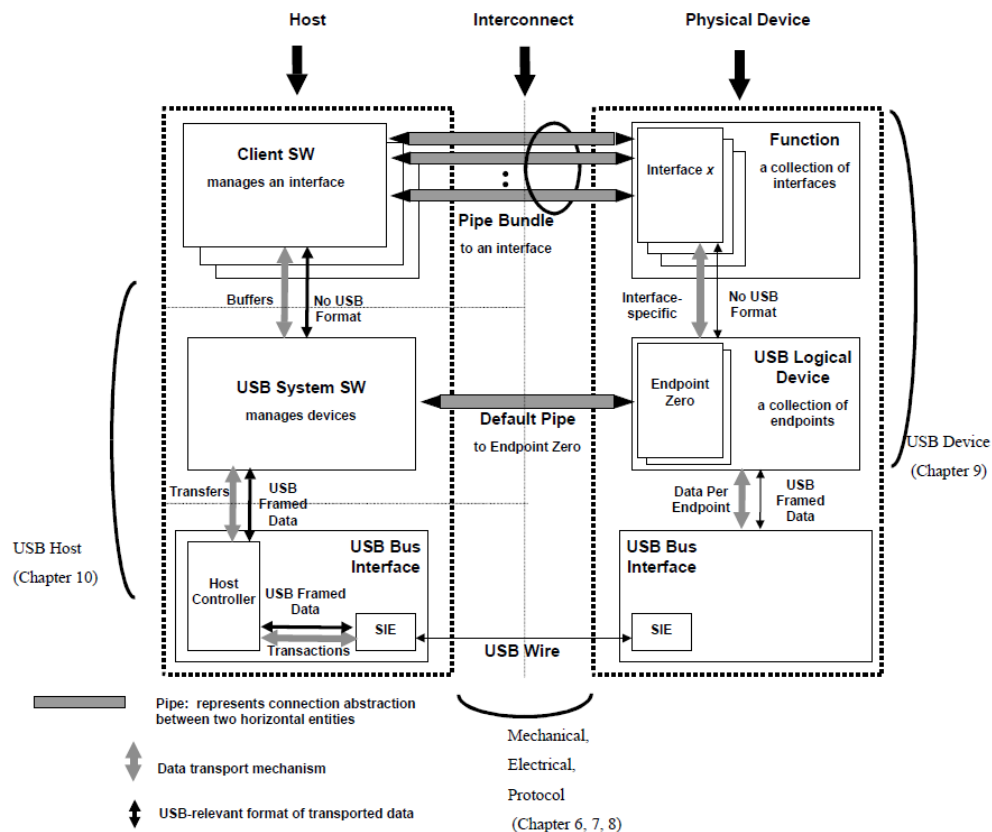


Abbildung 36: Host- und Device-Ansicht aus der USB-Spezifikation
(Figure 5-9 aus [USB25b])

8.1.2. Auswahl und Begründung für USB CDC

Eine direkte Kommunikation über eigene Endpoints hätte die Entwicklung eines Protokolls und insbesondere die Implementierung eines benutzerdefinierten Gerätetreibers auf Host-Seite erforderlich gemacht. Dies ist aufwändig und fehleranfällig, da die USB-Spezifikation komplex ist und die Treiberentwicklung tiefes Wissen über Betriebssystem-APIs und Kernel-Schnittstellen erfordert [USB25a].

Stattdessen wurde die USB Communication Device Class (CDC) gewählt. Sie emuliert eine serielle Schnittstelle ("Virtual COM Port") und erlaubt eine einfache Integration in bestehende PC-Software. Ein großer Vorteil ist, dass Windows (ab Windows 10) bereits den integrierten Treiber Usbser.sys für CDC/ACM-Geräte mitliefert und automatisch lädt, solange alle Daten korrekt im USB-Deskriptor gesetzt sind [Mic25].

Die Hauptvorteile der CDC-Auswahl sind:

- Einfache Implementierung mit CubeMX (automatische Generierung von Middleware und Treibergerüst).
- Keine Notwendigkeit eigener PC-Treiber.
- Direkter Zugriff aus PC-Programmiersprachen über gängige serielle Schnittstellen-Bibliotheken.

Ein Nachteil dieser Wahl ist jedoch der zusätzliche Protokoll-Overhead. Die effektive Datenrate der USB-Full-Speed-Schnittstelle (12 Mbit/s) wird dadurch nicht erreicht. Eine speziell implementierte Endpoint-Kommunikation könnte eine höhere Performance ermöglichen, erfordert aber deutlich mehr Entwicklungsaufwand [STm19].

8.1.3. Kommunikationssituation PC ↔ Mikrocontroller

Die Kommunikation erfolgt in der Projektrealisierung wie folgt:

PC-seitig wird ein Python-Skript eingesetzt, das über das Package *pyserial* mit dem virtuellen *COM-Port* kommuniziert. Hierbei öffnet das Skript die *CDC*-Schnittstelle, sendet Steuerkommandos und empfängt Messdaten.

Mikrocontroller-seitig wird die *USB-CDC-Klasse* der *STM32Cube Middleware* genutzt. Die Konfiguration (z. B. Endpoints, Deskriptoren) erfolgte über *STM32CubeMX*.

Die generierte Middleware stellt die Funktionen `CDC_Transmit_FS()` und `CDC_Receive_FS()` bereit, die im Anwendungsprogramm (Dateien `app.c/app.h`) angepasst und aufgerufen wurden.

Der *USB_DEVICE-Treiber* übernimmt die Initialisierung und Verwaltung der USB-Schnittstelle. Das eigentliche Applikationsprogramm ruft diese Funktionen auf, um Daten an den Host zu senden bzw. von ihm zu empfangen.

Damit ergibt sich eine klare Schichtentrennung:

- *USB_DEVICE-Treiber* und Middleware: Low-Level-USB-Verwaltung, Deskriptoren, Endpoints.
- *Applikationscode* (`app.c/.h`): Logik des Oszilloskops, Aufruf von `CDC_Transmit_FS` für Messdaten, Implementierung von Empfangslogik in `CDC_Receive_FS`.
- *PC-Seite* (Python): Verarbeitung der Datenströme über serielle Schnittstellen-Funktionen.

8.2. Konzept der Kommunikation

Der nachfolgenden Abbildung 37 lassen sich die definierten String-Befehle entnehmen, die für die Kommunikation notwendig sind. Weiterhin veranschaulicht das Diagramm einen typischen Kommunikationsablauf (ohne Konfiguration im Idle-Zustand).

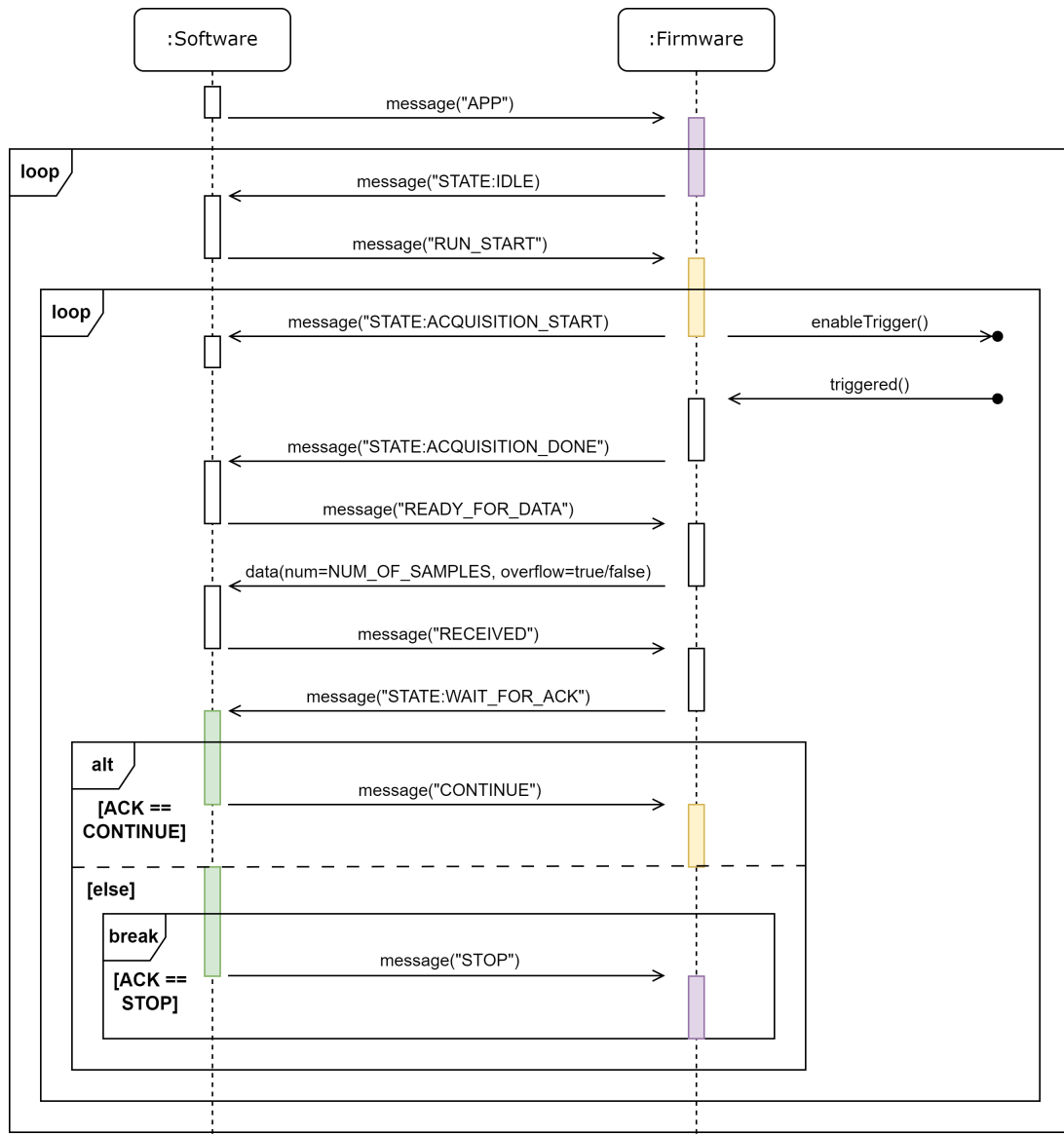


Abbildung 37: UML-Sequenzdiagramm: Kommunikation ohne Konfiguration

9. Software (SW)

9.1. Entwurf

9.1.1. Auswahl des Frameworks

Die Programmiersprache Python wurde für die Umsetzung gewählt, da sie eine breite Unterstützung für wissenschaftliche und hardware-nahe Programmierung bietet. Durch umfangreiche Bibliotheken wie *numpy*⁷ für numerische Auswertung, *matplotlib*⁸ für die grafische Darstellung der Daten, *pyserial*⁹ für Schnittstellenkommunikation sowie leistungsfähige Frameworks für GUI-Entwicklung wie *PySide6*¹⁰, eignet sich Python besonders für prototypische Entwicklung. Python ist eine dynamische Sprache mit einfachen Syntax, so dass Funktionalitäten rasch entwickelt, getestet und iterativ verbessert werden können. Ein weiterer Vorteil ist die Portabilität und einfache Installation. Python ist auf allen wichtigen Betriebssystemen verfügbar und einfach zu installieren. Außerdem können Pakete einfach in einer virtuellen Umgebung gebündelt werden, um den eigenen PC nicht unnötig zu belasten. Alle für unser Projekt benötigten Pakete sind in der Datei *requirements.txt* aufgelistet. Eine Installationsanleitung findet sich im Anhang A.

Die Benutzeroberfläche wurde mit dem Framework *PySide6* realisiert. Dies ist eine speziell für Python abgewandelte Version von Qt. *PySide6* stellt viele Klassen und Objekte zur Verfügung, die es dem Nutzer einfach machen, ohne viel Erfahrung eine Benutzeroberfläche zu programmieren. Der Vorteil gegenüber der Nutzung von Qt für C++ liegt, darin, dass sich der Nutzer nicht selbst um die Speicherverwaltung kümmern muss.

9.1.2. Architektur Gesamtübersicht

Zur Übersichtlichkeit wurde das Programm in mehrere Unterprogramme zerlegt. Abbildung 38 zeigt, welche Unterprogramme, in der Abbildung blau dargestellt, zu welcher logischen Einheit gehören. Zusätzlich zeigt diese die groben Zusammenhänge der einzelnen Einheiten. In orange sind Klassen des Programms *oszi_oberflaeche.py* dargestellt, welche mit anderen Unterprogrammen interagieren. Die Klasse *StatusProvider* empfängt

⁷[numpy-Dokumentation der verwendeten Version 2.3](#)

⁸[matplotlib-Dokumentation für die verwendete Version 3.10.3](#)

⁹[pyserial-Dokumentation \(verwendete Version: 3.5\)](#)

¹⁰[PySide6-Dokumentation \(verwendete Version: 6.9.2\)](#)

die Statusmeldungen vom Hardwareinterface und verteilt diese an das richtige Fenster der Benutzeroberfläche. Die Klasse *MeasureWorker* kümmert sich um den Aufruf der Messfunktionen, die vom Benutzer ausgewählt worden sind.

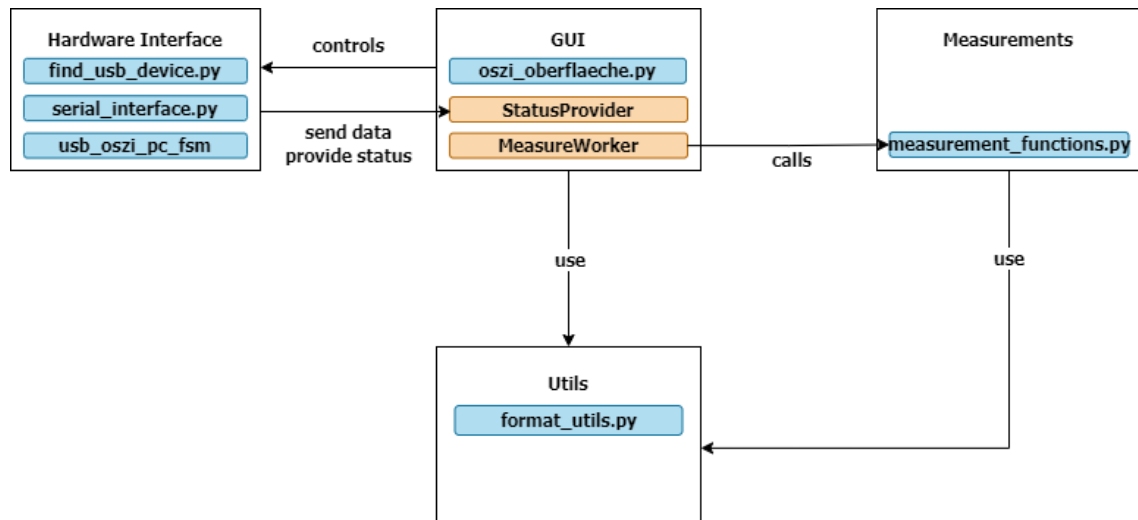


Abbildung 38: Paketdiagramm zur Darstellung der Unterprogramme

9.1.3. Konzept Hardwareinterface

Um das Hardwareinterface zu realisieren wurde die Zustandsmaschine, die auf den Microcontroller läuft auf den PC abgebildet. Das Zustandsdiagramm des PCs befindet sich im Anhang A. Mit dem Unterprogramm *find_my_device.py* wird der passende COM-Port automatisch gefunden. Mit *serial_interface.py* werden die Methoden der Python Klasse *serial_interface* überschrieben. Die Kommunikation zwischen PC und Microcontroller wird über diese Funktionen realisiert. Die Zustandsmaschine soll in einem separaten Thread laufen, um die Benutzeroberfläche responsiv zu halten.

9.1.4. Konzept Benutzeroberfläche

Die Benutzeroberfläche ist objektorientiert konzipiert und basiert auf einer modularen Aufteilung in mehrere Klassen, die jeweils spezifische Teilfunktionen realisieren. Abbildung 39 zeigt die wichtigsten Klassen sowie deren Beziehungen untereinander.

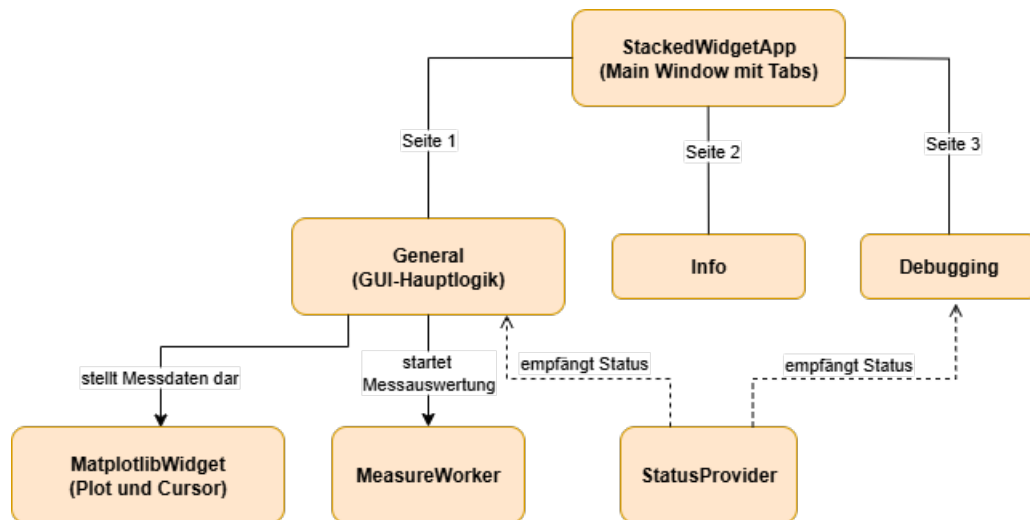


Abbildung 39: Übersicht der Klassen der Benutzeroberfläche

Das Hauptfenster der Anwendung wird durch die Klasse *StackedWidgetApp* repräsentiert. Sie instanziiert die drei Hauptseiten *General*, *Info* und *Debugging* und verbindet diese über eine Seiten-Navigation. Zusätzlich wird eine Instanz der Klasse *StatusProvider* erzeugt, welche den gezielten Austausch von Statusmeldungen zwischen PC bzw. μC und GUI ermöglicht und somit sowohl der *General*- als auch der *Debugging*-Seite zur Verfügung steht.

Die Funktionalität der einzelnen Seiten ist wie folgt aufgeteilt:

- **Info:** Stellt dem Anwender eine kompakte Bedienungsanleitung zur Verfügung.
- **Debugging:** Ermöglicht das manuelle Konfigurieren von Abtastfrequenz und Referenzspannung zu Test- und Analysezwecken.
- **General:** Beinhaltet die Hauptbenutzeroberfläche. Hier werden sämtliche grafische Elemente erzeugt und die zugehörige Logik implementiert. Innerhalb dieser Klasse werden die folgenden Komponenten initialisiert:

- *UsbOszGui* Synchronisiert die Benutzeroberfläche mit der Zustandsmaschine (FSM) auf PC- und Mikrocontroller-Seite.
- *MatplotlibWidget* Kümmt sich um die Visualisierung der Messdaten sowie um die Implementierung von Achseneinstellungen und Cursorfunktionen.
- *MeasureWorker* Verwaltet die Ausführung der Messfunktionen und stellt deren Ergebnisse zur Anzeige bereit.
- *CustomStepSpinBox* Realisiert benutzerdefinierte Drehregler zur komfortablen Parametereinstellung (nicht im Klassendiagramm dargestellt).

9.1.5. Auswahl und Definition der Messungen

Bei der Auswahl der Messungen wurde sich am Benutzerhandbuch des Agilent InfiniiVision 2000 X-Series Oszilloskop orientiert siehe [Keysight2025]. Folgende Messungen sollen mit dem USB-Oszilloskop möglich sein:

- **Maximum:** Höchste Wert des Signals.
- **Minimum:** Niedrigste Wert des Signals.
- **Dach:** Häufigste Wert im oberen Wellenformteil. Falls dieser nicht eindeutig bestimmt werden kann ist der Dach Wert gleich dem Maximum.
- **Base:** Häufigste Wert im unteren Wellenformteil. Falls dieser nicht eindeutig bestimmt werden kann ist der Base wert gleich dem Minimum.
- **Amplitude:** Differenz zwischen Dach und Base.
- **Spitze Spitze:** Differenz zwischen Maximum und Minimum.
- **Mittelwert:** Mittelwert über vollständige Perioden. Dieser ergibt sich folgendermaßen:

$$\text{Mittelwert} = \frac{\sum x_i}{n}$$

mit x_i = Wert am Punkt i und n = Anzahl der Messwerte.

- **DC RMS:** Quadratischer Mittelwert über vollständige Perioden. Dieser ergibt sich folgendermaßen:

$$\text{RMS}_{\text{DC}} = \sqrt{\frac{\sum_{i=1}^n x_i^2}{n}}$$

mit x_i = Wert am Punkt i und n = Anzahl der Messwerte.

- **AC RMS:** Quadratischer Mittelwert, mit entfernter DC-Komponente über vollständige Perioden. Dieser ergibt sich folgendermaßen:

$$\text{RMS}_{\text{DC}} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

mit x_i = Wert am Punkt i und n = Anzahl der Messwerte und $\bar{x}$ = Mittelwert über vollständige Periode.

- **Periode:** Zeit zwischen den mittleren Schwellenpunkten zweier aufeinanderfolgender Flanken.
- **Frequenz:** Ist als Kehrwert der Periode definiert.

10. Implementierung

Die nachfolgende Beschreibung gibt einen Überblick über die Umsetzung der Software. Diese schließt sowohl das Hardwareinterface, zur Kommunikation mit dem μC ein, sowie die Benutzeroberfläche. Für genaue Informationen zur Implementierung sei auf die Quellcode-Dateien verwiesen.

10.1. Hardwareinterface

Die Implementierung der FSM auf dem PC erfolgte nach dem in Anhang A dargestellten Zustandsdiagramm. Hierfür wurde das Python Paket *python-statemachine* verwendet. Dieses ermöglicht es Zustände und Zustandstransitionen einfach zu erstellen. Der Zustandsautomat wurde folgendermaßen implementiert:

```
1 class usbOszPC(StateMachine):
2     init = State("Init", initial=True)
3     idle = State("Idle")
4     acquiring_start = State("Acquiring_Start")
5     acquiring_stop = State("Acquiring_Stop")
6     transmission = State("Transmission")
7     wait_for_ack = State("Wait_For_Ack")
8
9     app_started = init.to(idle)
10    run = idle.to(acquiring_start)
11    acquisition_done = acquiring_start.to(acquiring_stop)
12    ready_for_data = acquiring_stop.to(transmission)
13    data_transfer_complete = transmission.to(wait_for_ack)
14    processing_complete = wait_for_ack.to(acquiring_start)
15    stop_acquisition = wait_for_ack.to(idle)
16    reset = (
17        init.to(init) | idle.to(init) | acquiring_start.to(init) |
18        acquiring_stop.to(init) |
19        transmission.to(init) | wait_for_ack.to(init)
20    )
```

Listing 6: Implementierung der Zustandsmaschine `usbOszPC` in Python

Der Befehl zum Zustandswechsel wird dem μC immer am Ende des vorherigen Zustands gesendet. Beim Eintritt in den nächsten Zustand wird dann überprüft, ob der μC bereits

in den nächsten Zustand gewechselt hat. Das heißt der μC wechselt immer zuerst in den nächsten Zustand und die PC FSM immer erst, wenn der μC bereits gewechselt hat. Eine Ausnahme besteht im Zustand *transmission*, hier wird nicht auf die Information vom μC gewartet, da der Speicher sonst mit Daten überflutet wird und Daten verloren gehen würden.

Nachdem alle Daten übertragen worden sind, sendet der μC ein weiteres Byte, dass einen Überlauf signalisiert. Dieses wird direkt ausgewertet und je nach Ergebnis eine Meldung in die *status_queue* geschoben, *OVERFLOW* oder *NO_OVERFLOW*.

Anschließend werden die Rohdaten mit der Funktion *calculate_voltage()* in Spannungswerte umgerechnet. Dafür wird folgende Formel verwendet:

$$\frac{\text{raw_value}}{4095} \cdot v_{\text{ref}} \cdot 10^{-3} \cdot v_{\text{div_factor}}$$

Die 4095 ergeben sich aus der Auflösung des ADCs von 12 Bit. v_{ref} ist die eingestellte Referenzspannung. Der *div_factor* ergibt sich aus dem Spannungsteiler im AF.

10.2. Kommunikation zwischen Hardwareinterface und Benutzeroberfläche

Die Kommunikation zwischen den beiden Threads wurde mit zwei Queues realisiert, die *command_queue* und die *status_queue*. Um zu verhindern, dass Daten überschrieben werden oder verloren gehen, kann immer nur ein Thread schreibend zugreifen und der andere nur lesend. So schreibt der GUI-Thread Befehle in die *command_queue*, der FSM-Thread liest diese aus und führt basiert auf seinem aktuellen Zustand und den Befehlen die passende Codezeile aus. Dabei schreibt er bei Zustandswechsel oder im Fehlerfall eine Nachricht in die *status_queue*, welche wiederum vom Thread der *StatusProvider* Klasse, die sich im Programm der Benutzeroberfläche befindet, ausgewertet wird.

Die *StatusProvider* Klasse liest die *status_queue* aus und sendet eine Nachricht an die jeweilige Funktion *handle_status*, die sich in den Klassen *General* und *Debugging* befindet. Hier wird entschieden, ob die Nachricht für die Seite der GUI bestimmt war. Hier wird dann die eigentliche Reaktion auf die Statusmeldung ausgeführt.

Die *StatusProvider* Klasse wird bei Programmstart als Thread instanziiert und fragt die *status_queue* alle 100 ms ab.

10.3. Benutzeroberfläche

- Umrechnung der Werte vom uC in Spannungswerte - Plot -> einmal aufbauen nur aktualisiert - Wie wurden Cursor realisiert - Queues - FSM GUI - FSM PC - Erstellen der Threads - Implementierung der Messungen (Periode, Dach und Base)

10.4. SW-Test

- Zunächst wurde die Funktionalität der Zustandsmaschine auf dem PC unabhängig vom μC getestet, indem die Transitionen, die vom μC kommen, durch Tasten simuliert wurden.
- Weitere Tests wurden mit einem Programm auf den uC gemacht, das feste Werte an den PC überträgt. - Abschließend wurde die Funktionalität aller GUI-Elemente mit dem Gesamtsystem getestet, siehe ??

11. Zusammenführung

Nachdem die Tests der Einzelkomponenten (HW, FW, SW) erfolgt waren, konnte deren Zusammenführung durchgeführt werden.

Hierbei wurden zunächst die Software mit der Firmware verbunden und die einwandfreie Kommunikation zwischen μC und der Rechner-Anwendung überprüft. Nachdem dies erfolgreich getestet wurde, konnte die Oszi-Baugruppe auf das NUCLEO-Board aufgesteckt werden. Der Gesamtaufbau kann Abbildung 40 entnommen werden.

11.1. Testaufbau

Beim Anschluss sind die Tabelle 2 und die darunterliegenden Hinweise zu beachten.

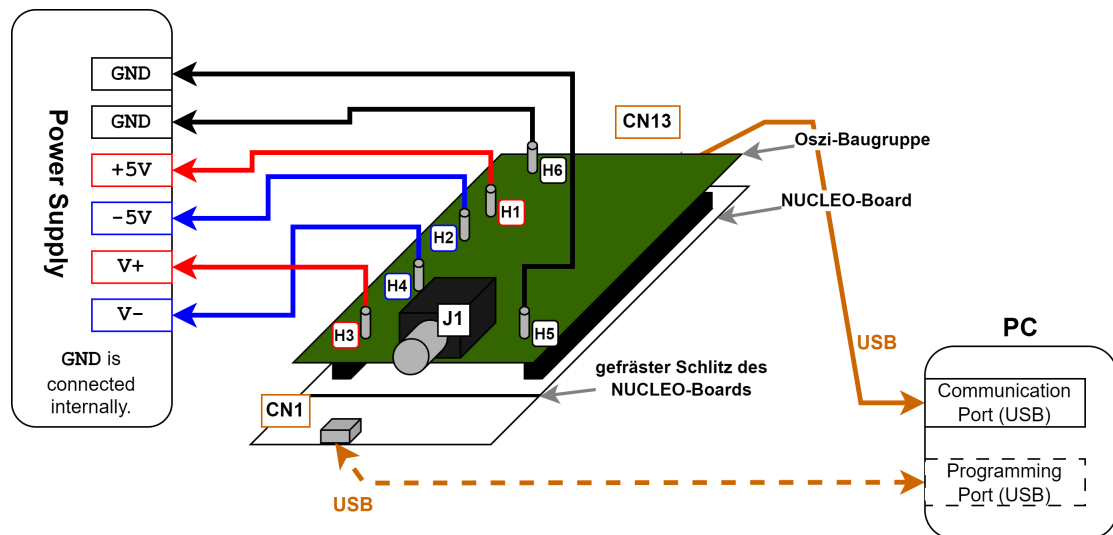


Abbildung 40: Übersichtsschaltbild für den Testaufbau des Projekts

Oszi-Anschluss	externer Anschluss	Beschreibung
<i>H1</i> (Baugruppe)	$+5\text{ V} + U_D$	
<i>H2</i> (Baugruppe)	$-5\text{ V} - U_D$	
<i>H3</i> (Baugruppe)	$V+ = +12\text{ V} + U_D$	
<i>H4</i> (Baugruppe)	$V- = -12\text{ V} - U_D$	
<i>H5, H6</i> (Baugruppe)	0V (GND)	
<i>J1</i> (Baugruppe, BNC)	DUT (Device Under Test)	
<i>CN13</i> (NUCLEO, micro-USB)	PC (USB)	Kommunikationsanschluss
optional:		
<i>CN1</i> (NUCLEO, micro-USB)	PC (USB)	Programmierschluss

Tabelle 2: Verbindungstabelle

Hinweise:

- Aufgrund der eingebauten Verpolungsschutz-Dioden ist jeweils eine zusätzliche Spannung von $U_D \approx 0.3\text{ V}$ zu berücksichtigen.
- Zunächst müssen die Versorgungsspannungen $\pm 5\text{ V}$ eingeschaltet werden, danach die Versorgungsspannungen $\pm 12\text{ V}$.
- Anschließend kann die Kommunikationsleitung an CN13 angeschlossen und mit dem PC verbunden werden.
- Als letztes wird das Signal an J1 eingespeist.
- Über den USB-Port CN1 kann optional die Programmierung des Mikrocontrollers und/oder der Aufbau einer Debug-Verbindung erfolgen.

11.2. Funktionstest

Um die Funktion des Oszilloskops zu überprüfen wurde eine Reihe an Testsignalen mittels Funktionsgenerator eingespeist. Dies sollten dazu dienen die Performanz der Baugruppe zu testen. Die resultierenden Signale sind den folgenden Abbildungen zu entnehmen.

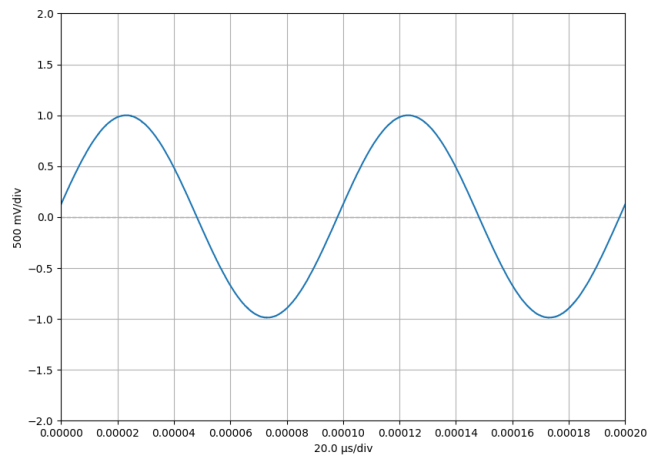


Abbildung 41: Testsignal: Sinus ($f = 10 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)

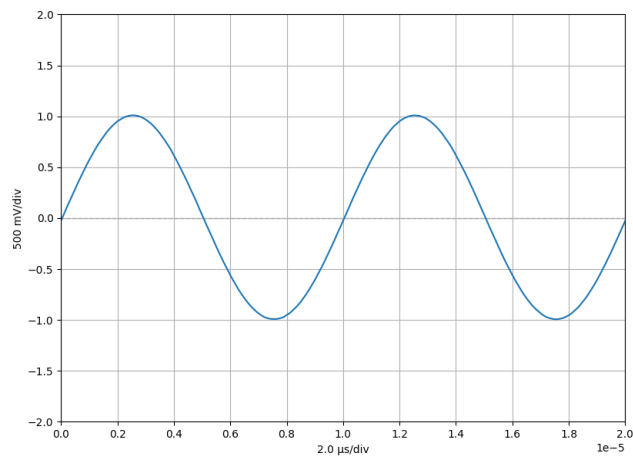


Abbildung 42: Testsignal: Sinus ($f = 100 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)

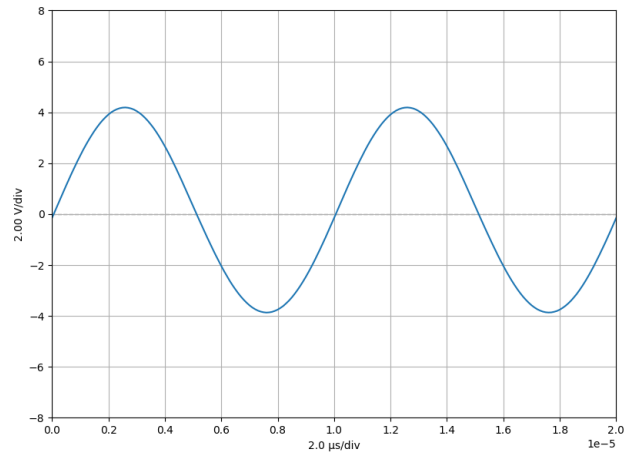


Abbildung 43: Testsignal: Sinus ($f = 100 \text{ kHz}$, $V_{pp} = 8 \text{ V}$)

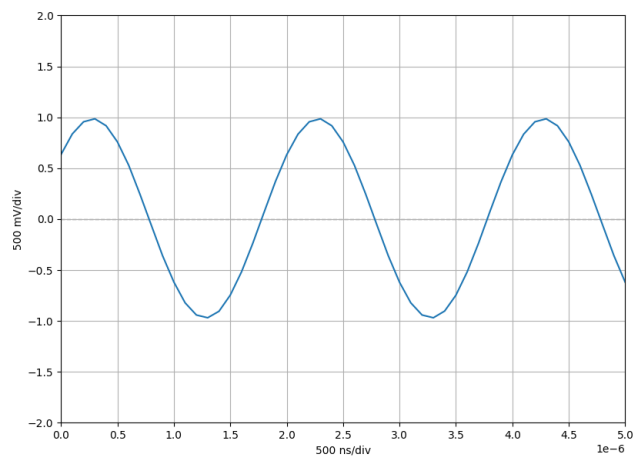


Abbildung 44: Testsignal: Sinus ($f = 500 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)

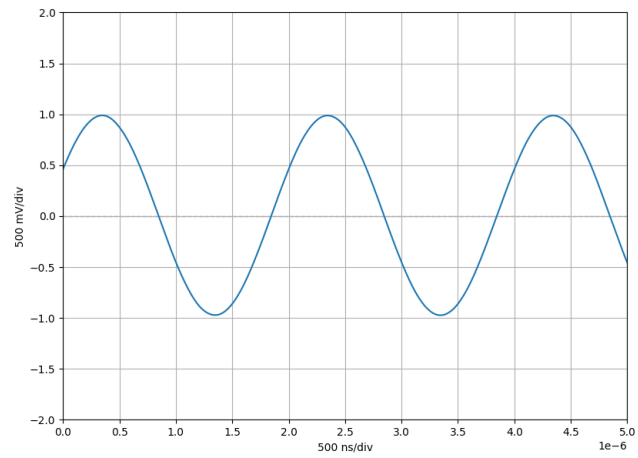


Abbildung 45: Testsignal: Sinus ($f = 500 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)
Polynom-Interpolation

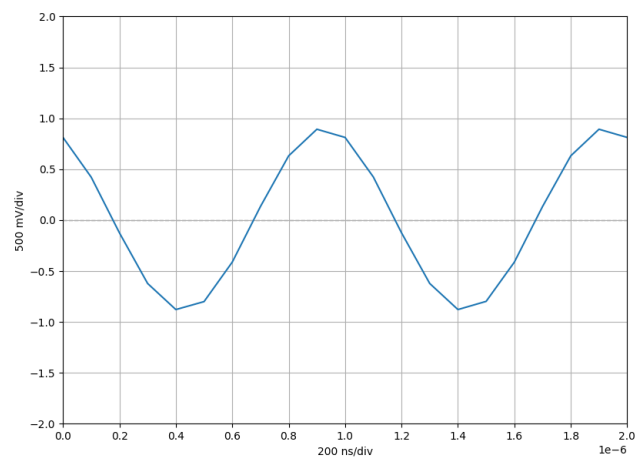


Abbildung 46: Testsignal: Sinus ($f = 1 \text{ MHz}$, $V_{pp} = 2 \text{ V}$)

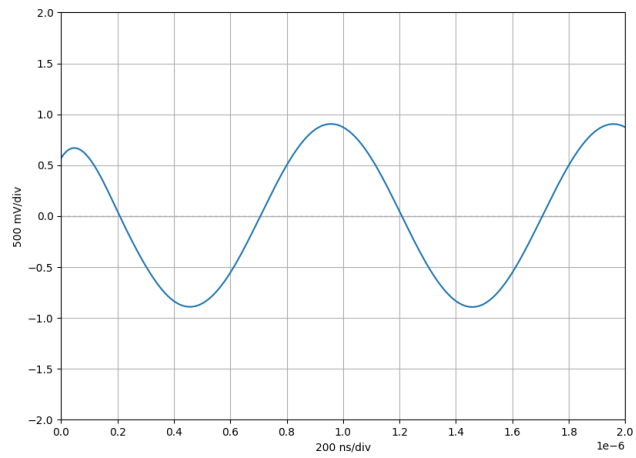


Abbildung 47: Testsignal: Sinus ($f = 1 \text{ MHz}$, $V_{pp} = 2 \text{ V}$)
Polynom-Interpolation

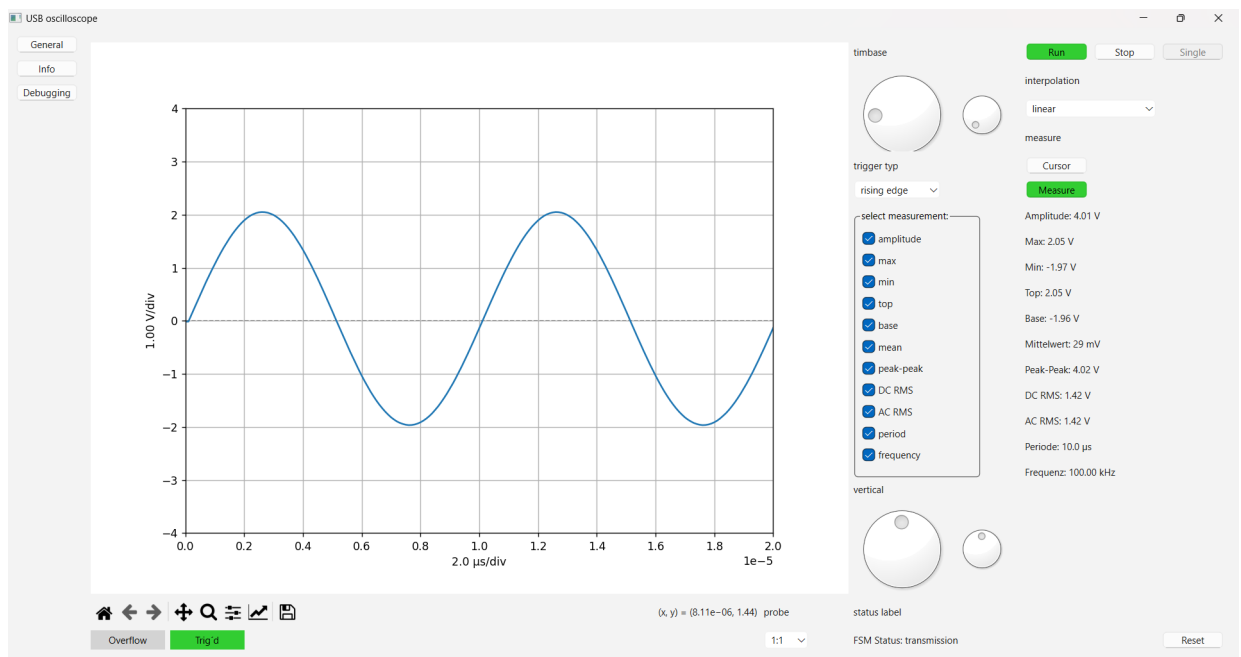


Abbildung 48: Testsignal: Sinus ($f = 1 \text{ MHz}$, $V_{pp} = 4 \text{ V}$)
Überprüfung mittels Measure-Funktionen

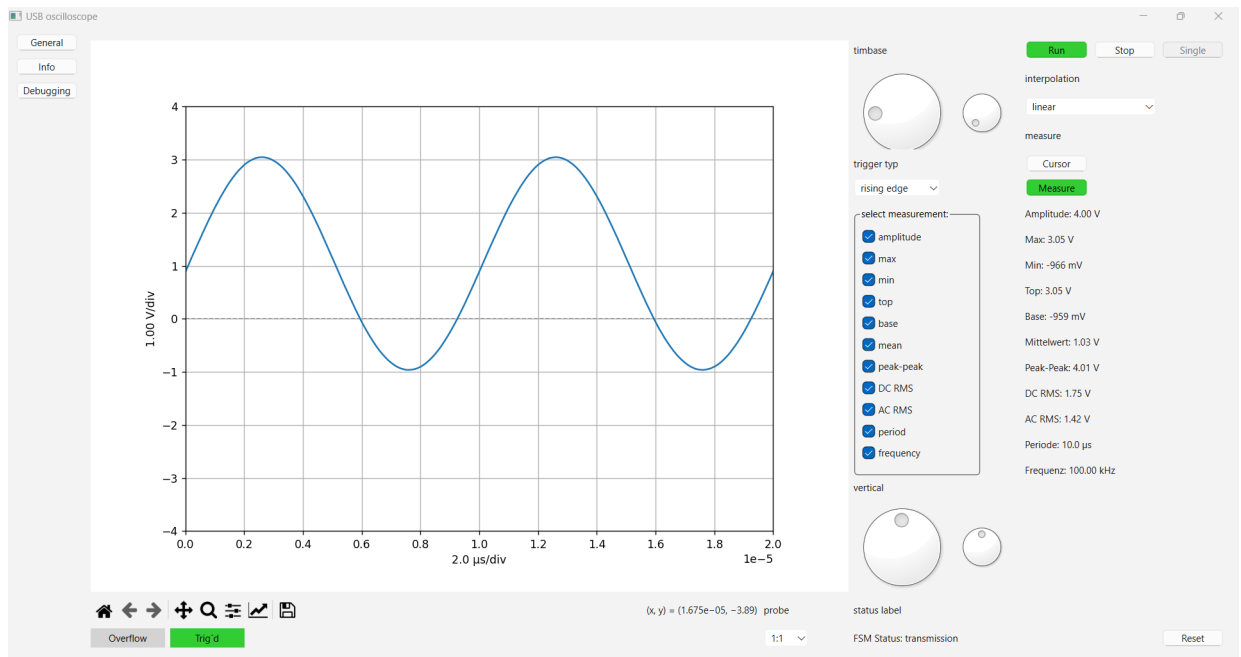


Abbildung 49: Testsignal: Sinus ($f = 100 \text{ kHz}$, $V_{pp} = 4 \text{ V}$, $V_{offset} = 1 \text{ V}$)
Überprüfung mittels Measure-Funktionen

Rechnerische Überprüfung des Measure-Ergebnisses von $AC-RMS$ und $DC-RMS$
(V_1 : 1. Oberwelle, V_0 : Gleichanteil):

$$V_{AC-RMS} = \frac{\hat{V}_1}{\sqrt{2}} = \frac{2V}{\sqrt{2}} = 1.41V \quad (16)$$

$$V_{DC-RMS} = \sqrt{\left(\frac{\hat{V}_1}{\sqrt{2}}\right)^2 + V_0^2} = \sqrt{\left(\frac{2V}{\sqrt{2}}\right)^2 + (1V)^2} = 1.73V \quad (17)$$

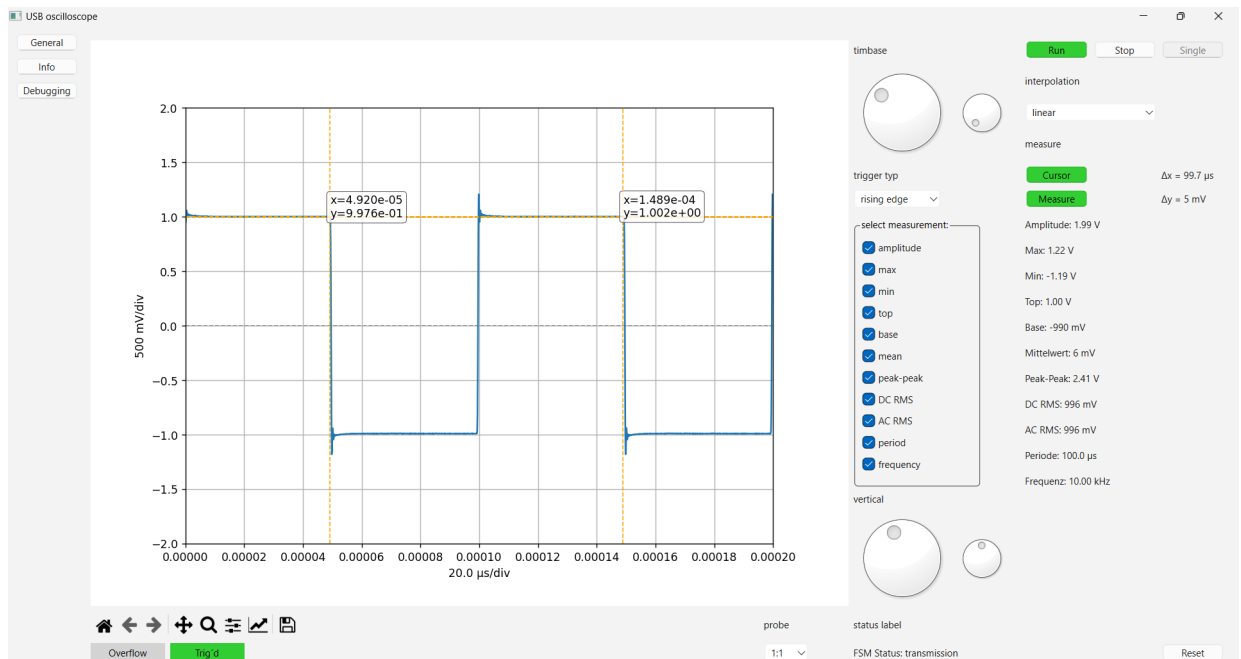


Abbildung 50: Testsignal: Rechteck ($f = 10 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)
Überprüfung mittels Measure-Funktionen und Cursorsn

Man beachte die Top- und Base-Messung, die ist top.

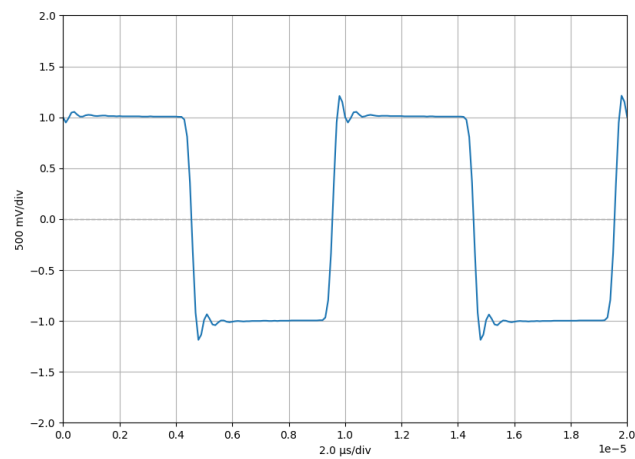


Abbildung 51: Testsignal: Rechteck ($f = 100 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)

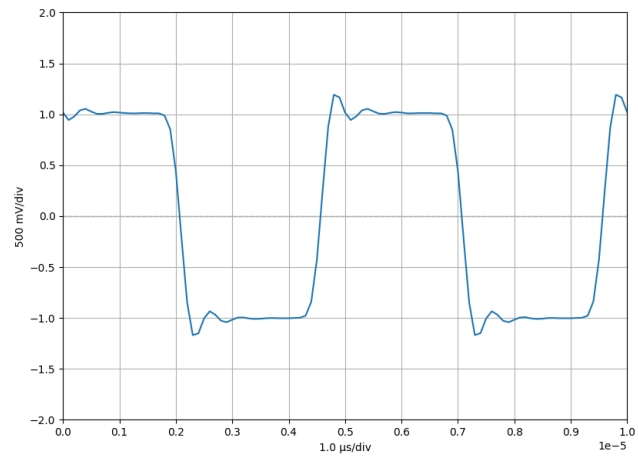


Abbildung 52: Testsignal: Rechteck ($f = 200 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)

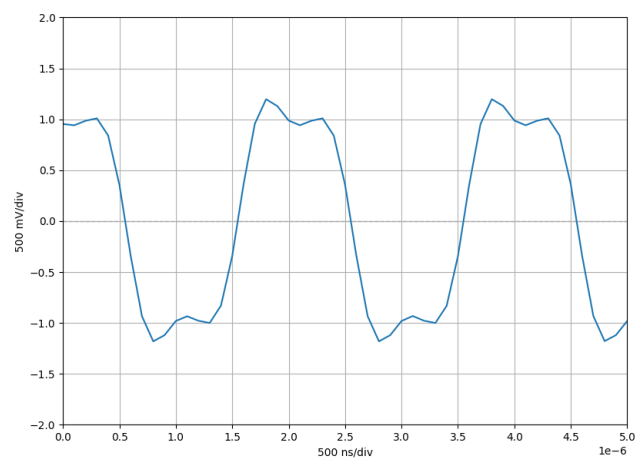


Abbildung 53: Testsignal: Rechteck ($f = 500 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)

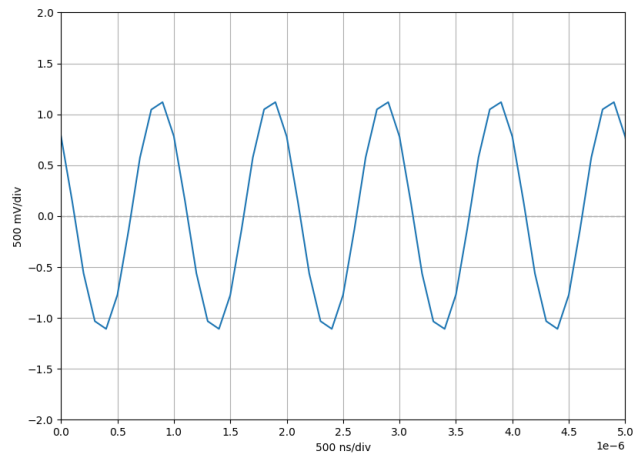


Abbildung 54: Testsignal: Rechteck ($f = 1 \text{ MHz}$, $V_{pp} = 2 \text{ V}$)

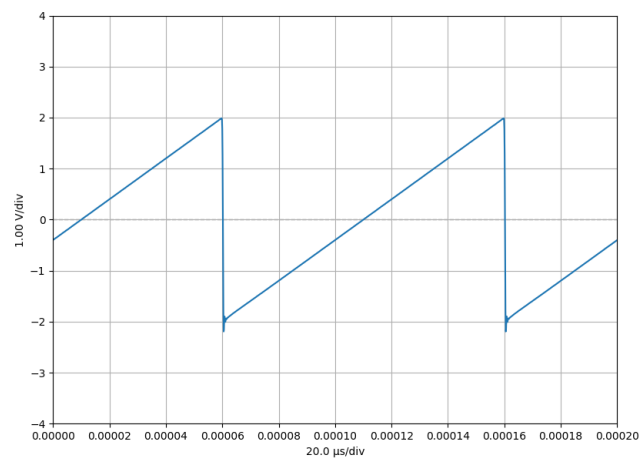


Abbildung 55: Testsignal: Sägezahn ($f = 10 \text{ kHz}$, $V_{pp} = 4 \text{ V}$)

12. Ergebnisse

12.1. Vergleich mit ursprünglichen Anforderungen

Zunächst erfolgt ein Vergleich mit den Anforderungen, welche zu Beginn des Projekts definiert wurden (siehe Unterabschnitt 4.2).

Der vorliegende Prototyp implementiert ein 1-kanaliges Messsystem, welches zusammen mit der programmierten PC-Anwendung als Oszilloskop verwendet werden kann. Die Verbindung zwischen PC und Baugruppe erfolgt über USB.

Die angestrebte Bandbreite von 1 MHz wurde erreicht. Abweichungen vom eingespeisten Signal, sowie Maßnahmen zur Kompensation dieser Effekte wurden bereits im Unterabschnitt 5.3 (HW-Test) ausführlich beschrieben (merkliche Abweichungen bei Signalen mit Frequenzen ab 500 kHz). Die Abtastrate von 10 MS/s konnte durch die Auswahl der entsprechenden Bauelemente (ADC, μC , etc.) gewährleistet werden.

Mit der vorliegenden Hardware (AFE und ADC) lässt sich ein Eingangsspannungsbereich ± 5 V mit einer Auflösung von 12 Bit betrachten. Mittels herkömmlicher 10:1 Tastköpfe lässt sich der Eingangsspannungsbereich auf ± 50 V vergrößern. Für die nach den Anforderungen festgelegte Verwendung eines Offsets wären Ergänzungen im Layout der Baugruppe notwendig (beschrieben in Unterabschnitt 5.3).

12.2. Benutzung des USB-Oszilloskops

Die Verwendung des Oszilloskops setzt den Aufbau gemäß Abbildung 40 in Unterabschnitt 10.1 voraus.

Das Endergebnis der entwickelten Benutzeroberfläche ist in Abbildung 56 dargestellt. Die Bedienung und Einstellmöglichkeiten orientieren sich stark an einem klassischen Oszilloskop und bieten dem Anwender zahlreiche sinnvolle Funktionen für die Signalbetrachtung und -auswertung.

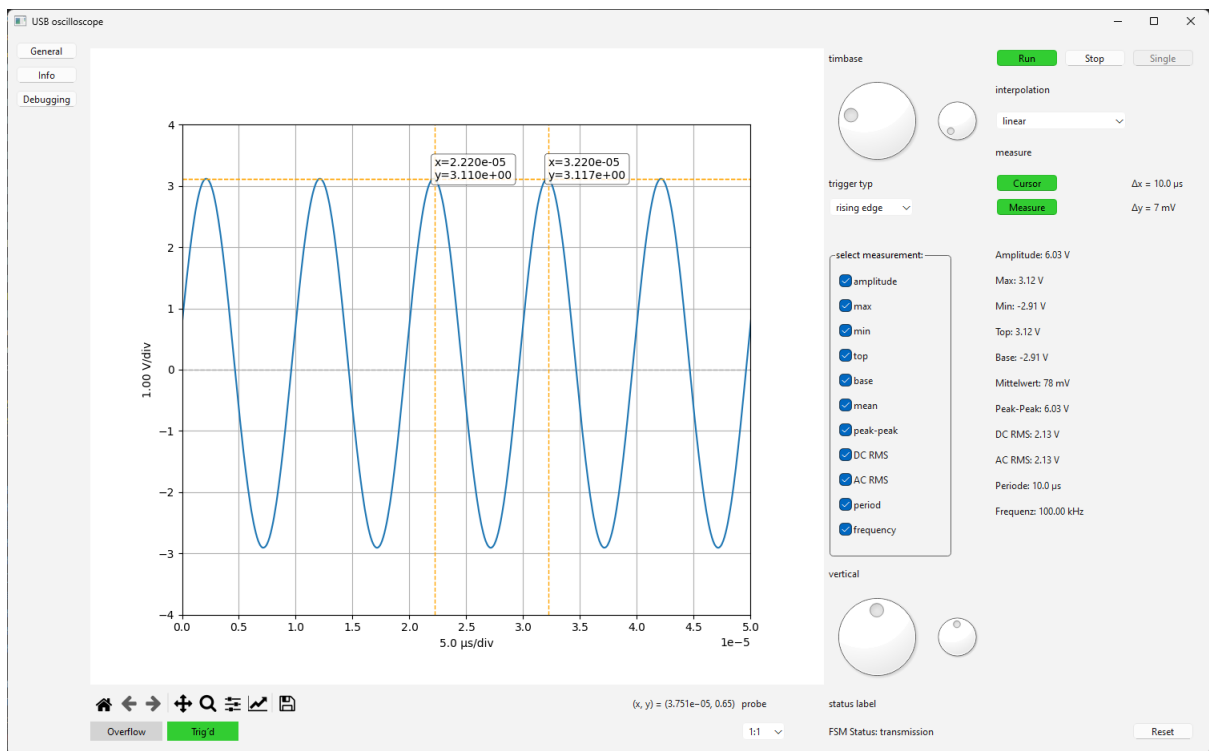


Abbildung 56: Benutzeroberfläche

12.2.1. Hauptfunktionen

- **Live-Plot:** Im linken Bereich der GUI werden die aktuell aufgenommenen Messdaten als Signalverlauf angezeigt. Über die Navigationstoolbar unterhalb des Plots können spezifische Ausschnitte des Signals betrachtet, die Ansicht vergrößert und der Plot als Bild exportiert werden.

- **Statusanzeige:** Unterhalb der Toolbar signalisiert die farbige „Overflow“-Anzeige (rot bei Überlauf) einen Buffer-Überlauf. Das danebenliegende grüne „Trig’d“-Label zeigt an, ob ein Triggerereignis vorliegt (erlischt, wenn keine neuen Daten vorliegen).
- **Tastkopf-Einstellung:** Rechts unterhalb des Plots kann der Benutzer wählen, ob ein 10:1 Tastkopf angeschlossen ist.

12.2.2. Bedienelemente und Einstellmöglichkeiten

- **Modus-Auswahl:** Oben rechts kann zwischen „Run-Modus“ (kontinuierliche Messung) und „Single-Shot-Modus“ (einmalige Messung) gewählt werden. Die Messung kann im Run-Modus jederzeit pausiert („Stop“-Button) oder wieder gestartet werden.
- **Zeitbasis und Achsverschiebung:** Über zwei Drehregler wird die horizontale Zeitbasis (großer Regler) sowie die Verschiebung der X-Achse (kleiner Regler) eingestellt.
- **Trigger-Einstellungen:** Mithilfe eines Auswahlfensters kann der Triggertyp (steigende oder fallende Flanke) gewählt werden.
- **Interpolation:** Zusätzlich steht eine Auswahl zur Signalinterpolation bereit (z. B. linear, spline, steps, points).
- **Messfunktion und Cursor:** Die Messfunktionen lassen sich mithilfe der „Measure“-Schaltfläche aktivieren. Die gewünschten Kennwerte (Amplitude, Mittelwert, Frequenz usw.) können über eine Checkbox ausgewählt werden; die berechneten Werte werden unmittelbar angezeigt. Mit dem Cursor-Button werden zwei Kreuzcursor aktiviert, deren Differenzen (Δx , Δy) links daneben angezeigt werden.
- **Vertikale Achseneinstellung:** Zwei weitere Drehregler erlauben die Einstellung von Skalierung (Volt pro Kästchen) und Verschiebung der Y-Achse (Nullpunkt).
- **Status-Label:** Das Status-Label informiert über den aktuellen Zustand der Zustandsmaschine (PC und Mikrocontroller) und gibt Hinweise zu Änderungen der Trigger- oder Interpolationsart, Verbindungsstatus oder Fehlern.
- **Reset-Schaltfläche:** Damit kann die Zustandsmaschine zurückgesetzt werden, falls sie in einem ungültigen Zustand verharrt.

Durch das Wechseln auf die Seite *Debugging* bieten sich dem Benutzer noch weitere Einstellmöglichkeiten, siehe Abbildung 57.

- **Einstellen der Abtastfrequenz:** Eine Spinbox erlaubt die Einstellung der Abtastfrequenz zwischen 20 kHz und 10 MHz. Unter dieser wird die vom μC tatsächlich eingestellte Abtastfrequenz angezeigt.
- **Einstellen der Referenzspannung:** Eine weitere Spinbox erlaubt die Einstellung der Referenzspannung in 1 mV Schritten.
- **Timing Informationen:** Zusätzlich wird die Übertragungszeit und die Zeit, die zur Umwandlung der Rohdaten in Spannungswerte benötigt wird, angezeigt.

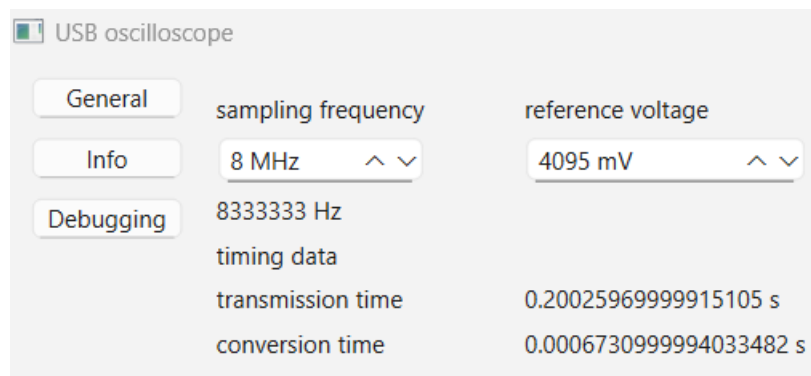


Abbildung 57: Debug-Fenster der Benutzeroberfläche

12.2.3. Besonderheiten und Bedienhinweise

- Änderungen der Trigger- oder Interpolationsart und Einstellungen im Debug-Fenster sind **ausschließlich im Idle-Zustand** des Mikrocontrollers möglich. Die Messung muss zuvor gestoppt werden.
- Die Einstellungen der horizontalen und vertikalen Achsen werden beim Schließen der GUI gespeichert und beim nächsten Öffnen wiederhergestellt.

Zusammenfassend bietet die GUI des USB-Oszilloskops eine nutzerfreundliche und praxisorientierte Oberfläche mit allen aus einem klassischen Tischoszilloskop bekannten Einstellungsmöglichkeiten.

13. Fazit und Ausblick

Da das Ziel des Projekts die Validierung unseres Konzepts zur Signalerfassung, -konditionierung und -verarbeitung für ein USB-Oszilloskop war, wurden weitergehende Aspekte nicht behandelt. Hierbei handelt es sich um Aspekte, die für die Validierung des Konzeptes nicht selbst entwickelt werden mussten. Hierunter fällt vorwiegend die Verwendung externer Netzteile zur Versorgung der Baugruppe.

14. Literaturverzeichnis

Literatur

- [Ana01a] AnalogDevices. „Pipeline ADCs Come of Age“. In: *Analog Devices - Technical Articles* (20. Nov. 2001). URL: <https://www.analog.com/en/resources/technical-articles/pipeline-adcs-come-of-age.html> (besucht am 16.09.2025).
- [Ana01b] AnalogDevices. „Understanding Flash ADCs“. In: *Analog Devices - Technical Articles* (2. Okt. 2001). URL: <https://www.analog.com/en/resources/technical-articles/understanding-flash-adcs.html> (besucht am 16.09.2025).
- [Ana01c] AnalogDevices. „Understanding Pipelined ADCs“. In: *Analog Devices - Technical Articles* (2. Okt. 2001). URL: <https://www.analog.com/en/resources/technical-articles/understanding-pipelined-adcs.html> (besucht am 16.09.2025).
- [Bäs19] Prof. Dr.-Ing. J. Bäsig. *Automaten und ihre Anwendung. Skriptum zur Vorlesung Informatik 2*. 12. Aufl. TH Nürnberg, 2019.
- [Ber23] Herbert Bernstein. *NF- und HF-Messtechnik: Messen mit Oszilloskopen, Netzwerkanalysatoren und Spektrumanalysator*. Springer Fachmedien Wiesbaden, 2023. ISBN: 9783658391164. DOI: [10.1007/978-3-658-39116-4](https://doi.org/10.1007/978-3-658-39116-4).
- [Ber24] Herbert Bernstein. *Messelektronik und Sensoren: Grundlagen der Messtechnik, Sensoren, analoge und digitale Signalverarbeitung*. Springer Fachmedien Wiesbaden, 2024. ISBN: 9783658389291. DOI: [10.1007/978-3-658-38929-1](https://doi.org/10.1007/978-3-658-38929-1).
- [IEE23] IEEE. „IEEE Standard for Terminology and Test Methods for Analog-to-Digital Converters“. In: *IEEE Std 1241-2023 (Revision of IEEE Std 1241-2010)* (2023). DOI: [10.1109/IEEESTD.2023.10269815](https://doi.org/10.1109/IEEESTD.2023.10269815).
- [Lan24] Niklas Lang. „Was ist Threading und Multiprocessing in Python?“ In: *Data Base Camp* (6. Apr. 2024). URL: <https://databasecamp.de/python/threading-und-multiprocessing> (besucht am 08.05.2025).
- [Mic25] Microsoft. *USB device class drivers included in Windows*. 13. Juni 2025. URL: <https://learn.microsoft.com/en-us/windows-hardware/drivers/usbcon/supported-usb-classes#usb-device-classes> (besucht am 29.09.2025).

- [MPS] MPS. *Types Of ADCs*. Monolithic Power Systems. URL: <https://www.monolithicpower.com/en/learning/mpscholar/analog-to-digital-converters/introduction-to-adcs/types-of-adcs> (besucht am 16. 09. 2025).
- [Müh20] Thomas Mühl. *Elektrische Messtechnik: Grundlagen, Messverfahren, Anwendungen*. Springer Fachmedien Wiesbaden, 2020. ISBN: 9783658291167. DOI: [10.1007/978-3-658-29116-7](https://doi.org/10.1007/978-3-658-29116-7).
- [Pad17] Kushwanthi Padmanabhuni. „Confused About Choosing the Right Amplifier for Your Filter? Here’s the Answer to Put an End to Your Worries—the Analog Filter Wizard Design Tool!“ In: *Analog Dialogue* 51 (Nov. 2017). URL: <https://www.analog.com/en/resources/analog-dialogue/studentzone/studentzone-october-2017.html> (besucht am 15. 09. 2025).
- [Pin20] Art Pini. „The Basics of Anti-Aliasing Low-Pass Filters (and Why They Need to be Matched to the ADC)“. In: *DigiKey - Articles & Blogs* (24. März 2020). URL: www.digikey.de/en/articles/the-basics-of-anti-aliasing-low-pass-filters (besucht am 15. 09. 2025).
- [Smi99] Steven W. Smith. *The Scientist & Engineer’s Guide to Digital Signal Processing*. Analog Devices, 1999. URL: https://www.analog.com/en/resources/technical-books/scientist_engineers_guide.html (besucht am 1999).
- [SRZ22] Elmar Schrüfer, Leonhard M. Reindl und Bernhard Zagar. *Elektrische Messtechnik: Messung elektrischer und nichtelektrischer Größen*. Carl Hanser Verlag GmbH & Co. KG, Aug. 2022. ISBN: 9783446474437. DOI: [10.3139/9783446474437](https://doi.org/10.3139/9783446474437).
- [STm16] STmicroelectronics. *AN4031 - Using the STM32F2, STM32F4 and STM32F7 Series DMA controller*. Juni 2016. URL: https://www.st.com/resource/en/application_note/dm00046011-using-the-stm32f2-stm32f4-and-stm32f7-series-dma-controller-stmicroelectronics.pdf (besucht am 11. 09. 2025).
- [STm19] STmicroelectronics. *UM1734 - STM32Cube™ USB device library*. Feb. 2019. URL: https://www.st.com/resource/en/user_manual/um1734-stm32cube-usb-device-library-stmicroelectronics.pdf (besucht am 29. 09. 2025).

- [STm23] STmicroelectronics. *UM1905 - Description of STM32F7 HAL and low-layer drivers*. Feb. 2023. URL: https://www.st.com/resource/en/user_manual/um1905-description-of-stm32f7-hal-and-lowlayer-drivers-stmicroelectronics.pdf (besucht am 29.09.2025).
- [STm24] STmicroelectronics. *Reference Manual for STM32F76xxx and STM32F77xxx advanced Arm®-based 32-bit MCUs*. Juli 2024. URL: https://www.st.com/resource/en/reference_manual/rm0410-stm32f76xxx-and-stm32f77xxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf (besucht am 11.09.2025).
- [STm25a] STmicroelectronics. *Datasheet for STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx*. Juli 2025. URL: <https://www.st.com/resource/en/datasheet/stm32f767zi.pdf> (besucht am 11.09.2025).
- [STm25b] STmicroelectronics. *UM2609 - STM32CubeIDE user guide*. Juni 2025. URL: https://www.st.com/resource/en/user_manual/um2609-stm32cubeide-user-guide-stmicroelectronics.pdf (besucht am 29.09.2025).
- [Urb20] Prof. Dr. Peter Urbanek. *Das MCT Skript. Vom Mikrochip zum Rechnersystem. Ein Grundlagenbuch mit vielen Beispielen*. Version 3.5. TH Nürnberg, 2020.
- [USB25a] USB-IF. *Class definitions for Communication Devices 1.2*. 28. Mai 2025. URL: <https://www.usb.org/document-library/class-definitions-communication-devices-12> (besucht am 29.09.2025).
- [USB25b] USB-IF. *USB 2.0 Specification*. 3. Juni 2025. URL: <https://www.usb.org/document-library/usb-20-specification> (besucht am 29.09.2025).

15. Abbildungsverzeichnis

Abbildungsverzeichnis

1.	Blockschaltbild eines Digitaloszilloskops (Abb. 14.1 aus [Müh20, S. 216])	7
2.	Gegenüberstellung Echtzeitabtastung (a) und Äquivalenzzeitabtastung (b) (Abb. 14.2 aus [Müh20, S. 216])	7
3.	Punktendarstellung (a) und lineare Interpolation (b) (Abb. 14.4 aus [Müh20, S. 218])	8
4.	Lineare Interpolation (a) und si-Interpolation (b) (Abb. 14.5 aus [Müh20, S. 219])	8
5.	Messsignal $s(t)$ mit eingestellter Triggerschwelle und Triggerzeitpunkt (Teilabbildung der Abb. 14.3 aus [Müh20, S. 217])	9
6.	Funktionsblöcke einer Triggereinrichtung (Abb. 13.8 aus [Müh20, S. 212])	9
7.	Tastteiler am Eingang eines Oszilloskops (Bild 2.42 aus [SRZ22, S. 105])	10
8.	Rechteckimpulse an RC-Spannungsteiler (Bild 2.43 aus [SRZ22, S. 106]) (a) unterkompensiert, (b) richtig kompensiert, (c) überkompensiert	10
9.	Aliasing: Erzeugung einer Scheinfrequenz durch unpassende Abtastrate (Bild 2.71 aus [Ber24, S. 154])	11
10.	Abtasttheorem (Bild 4.30 aus [Ber23, S. 314]) (a) Betragsspektrum des bandbegrenzten Signals (b) Vervielfachung des Basisspektrums durch Abtastung mit $f_S > 2 \cdot f_g$ (c) Aliasing infolge zu niedriger Abtastrate ($f_S < 2 \cdot f_g$)	12
11.	Flash-Umsetzer mit 2 Bit, einschl. Kodeumsetzer Ersteller: Saure, Quelle: [Link] (Lizenz: CC BY-SA 3.0)	14
12.	12-bit pipelined ADC Ersteller: MPS, Quelle: [Link] (Ch. 2, Fig. 4)	15
13.	(a) Zustandsdiagramm (nach [Bäs19]); (b) UML-Zustandsdiagramm	20
14.	Blockdiagramm STM32-DMA-Controller aus [STm16, S. 7]	22
15.	FIFO-Struktur aus [STm16, S. 11] (Teilabbildung)	23
16.	DMA Quell- und Zieladressinkrementierung aus [STm16, S. 10]	23
17.	peripheral-to-memory-Transfer	24
18.	Systemarchitektur für STM32F76xxx μ C aus [STm24, S. 72]	25
19.	Anwendung eines MAF unterschiedlicher Fensterbreite (a) Eingangssignal, (b) Ausgangssignal $M = 11$, (c) Ausgangssignal $M = 51$ (Figure 15-1 aus [Smi99, S. 279])	28

20.	Vergleich Multi-Threading und Multi-Processing	30
21.	V-Modell	31
22.	Gesamtkonzept des USB-Oszilloskops (nach [Müh20, S. 216])	35
23.	Prinzipschaltbild des Analog-Frontends (AFE)	36
24.	Aufbau zum Test der Hardware	37
25.	Beispielabbildung für ein NUCLEO-144-Board, wie das NUCLEO-F767ZI (Quelle: [Link])	40
26.	Funktionsschema der Abtastung	41
27.	Funktionsprinzip DMA Abtastung - Adresswerte dienen nur der Übersicht	43
28.	Zeitdiagramm und Registerübersicht des Hardware-Timers für die Synchro- nisation von ADC und DMA (Sampling am μC)	44
29.	Zustandsdiagramm der Firmware	46
30.	Clock-Tree-Ansicht in STM32CubeMX	51
31.	Expression-Debug-Ansicht zur Überprüfung der Abtastwerte	57
32.	Kommunikationsablauf im Terminal-Programm <i>hterm</i>	58
33.	Beispielhafte Konsolenausgabe bei Ablauf eines Python-Testskripts	59
34.	Signalverlauf ohne Filter 10kHz Rechtecksignal mit überlagertem Rauschen (1Vpp; Bandbreite: 1MHz)	60
35.	Signalverlauf rekursiver gleitender Mittelwertfilter (M=51) 10kHz Rechteck- signal mit überlagertem Rauschen (1Vpp; Bandbreite: 1MHz)	60
36.	Host- und Device-Ansicht aus der USB-Spezifikation (Figure 5-9 aus [USB25b])	62
37.	UML-Sequenzdiagramm: Kommunikation ohne Konfiguration	64
38.	Paketdiagramm zur Darstellung der Unterprogramme	66
39.	Übersicht der Klassen der Benutzeroberfläche	67
40.	Übersichtsschaltbild für den Testaufbau des Projekts	72
41.	Testsignal: Sinus ($f = 10 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)	74
42.	Testsignal: Sinus ($f = 100 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)	74
43.	Testsignal: Sinus ($f = 100 \text{ kHz}$, $V_{pp} = 8 \text{ V}$)	75
44.	Testsignal: Sinus ($f = 500 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)	75
45.	Testsignal: Sinus ($f = 500 \text{ kHz}$, $V_{pp} = 2 \text{ V}$) Polynom-Interpolation	76
46.	Testsignal: Sinus ($f = 1 \text{ MHz}$, $V_{pp} = 2 \text{ V}$)	76
47.	Testsignal: Sinus ($f = 1 \text{ MHz}$, $V_{pp} = 2 \text{ V}$) Polynom-Interpolation	77
48.	Testsignal: Sinus ($f = 1 \text{ MHz}$, $V_{pp} = 4 \text{ V}$) Überprüfung mittels Measure-Funktionen	77

49.	Testsignal: Sinus ($f = 100 \text{ kHz}$, $V_{pp} = 4 \text{ V}$, $V_{offset} = 1 \text{ V}$) Überprüfung mittels Measure-Funktionen	78
50.	Testsignal: Rechteck ($f = 10 \text{ kHz}$, $V_{pp} = 2 \text{ V}$) Überprüfung mittels Measure-Funktionen und Cursors	79
51.	Testsignal: Rechteck ($f = 100 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)	79
52.	Testsignal: Rechteck ($f = 200 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)	80
53.	Testsignal: Rechteck ($f = 500 \text{ kHz}$, $V_{pp} = 2 \text{ V}$)	80
54.	Testsignal: Rechteck ($f = 1 \text{ MHz}$, $V_{pp} = 2 \text{ V}$)	81
55.	Testsignal: Sägezahn ($f = 10 \text{ kHz}$, $V_{pp} = 4 \text{ V}$)	81
56.	Benutzeroberfläche	83
57.	Debug-Fenster der Benutzeroberfläche	85
58.	Übersichts-Blockschaltbild (erstellt für die Konzeptvorstellung)	93

A. Anhang

A.1. Blockschaltbilder

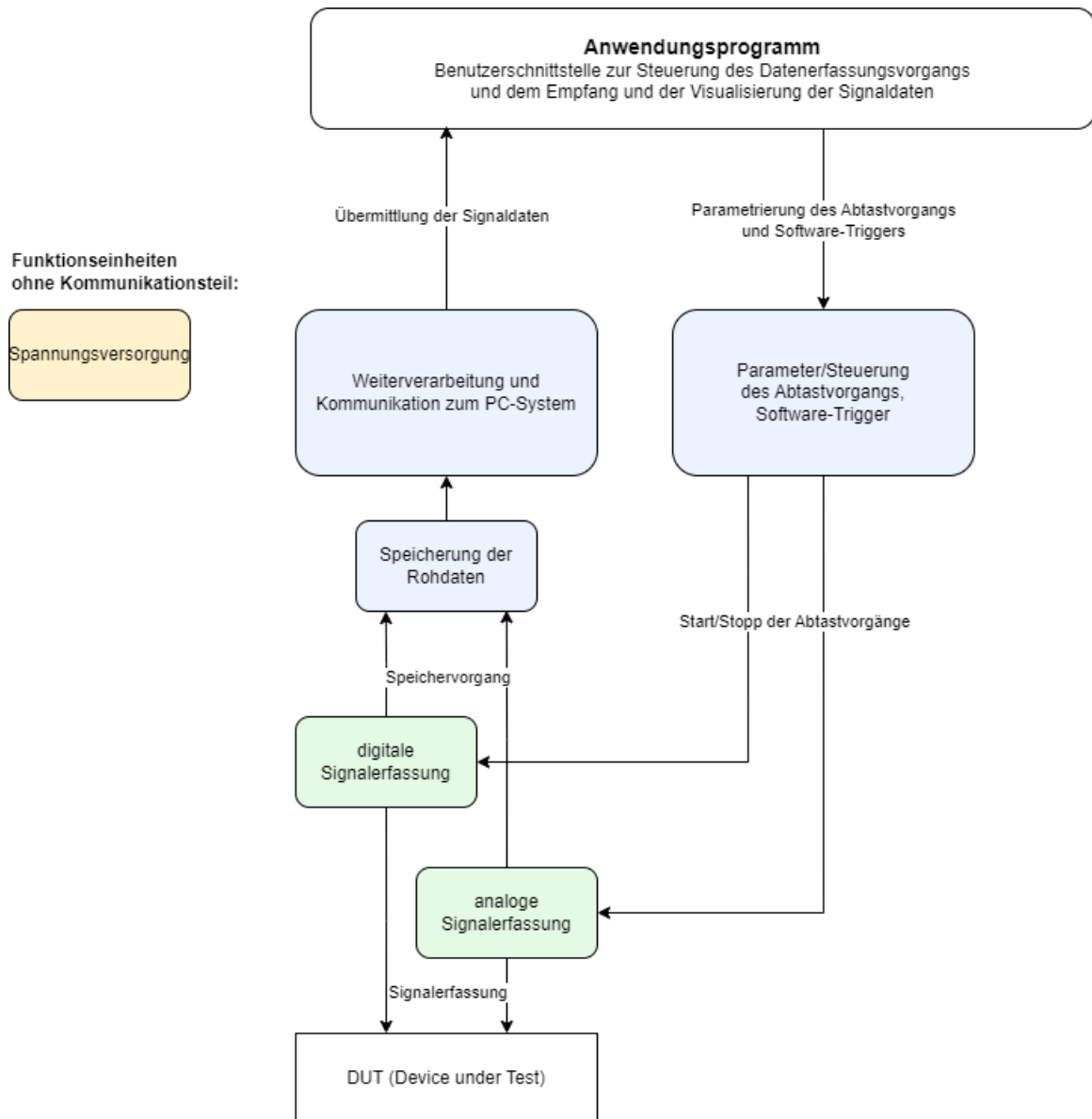


Abbildung 58: Übersichts-Blockschaltbild (erstellt für die Konzeptvorstellung)